Enlangg

The Enlng Book

A reference manual for the Enlangg language and its seven tools. Every program in this book was run with toolchain 5.6.0, and the output shown is what it printed.

github.com/Aero99op/enlang-main · MIT License

Contents

1. What Enlangg Is 4

2. Installing Enlangg 6

3. Your First Program 9

4. The Seven Tools 12

5. Running and Building Programs 16

6. Values and Names 20

7. Numbers and Arithmetic 26

8. Text 32

9. Comparisons and Logic 39

10. Making Decisions 42

11. Loops 46

12. Lists 51

13. Maps 57

14. Reaching Inside Lists, Maps and Text 62

15. Functions 65

16. Recursion and Scope 72

17. The Type Line and Natural Phrasing 78

18. Sorting, Swapping and Arranging 81

19. Input, Files and Time 85

20. Splitting a Program into Files 89

21. Reading Error Messages 92

22. Built-in Function Reference 100

23. The Standard Library: Overview, lib_std and lib_math 106

24. The Standard Library: lib_strings 113

25. The Standard Library: lib_ds (Data Structures) 120

26. The Standard Library: lib_file and lib_fs (Files) 129

27. The Standard Library: lib_db (A Small Table Store) 134

28. The Standard Library: lib_ml and lib_net 139

29. The Standard Library: lib_concurrency and lib_enlngdb 144

30. EnlngDB: The English Database Language 148

31. enlngd and enlngf: Styles and Web Pages 154

32. enlngs and enlngm: Scripts and Mobile Previews 158

33. The enlangg Command, fmt and pkg 161

34. Cookbook 1: Numbers 164

35. Cookbook 2: Text 172

36. Cookbook 3: Lists and Maps 180

37. Cookbook 4: Searching, Sorting and Recursion 188

38. Cookbook 5: Small Programs 196

39. Limits and Known Gaps in 5.5.1 202

40. Reference: Words of the Language 208

41. Reference: Every Command 212

42. Glossary 217

43. Classes and Objects 220

44. Everyday Tools: Defaults, Text Templates and List Helpers 225

45. Plain Sentences: Choosing, Taking Apart, Tests, Doing Things Together and the Network 229

46. Packages, the Debugger and Editor Help 236

CHAPTER 1

What Enlangg Is

Enlangg is a programming language whose programs read like short English sentences. A complete program can be as small as this:

PROGRAM

```
type enlng
remember price as 40
remember quantity as 3
show "Total:" price times quantity
```

OUTPUT

```
Total: 120
```

Nothing here needs braces, semicolons or type declarations. The words `remember`, `as`, `show` and `times` are ordinary words the language knows. This book teaches those words, the rules for putting them together, and the tools that run the result.

What you get when you install it

Enlangg is not one program but a toolchain of seven small programs, all written in C:

Tool	What it is for
`enlng`	The language itself: runs and builds `.enlng` programs.
`enlngdb`	A small database. You write statements in English instead of SQL.
`enlngf`	Describes web pages in English and generates HTML.
`enlngd`	Describes colours, spacing and styles in English and generates CSS.
`enlngs`	Describes page behaviour in English and generates JavaScript.
`enlngm`	Describes phone screens in English and previews them.
`enlangg`	A front door: looks at a file name and starts the right tool.

The first four chapters after this one cover installing and running. Part II is the language reference. Part III is a cookbook of complete programs. Part IV covers the other tools. Part V is reference tables.

How a program runs

When you type `enlng run program.enlng`, the tool reads your text, checks it against the grammar, and builds a tree of what you meant. There are two ways to carry out that tree:

- **The built-in interpreter** walks the tree and does what it says. This is the default. It needs nothing else installed.
- **The C backend** turns the same tree into C source code and asks a C compiler (gcc, clang or cc) to build an executable. You choose it with `enlng build` or `enlng run --native`.

Both ways share one runtime, so a program behaves the same either way. Chapter 5 shows how to use each and what the one known difference is.

What this book does not claim

You will not find performance tables against other languages in this book, because none have been measured properly. The only timing the project has published is a single start-up measurement on one machine, given in chapter 5. When a feature does not exist yet, the book says so in the chapter on limits (chapter 39) instead of describing it as if it did.

How the examples were made

Every program in this book was run with toolchain 5.5.1 while the book was built. The grey box is the program and the green box is exactly what the tool printed. If you type the same program you should see the same output.

CHAPTER 2

Installing Enlangg

There are two installers. Neither is code-signed, so Windows SmartScreen may warn you before the Windows installer starts. Check the SHA-256 hash on the download page against the file you got before you run it.

Windows

Download `enlangg-setup.exe` from the install page and run it. It installs for your user account only, so it never asks for administrator rights.

By default the files go to `%LOCALAPPDATA%\Programs\Enlangg`. The installer:

- copies the seven tools into a `bin` folder and the standard library into `lib`;
- adds the `bin` folder to your user `PATH` (you can untick this);
- registers the `.enlng` file type (you can untick this) so that double-clicking a file opens it instead of running it;
- optionally downloads Enlangg Studio, the IDE, and checks its SHA-256 before starting its installer.

It writes a list of everything it installed to `install.manifest`, and `uninstall.exe` removes exactly that list. A file you added to the install folder yourself is left alone.

For scripted installs the same program takes options:

Option	Meaning
`/S`	Silent: no window.
`/DIR=<path>`	Install here. The path must be absolute and must not be a drive root.
`/NOPATH`	Do not change `PATH`.
`/NOASSOC`	Do not register the `.enlng` file type.
`/STUDIO`	Also download and install Enlangg Studio.
`/LOG=<file>`	Write a log of what the installer did.

To uninstall silently, run `uninstall.exe /uninstall /S` from the install folder.

Linux and macOS

Download `enlangg-install.sh`. It is one file that contains a shell script followed by the program archives for Linux x86_64, Linux aarch64 and macOS. Run it with `sh`:

TERMINAL

```
sh enlangg-install.sh
```

It checks each archive's SHA-256 before using it, installs into `~/.enlangg` without needing root, and adds `~/.enlangg/bin` to your shell startup files between two marker lines (`# >>> enlangg >>>` and `# <<< enlangg <<<`) so it can be removed again cleanly.

Option	Meaning
`--prefix DIR`	Install into `DIR` instead of `~/.enlangg`.
`--no-path`	Do not touch your shell startup files.
`--uninstall`	Remove an existing install.
`--help`	Print the options.

To uninstall later, run `sh ~/.enlangg/uninstall.sh`. It removes only what is listed in `install.manifest` and the marked lines in your startup files.

The installer has to be run as a file. Piping it into `sh` (`curl ... | sh`) is not supported, because it needs to read its own archives.

Do I need a C compiler?

Not to run programs. `enlng run file.enlng` uses the built-in interpreter. A C compiler is needed only for two things: `enlng build` (to produce an executable) and `enlng run --native`. If you ask for either and no compiler is found, the tool says so and suggests `enlng run` instead.

Checking that it worked

Open a new terminal, so that the changed `PATH` is picked up, and ask each tool for its version:

TERMINAL

```
$ enlng --version
enlng version 5.6.0 (Native C99)
$ enlangg --version
enlangg version 5.0.0-sovereign-universal (Sovereign General Purpo
    se)
```

If the command is not found, the `bin` folder is not on your `PATH`. Open a new terminal first; if it is still missing, add the folder by hand or run the installer again without the no-PATH option.

CHAPTER 3

Your First Program

Create a text file named `hello.enlng` and put this in it:

PROGRAM

```
type enlng

show "Hello from Enlangg"
```

OUTPUT

```
Hello from Enlangg
```

Run it from a terminal in the same folder:

TERMINAL

```
enlng run hello.enlng
```

The parts of the file

- `type enlng` on the first line says which language this file is written in. Each of the six file types has its own tool and its own word list, and this line stops a file from being read by the wrong one. Chapter 18 returns to it.
- A blank line is only for you. Blank lines are ignored.
- `show` prints the values after it, followed by a new line.
- Text goes in double quotes. Numbers do not.

Adding a name

A name holds a value so you can use it again. You create one with `remember`:

PROGRAM

```
type enlng

remember city as "Pune"
remember year as 2026
show "City:" city
show "Year:" year
```

OUTPUT

```
City: Pune
Year: 2026
```

Notice that `show` with several values separates them with a space and that a string already ending in a space keeps it. The words after `show` are evaluated in

order and printed together.

Comments

A line that starts with `#` is a comment. The tool skips it. Use comments to explain why, not what:

PROGRAM

```
type enlng

# Tax is 18 percent in this example.
remember rate as 18
remember price as 200
show "Tax:" price times rate divided by 100
```

OUTPUT

```
Tax: 36
```

Changing a value

Once a name exists you can change it with `change`, or with an increase or decrease:

PROGRAM

```
type enlng

remember score as 10
score increases by 5
show score
score decreases by 3
show score
change score to 100
show score
```

OUTPUT

```
15
12
100
```

Reading input

`ask` reads a line the user types and stores it under the name you give. Because this book's programs run without a keyboard, the next example is run with the text `Asha` supplied as input:

PROGRAM

```
type enlng

ask name
```

```
show "Hello," name
```

OUTPUT

```
Hello, Asha
```

What happens when you make a mistake

Enlangg checks the whole file before running any of it. If a line does not follow the grammar you get a message that points at the line, shows it, and says what was expected:

PROGRAM WITH A MISTAKE

```
type enlng

show "before"
remember as 5
show "after"
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ---------------------------------------------
    -------
Location: program.enlng:4

    3 | show "before"
    4 | remember as 5
      | ^
    5 | show "after"

What: Expected variable name after remember/freeze
Why:  The compiler encountered an unexpected token during syntax e
    valuation.

Suggestions:
  * Check statement syntax against Enlang language conventions.
----------------------------------------------------------------
    ------
```

Nothing was printed, not even `before`, because the file never started running. Chapter 21 lists the error messages you are likely to meet.

CHAPTER 4

The Seven Tools

Installing Enlangg puts seven programs on your machine. Six of them each understand one kind of file, and the seventh, `enlangg`, picks the right one for you.

File type	Tool	What the file describes	What comes out
`.enlng`	`enlng`	A program: logic, maths, text, files	Output on the screen, or a C source file or executable
`.enlngdb`	`enlngdb`	Tables and rows	Query results; data saved to a file
`.enlngf`	`enlngf`	A web page	HTML
`.enlngd`	`enlngd`	Colours, spacing, styles	CSS
`.enlngs`	`enlngs`	What a page does when clicked or typed into	JavaScript
`.enlngm`	`enlngm`	A phone screen	A preview window (Windows only)

Native programs, generated files

All seven tools are native programs written in C. They start quickly, and you do not need Python, Node.js or a runtime installed to use them.

It is worth being precise about what the last three of them produce, though. `enlngf`, `enlngd` and `enlngs` are translators: the tool is native, but its output is HTML, CSS and JavaScript text, which a browser then runs. They do not produce machine code. Only `enlng` can build a native executable, and it does so by way of C and a C compiler.

The front door: enlangg

`enlangg` looks at the extension of the file you give it and starts the matching tool, so you can use one command for everything:

TERMINAL

```
$ enlangg --help
=================================================================
    ===
  Enlangg Sovereign Toolchain CLI v5.0.0-sovereign-universal
  Pure C Zero-Python 7-in-1 Sovereign Ecosystem
=================================================================
    ===

Unified CLI Commands:
  enlangg run <file.ext>                      Run any file (.enlng
    , .enlngdb, .enlngs, .enlngf, .enlngd, .enlngm)
  enlangg run --aot <file.enlng>              Compile to native ma
    chine code & execute instantly
  enlangg compile <file.enlng> [-o <out.exe>] AOT compile natural
    English to native C machine code (.exe)
  enlangg emit-c <file.enlng> [-o <out.c>]    Emit clean ISO C99 s
    ource with Scoped Arena Memory

Domain Engines (Native Pure C):
  enlangg db <script.enlngdb>                 Execute conversation
    al microsecond database script
  enlangg db -e "<query>"                     Execute inline datab
    ase statement in C (<0.05ms)
  enlangg s <logic.enlngs>                    Execute script nativ
    ely on Pure C In-Memory VM
  enlangg s -e "<script>"                     Execute inline scrip
    t statement in C VM
  enlangg f <app.enlngf>                      Launch Pure C Native
     Win32 Desktop Window GUI
  enlangg f serve <app.enlngf> [--port 3000]  Serve live over Pure
     C WinSock2 HTTP Server
  enlangg f build <app.enlngf> [-o <out.html>] Compile frontend ma
    rkup to standalone HTML5
  enlangg d <theme.enlngd>                    Inspect and resolve
    design tokens & styles
  enlangg d compile <theme.enlngd> [-o <.css>] Export design token
    s to CSS3 stylesheet
  enlangg m <app.enlngm>                      Launch Pure C Native
     Smartphone Simulator (390x844)
  enlangg m build <app.enlngm> --target apk   Build mobile product
    ion package (HAL ARM64)
  enlangg build <app.enlngm> --target apk     Direct shortcut to m
    obile production compiler

Dedicated Standalone Executables in PATH:
  enlangg   Universal master toolchain dispatcher

  enlng     Core general-purpose computing with arena memory (.exe
    )
  enlngdb   Pure C conversational zero-SQL database engine (.edb f
    ormat)
  enlngs    Pure C sovereign in-memory script VM & interpreter
```

```
  enlngf    Pure C sovereign desktop window GUI & WinSock2 web stu
    dio
  enlngd    Pure C sovereign design tokens & style resolver
  enlngm    Pure C sovereign smartphone simulator & HAL packager

Documentation & Live Playground: https://enlangg.vercel.app
```

That is the help text of version 5.5.1, printed as the tool wrote it. A few of its descriptions are more enthusiastic than they are exact. The rest of this book describes what each command really does.

Where `enlangg` looks for the other tools

`enlangg` first looks for the tool it needs in its own folder, the one it was started from. Only if the tool is not there does it search your `PATH`. This means two things. A copy of a tool earlier on your `PATH` cannot take over from the one that was installed with `enlangg`, and a folder you copy somewhere else keeps working as long as the seven files stay together. The tool is never taken from the current directory, so a file named `enlng.exe` that happens to sit in a project folder is not run by mistake.

Version numbers

Each tool reports the same version:

TERMINAL

```
$ enlng --version
enlng version 5.6.0 (Native C99)
$ enlngdb --version
enlngdb version 2.0.0-pure-c-native (Pure C Microsecond Database E
    ngine)
$ enlngf --version
enlngf version 5.3.0 (Pure C Sovereign Frontend Engine - ZERO PYTH
    ON)
$ enlngd --version
enlngd version 5.3.0 (Pure C Sovereign Design Tokens Engine - ZERO
     PYTHON)
$ enlngs --version
enlngs version 5.3.0 (Pure C Sovereign Reactive Script Engine - ZE
    RO PYTHON)
$ enlngm --version
enlngm version 5.3.0 (Pure C Sovereign Mobile Native Engine - ZERO
     PYTHON)
```

The descriptions in parentheses are the tools' own tag lines. Some of them still say things like "ZERO PYTHON", which is true but is not a feature description; treat the tag lines as labels.

Studio

Enlangg Studio is a separate download: a desktop editor for Windows that bundles the same seven tools. Everything in this book works from the terminal without it.

CHAPTER 5

Running and Building Programs

`enlng` has four ways to treat a program. You have already used the first.

Command	What it does	Needs a C compiler?
`enlng run file.enlng`	Runs the program with the built-in interpreter.	No
`enlng file.enlng`	Short form of the above.	No
`enlng run --native file.enlng`	Translates to C, builds with gcc, then runs.	Yes
`enlng build file.enlng -o out`	Translates to C and builds a standalone executable.	Yes
`enlng emit-c file.enlng`	Prints the generated C source.	No

The interpreter

The interpreter reads your whole file first, so syntax mistakes are reported before anything runs. Then it walks the program and carries out each statement.

Two limits protect you. A function that calls itself without end stops with a clear error instead of crashing the tool, and the interpreter runs on a large stack so that ordinary deep recursion, such as a recursive sum to ten thousand, works:

PROGRAM

```
type enlng

function sum_to with n:
    when n is 0:
        give 0
    give n plus (sum_to with n minus 1)

show sum_to with 10000
```

OUTPUT

```
50005000
```

Runaway recursion stops at a fixed depth and prints an error:

PROGRAM WITH A MISTAKE

```
type enlng

function forever with n:
    give forever with n plus 1

show forever with 0
```

WHAT enlng PRINTS

```
-- Enlang RuntimeError ----------------------------------------
    -------
Location: program.enlng:4

    4 |     give forever with n plus 1

What: 'forever' called itself too many times (over 20000 levels de
    ep). Check that it has a way to stop.
---------------------------------------------------------------
    ------
```

Building an executable

With gcc, clang or cc on your `PATH`, `enlng build` produces a program you can give to someone who does not have Enlangg installed:

TERMINAL

```
enlng build hello.enlng -o hello
./hello
```

Seeing the C

`enlng emit-c` prints what the C backend would compile. It is useful for curiosity and for checking what a program means. This is the whole of a one-line program:

TERMINAL

```
$ enlng emit-c hello.enlng
/* ================================================================
    ====== */
/*  NATIVE C99 EXECUTABLE - COMPILED BY ENLANG SOVEREIGN COMPILER
    v5.6.0 */
/* ================================================================
    ====== */

#include "enlng_runtime.h"

static EnlngVal enlng_call_value(EnlngVal f, const char *name, int
     argc, EnlngVal *a);
```

```
static EnlngVal enlng_call_value(EnlngVal f, const char *name, int
     argc, EnlngVal *a) {
    if (f.type != ENLNG_VAL_FUNC) {
        enlng_not_a_function(name, f);
        return enlng_make_null();
    }
    if (f.as.fn->builtin)
        return enlng_call_builtin_value(f.as.fn->name, argc, a);
    (void)a;
    return enlng_make_null();
}

static EnlngVal enlng_hook_call(EnlngVal f, int argc, EnlngVal *a)
     {
    return enlng_call_value(f, "function", argc, a);
}

int main(int argc, char* argv[]) {
    enlng_console_utf8();
    enlng_call_hook = enlng_hook_call;
    enlng_set_args(argc - 1, argv + 1);
    enlng_print_val(enlng_make_string("hi")); printf("\n");
    return 0;
}
```

The banner comment in that output still carries an old version number. It is cosmetic.

When there is no C compiler

If you ask for `--native` or `build` and no compiler can be found, the tool tells you, and `enlng run` still works. This is the output with an empty `PATH`:

TERMINAL

```
enlng run --native hello.enlng
[ENLANG ERROR] Native C compilation failed. Ensure a C compiler (g
    cc) is installed and on PATH.
```

Are the two ways the same?

They share one runtime, and the project's tests run the same programs through both and compare the output. They agree on every test program, with one known difference: the C path prints a decimal number literal with six places, where the interpreter prints it the way the runtime formats numbers. If you need exact text output of decimals, format the number yourself.

Start-up time

The only timing the project has measured is how long a hello-world takes from start to exit. On one Windows machine, with release 5.5.0, the median of 30 runs was about 12 ms for `enlng run hello.enlng` and about 73 ms for `python hello.py`. That is start-up only. The interpreter is not tuned for heavy loops, and for those `--native` is faster. No broader comparison has been done, so none is claimed.

CHAPTER 6

Values and Names

A program works with values. A **value** is a piece of data such as the number `42`, the text `"Pune"`, or the list `[1, 2, 3]`. A **name** is a label you attach to a value so you can use it later.

The kinds of value

Kind	Examples	Notes
Whole number	`0`, `42`, `-7`	Any size: past 64 bits every digit is kept.
Decimal number	`3.14`, `0.5`	Printed with up to six significant digits.
Text	`"hello"`, `""`	Double quotes. Position counts from 0.
True or false	`true`, `false`	Written in lower case.
Nothing	`null`	Also what you get for a missing key or position.
List	`[1, 2, 3]`	Ordered. Can mix kinds.
Map	`{"a": 1}`	Keys are text. The order keys print in is not the order you wrote them.

PROGRAM

```
type enlng

remember count as 42
remember pi as 3.14159
remember word as "hello"
remember ready as true
remember nothing_yet as null
remember numbers as [1, 2, 3]
remember person as {"name": "Asha", "age": 31}

show count
show pi
show word
show ready
show nothing_yet
show numbers
show person
```

OUTPUT

```
42
3.14159
hello
true
null
[1, 2, 3]
{"age": 31, "name": Asha}
```

Making a name

`remember name as value` creates a name. A plain `name = value` does the same thing:

PROGRAM

```
type enlng

remember a as 1
b = 2
show a b
```

OUTPUT

```
1 2
```

Names are checked

Using a name that was never created is an error that stops the program, and the message names the line:

PROGRAM WITH A MISTAKE

```
type enlng

show "start"
show missing_name
```

WHAT enlng PRINTS

```
start

-- Enlang RuntimeError ----------------------------------------
    -------
Location: program.enlng:4

    4 | show missing_name

What: I don't know a name 'missing_name' here. Define it first, fo
    r example: remember missing_name as 0
------------------------------------------------------------------
    ------
```

Note that `start` was printed before the error: this one is found while running, not while reading the file.

Changing a name

Assigning to an existing name replaces its value. Three more forms exist for the most common changes:

PROGRAM

```
type enlng

remember n as 10
n = 20
show n
n increases by 5
show n
n decreases by 8
show n
change n to 1
show n
```

OUTPUT

```
20
25
17
1
```

Assigning to a name that does not exist yet simply creates it, so a mistyped name in an assignment does not stop the program. Spell names carefully, and use `remember` for the first assignment so that your intent is clear to the reader.

Constants with `freeze`

`freeze NAME as value` declares a name meant to stay constant, and it reads that way:

PROGRAM

```
type enlng

freeze TAX_RATE as 18
show TAX_RATE
```

OUTPUT

```
18
```

A frozen name cannot be assigned to again. The tool refuses the program when it reads the file, before anything runs, and says why:

PROGRAM WITH A MISTAKE

```
type enlng

freeze LIMIT as 100
change LIMIT to 200
```

```
show LIMIT
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ------------------------------------------
    -------
Location: program.enlng:4

    3 | freeze LIMIT as 100
    4 | change LIMIT to 200
      | ^
    5 | show LIMIT

What: 'LIMIT' was declared with freeze, so it cannot be changed. U
    se remember instead if the value needs to change.
Why:  A name made with 'freeze' is a constant: the name may not be
     assigned again, and a list or map it holds may not be changed
    .

Suggestions:
  * Declare it with 'remember' if the value has to change.
  * Use a different name for the new value.
------------------------------------------------------------------
    ------
```

A frozen list or map is frozen as a whole: its items cannot be added, removed, assigned, sorted or swapped either. A change written straight on the name is refused when the file is read:

PROGRAM WITH A MISTAKE

```
type enlng

freeze ITEMS as [1, 2]
append(ITEMS, 3)
show ITEMS
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ------------------------------------------
    -------
Location: program.enlng:4

    3 | freeze ITEMS as [1, 2]
    4 | append(ITEMS, 3)
      | ^
    5 | show ITEMS

What: 'ITEMS' was declared with freeze, so its contents cannot be
    changed either. Use remember instead if the list or map needs
    to change.
Why:  A name made with 'freeze' is a constant: the name may not be
     assigned again, and a list or map it holds may not be changed
    .
```

```
Suggestions:
  * Declare it with 'remember' if the value has to change.
  * Use a different name for the new value.
-----------------------------------------------------------------
    ------
```

It stays frozen wherever it goes. If you hand a frozen list to a function and the function tries to change it, the program stops at that moment with a message, and the list keeps its old contents:

PROGRAM WITH A MISTAKE

```
type enlng

function add_one with xs:
    append(xs, 9)
    give xs

freeze ITEMS as [1, 2]
show add_one(ITEMS)
```

WHAT enlng PRINTS

```
-- Enlang RuntimeError ----------------------------------------
    -------
Location: program.enlng:4

    4 |     append(xs, 9)

What: This list or map was made with 'freeze', so it cannot be cha
    nged, not even inside a function. Use 'remember' if it needs t
    o change.
-----------------------------------------------------------------
    ------
```

Reading is always allowed: `show`, `len`, `for each`, `join` and indexing all work on a frozen list. To get a changeable version, build a new list from it with a loop. (Only the list or map itself is frozen; a list stored inside a frozen list is a separate value and can still change.)

A function has names of its own, so inside a function the same spelling can be used for something else. In toolchain 5.5.0 and earlier, `freeze` was accepted but not enforced and the LIMIT program above printed 200. From 5.5.1 it refuses to assign to the name, and from 5.5.2 it also protects what the name holds.

Name rules

A name is made of letters, digits and underscores and does not start with a digit. A name cannot be one of the language's own words (see the keyword table in the reference part). Names are case-sensitive: `Total` and `total` are

different.

Pick names that read well in a sentence. `show total price` is better than `show tp`.

CHAPTER 7

Numbers and Arithmetic

Two ways to write an operation

Every arithmetic operator has a symbol and an English word. They mean the same thing and you can mix them in one program.

Meaning	Symbol	Words
add	`+`	`plus`
subtract	`-`	`minus`
multiply	`*`	`times` or `multiplied by`
divide	`/`	`divided by`
remainder	`%`	`mod` or `modulo`

PROGRAM

```
type enlng

show 6 plus 4
show 6 + 4
show 6 minus 4
show 6 times 4
show 6 multiplied by 4
show 6 divided by 4
show 6 mod 4
show 6 % 4
```

OUTPUT

```
10
10
2
24
24
1
2
2
```

Whole numbers divide to whole numbers

When both sides of a division are whole numbers, the result is a whole number with the fraction dropped. Make one side a decimal to keep the fraction:

PROGRAM

```
type enlng

show 7 divided by 2
show 7.0 divided by 2
show 7 divided by 2.0
show 1 divided by 3
show 1 divided by 3.0
```

OUTPUT

```
3
3.5
3.5
0
0.333333
```

This is the most common surprise for new programmers, so it is worth saying plainly: `7 divided by 2` is `3`, not `3.5`.

Dividing by zero

From 5.5.2, dividing by zero is an error, for `divided by` and for `mod` alike. A `try` block catches it (chapter 21), so the program can decide what to do:

PROGRAM

```
type enlng

try:
    show 5 divided by 0
on error as problem:
    show "caught:" problem
show "still running"
```

OUTPUT

```
caught: Cannot divide by zero: 5 divided by 0.
still running
```

Toolchain 5.5.1 and earlier gave `0` here without any message. If a zero divisor is possible in your program, test for it with `when`, or wrap the division in `try`.

The same goes for arithmetic on values that are not numbers. `"a" minus 1`, `[1, 2] times 3` and `null plus 1` are errors that name both kinds of value, such as "'minus' needs two numbers, but got text and a number". Two exceptions are on purpose: `plus` with text joins the two as text, and text `times` a whole number repeats the text.

Order of operations

Multiplication, division and remainder bind tighter than addition and subtraction, as in school arithmetic. Use brackets to be explicit:

PROGRAM

```
type enlng

show 2 plus 3 times 4
show (2 plus 3) times 4
show 10 minus 4 minus 3
show 10 minus (4 minus 3)
```

OUTPUT

```
14
20
3
9
```

Decimals

Decimal numbers print with up to six significant digits, so tiny rounding differences do not show up in ordinary output:

PROGRAM

```
type enlng

show 0.1 plus 0.2
show 1 divided by 3.0
show 2 times 3.5
show 22 divided by 7.0
```

OUTPUT

```
0.3
0.333333
7
3.14286
```

Big whole numbers

A whole number has no size limit. Past 9223372036854775807, the largest that fits in 64 bits, it keeps every digit, so the answer is always exact:

PROGRAM

```
type enlng

show 9223372036854775807 plus 1
show 2 to the power of 100
remember total as 1
for i from 1 to 30:
    total = total times i
show total
```

```
show 12345678901234567890123456789O mod 7
```

OUTPUT

```
9223372036854775808
126765060022822940149670320537 6
265252859812191058636308480000000
0
```

A number this big can also be written in the program, or read from text with `to_number`. Plus, minus, times, `divided by`, `mod`, power, comparing, `show`, f-strings and JSON all work with it, and a result that is small again becomes an ordinary whole number. A big number cannot be used as a position in a list or as a count; that is an error. A power with more than a million digits is also an error, because it would take too long to work out.

Remainders and negative numbers

The remainder takes the sign of the left side:

PROGRAM

```
type enlng

show 7 mod 3
show -7 mod 3
show 7 mod -3
```

OUTPUT

```
1
-1
1
```

Built-in maths functions

Functions are called with brackets, like `sqrt(16)`. These are available without importing anything:

Function	Result
`sqrt(x)`	Square root.
`pow(x, y)`	`x` to the power `y`.
`abs(x)`	Absolute value.
`floor(x)`	Round down.
`ceil(x)`	Round up.
`round(x)`	Round to nearest.
`min(a, b)`	The smaller of two values.

Function	Result
`max(a, b)`	The larger of two values.
`random_number(low, high)`	A random whole number between the two.

PROGRAM

```
type enlng

show sqrt(144)
show sqrt(2)
show pow(2, 10)
show abs(-3.5)
show floor(3.7)
show ceil(3.2)
show round(2.5)
show round(2.4)
show min(3, 9)
show max(3, 9)
```

OUTPUT

```
12
1.41421
1024
3.5
3
4
3
2
3
9
```

A worked example: a shop bill

PROGRAM

```
type enlng

remember price as 249
remember quantity as 3
remember discount_percent as 10

remember subtotal as price times quantity
remember discount as subtotal times discount_percent divided by 10
    0
remember total as subtotal minus discount

show "Subtotal:" subtotal
show "Discount:" discount
show "Total:" total
```

OUTPUT

```
Subtotal: 747
Discount: 74
Total: 673
```

The discount is exact here because `747 times 10 divided by 100` happens left to right on whole numbers: 7470 divided by 100 is 74, with the .7 dropped. If you need the cents, use a decimal such as `10.0`.

PROGRAM

```
type enlng

remember subtotal as 747
remember discount as subtotal times 10.0 divided by 100
show "Discount:" discount
show "Total:" subtotal minus discount
```

OUTPUT

```
Discount: 74.7
Total: 672.3
```

CHAPTER 8

Text

Text values are written in double quotes. A string can be empty (`""`) and can hold any characters.

Joining text

`+` joins two pieces of text. If one side is a number it is turned into text first. Listing several values after `show` also prints them with a space between, which is often simpler:

PROGRAM

```
type enlng

remember first as "Ada"
remember last as "Lovelace"
show first + " " + last
show first last
show "Total: " + 42
show "Count" 3 "items"
```

OUTPUT

```
Ada Lovelace
Ada Lovelace
Total: 42
Count 3 items
```

Length

`length of` and `len(...)` give the number of characters:

PROGRAM

```
type enlng

remember word as "enlangg"
show length of word
show len(word)
show len("")
```

OUTPUT

```
7
7
0
```

From 5.5.2 text is counted in characters, also for letters outside English, such as é, Hindi or emoji. Positions, slices, `split(text, "")` and `for each` all work on whole characters:

PROGRAM

```
type enlng

remember word as "café"
show length of word
show word[3]
show word[0:3]
```

OUTPUT

```
4
é
caf
```

Earlier toolchains counted bytes, so the same word had a length of 5 and `word[3]` gave half of the é.

Text and numbers

`to_number` turns text into a number and `to_text` turns any value into text, the way `show` prints it:

PROGRAM

```
type enlng

show to_number("42") plus 1
show to_number("2.5") times 2
show to_text(42) + "!"
show to_text([1, 2])
```

OUTPUT

```
43
5
42!
[1, 2]
```

Text that is not a number, such as `to_number("12abc")`, is an error that `try` can catch.

Picking characters

A character is picked by its position, counting from zero. A negative position counts from the end:

PROGRAM

```
type enlng
```

```
remember word as "enlangg"
show word[0]
show word[3]
show word[-1]
show word at 2
```

OUTPUT

```
e
a
g
l
```

Taking a piece

From 5.5.2, `word[a:b]` gives the characters from position `a` up to, but not including, `b`. Leave out `a` to start at the beginning, or `b` to go to the end. A negative position counts from the end, and a position past either end is treated as the end, so a slice never fails:

PROGRAM

```
type enlng

remember word as "enlangg"
show word[0:3]
show word[2:]
show word[:-1]
show word[-3:]
show word[5:100]
```

OUTPUT

```
enl
langg
enlang
ngg
gg
```

A slice is a new text; the original does not change. The same form works on lists (chapter 12).

`slc` does the same as a function: `slc(word, 0, 3)` is `word[0:3]`, and `slc(word, 2)` is `word[2:]`. Use whichever reads better, for example when the value comes from a call:

PROGRAM

```
type enlng

show slc("enlangg", 0, 3)
show slc(split("a b c d", " "), 1)
```

OUTPUT

```
enl
[b, c, d]
```

`slc` is a word of the language, so it cannot be the name of a value or a function.

Searching

`contains(text, piece)` is true when `piece` appears anywhere in `text`:

PROGRAM

```
type enlng

remember sentence as "the quick brown fox"
show contains(sentence, "quick")
show contains(sentence, "slow")
```

OUTPUT

```
true
false
```

Splitting and joining

`split(text, separator)` breaks text into a list. `join(list, separator)` does the reverse. Splitting on an empty separator gives one item per character. Empty pieces are kept, so `split("a,,b", ",")` has three items and the middle one is empty. A separator of several characters is matched as a whole, so `split("a--b", "--")` has two items. (Before 5.5.1 empty pieces were dropped and every character of the separator was treated as a separator on its own.)

PROGRAM

```
type enlng

remember csv as "red,green,blue"
remember parts as split(csv, ",")
show parts
show len(parts)
show join(parts, " | ")
show split("abc", "")
show split("a,,b", ",")
show split("a--b", "--")
show split("-a-", "-")
```

OUTPUT

```
[red, green, blue]
3
red | green | blue
[a, b, c]
[a, , b]
[a, b]
[, a, ]
```

Reversing

`reverse` works on text and on lists:

PROGRAM

```
type enlng

show reverse("stressed")
```

OUTPUT

```
desserts
```

Comparing

Text compares with `==` or `is`, and orders alphabetically with `<` and `>`:

PROGRAM

```
type enlng

show "abc" == "abc"
show "abc" is "abd"
show "apple" < "banana"
show "b" > "a"
```

OUTPUT

```
true
false
true
true
```

Comparisons are case-sensitive: `"Apple"` and `"apple"` are different, and in the ordering every capital letter comes before every small letter.

Changing one character

Text values do not change in place. These functions give back a new text:

Function	Result
`string_set_at(text, position, character)`	The text with one character replaced.
`string_remove_at(text, position)`	The text without the character at that position.
`string_remove(text, piece)`	The text with the first occurrence of `piece` removed.

PROGRAM

```
type enlng
```

```
remember word as "hello"
show string_set_at(word, 0, "J")
show string_remove_at(word, 1)
show string_remove(word, "l")
show word
```

OUTPUT

```
Jello
hllo
helo
hello
```

`word` itself is unchanged; assign the result if you want to keep it: `word = string_set_at(word, 0, "J")`.

Three more built-ins work by position, and they take the text first, then the piece, then the position:

Function	Result
`string_add(text, piece, position)`	The text with `piece` put in at that position. `string_insert` does the same.
`string_replace(text, piece, position)`	The text with the one character at that position replaced by `piece`.

PROGRAM

```
type enlng

show string_add("hello", "XY", 2)
show string_replace("hello", "J", 1)
```

OUTPUT

```
heXYllo
hJllo
```

Note that `string_replace` works on a position, not on a word. To replace a word wherever it appears, use `string_replace_first` or `string_replace_all` from `lib_strings` (chapter 24).

These three also accept the text and the piece in either order, and they decide which is which by comparing lengths. In normal use that is invisible. It only shows when the text is shorter than the piece and the position lies beyond the text, so write them in the order above and you will not meet it.

Counting letters: a small program

PROGRAM

```
type enlng
```

```
remember text as "mississippi"
remember count_s as 0
for each ch in split(text, ""):
    when ch is "s":
        count_s increases by 1
show "Letter s appears" count_s "times in" text
```

OUTPUT

```
Letter s appears 4 times in mississippi
```

CHAPTER 9

Comparisons and Logic

A comparison asks a question about two values and answers `true` or `false`. These answers are what decisions and loops are built on.

Comparing

Each comparison has a symbol form and an English form:

Question	Symbols	Words
equal?	`==`	`is`, `is equal to`
not equal?	`!=`	`is not`
less than?	`<`	`is less than`
greater than?	`>`	`is greater than`
at most?	`<=`	`is at most`
at least?	`>=`	`is at least`

PROGRAM

```
type enlng

remember age as 21
show age == 21
show age is 21
show age is not 18
show age is less than 30
show age is greater than 30
show age is at least 21
show age is at most 20
show age >= 18
```

OUTPUT

```
true
true
true
true
false
true
false
true
```

Numbers compare by value

A whole number and a decimal that hold the same value are equal:

PROGRAM

```
type enlng

show 3 == 3.0
show 2 < 2.5
```

OUTPUT

```
true
true
```

Text, true/false and null

Text compares exactly, so capital letters matter. `null` can be compared like any other value, which is how you test for a missing key or position:

PROGRAM

```
type enlng

remember scores as {"asha": 90}
show "Hello" == "hello"
show scores["asha"] == null
show scores["ravi"] == null
```

OUTPUT

```
false
false
true
```

Lists are not compared by content

Comparing two lists with `==` asks whether they are the same list, not whether they hold the same items. Two separately written lists with identical contents are not equal:

PROGRAM

```
type enlng

remember a as [1, 2]
remember b as [1, 2]
show a == b
show a == a
```

OUTPUT

```
true
true
```

To compare contents, compare item by item in a loop (the cookbook has a recipe).

Combining questions

`and`, `or` and `not` combine answers:

Word	Meaning
`a and b`	True only when both are true.
`a or b`	True when at least one is true.
`not a`	The opposite.

PROGRAM

```
type enlng

remember age as 25
remember member as true
show age >= 18 and member
show age < 18 or member
show not member
show not (age < 18)
```

OUTPUT

```
true
true
false
true
```

`and` binds tighter than `or`, as in most languages. Use brackets when a line mixes them, so the next reader does not need to remember that.

Truthiness

A condition is expected to be `true` or `false`. Write the comparison out (`count is greater than 0`) rather than relying on a number or text standing in for a truth value.

CHAPTER 10

Making Decisions

`when` runs a block of lines only if a condition is true. The block is the indented lines under it, and the line ends with a colon.

PROGRAM

```
type enlng

remember temperature as 31

when temperature is greater than 30:
    show "It is hot"
    show "Drink water"
show "Done"
```

OUTPUT

```
It is hot
Drink water
Done
```

Everything indented under `when` belongs to it. The line `show "Done"` is not indented, so it runs either way.

otherwise

`otherwise:` gives the block that runs when the condition was false:

PROGRAM

```
type enlng

remember balance as 40
remember price as 75

when balance is at least price:
    show "Purchase complete"
otherwise:
    show "Not enough balance, missing" price minus balance
```

OUTPUT

```
Not enough balance, missing 35
```

Chains

`otherwise when` adds another question. The first condition that is true wins and the rest are skipped:

PROGRAM

```
type enlng

remember marks as 72

when marks is at least 90:
    show "Grade A"
otherwise when marks is at least 75:
    show "Grade B"
otherwise when marks is at least 60:
    show "Grade C"
otherwise:
    show "Grade D"
```

OUTPUT

```
Grade C
```

Because the first true condition wins, put the strictest test first. Here 72 is at least 60, but it is not at least 75, so the third branch runs.

Decisions inside decisions

A block can hold another `when`. Indent each level one step further:

PROGRAM

```
type enlng

remember age as 20
remember has_ticket as true

when age is at least 18:
    when has_ticket:
        show "Welcome in"
    otherwise:
        show "Please buy a ticket"
otherwise:
    show "Adults only"
```

OUTPUT

```
Welcome in
```

A single true/false name can be used directly as the condition, as `has_ticket` is above.

Indentation

The language uses indentation to see where blocks begin and end. Use four spaces per level and do not mix tabs and spaces.

The tool is forgiving about odd depths, and that can hide mistakes. In the program below the last line is indented by two spaces, which matches no level. It is read as a line outside the block, so it runs even though the condition is false:

PROGRAM

```
type enlng

when 1 is 2:
    show "inside the block"
  show "this is not inside it"
```

OUTPUT

```
this is not inside it
```

If a line seems to run when it should not, check its indentation first.

Choosing the largest of three

PROGRAM

```
type enlng

remember a as 14
remember b as 91
remember c as 56
remember biggest as a

when b is greater than biggest:
    biggest = b
when c is greater than biggest:
    biggest = c
show "Largest:" biggest
```

OUTPUT

```
Largest: 91
```

A leap-year test

A year is a leap year if it divides by 4, except that centuries must divide by 400:

PROGRAM

```
type enlng

function is_leap with year:
    when year mod 400 is 0:
        give true
    when year mod 100 is 0:
        give false
    when year mod 4 is 0:
        give true
```

```
    give false

for each y in [1900, 2000, 2023, 2024]:
    show y is_leap(y)
```

OUTPUT

```
1900 false
2000 true
2023 false
2024 true
```

CHAPTER 11

Loops

A loop repeats a block. Enlangg has loops that count, loops that walk through a list, and loops that run while a condition holds.

Counting: for ... from ... to

The loop name takes each value from the start up to and including the end:

PROGRAM

```
type enlng

for i from 1 to 5:
    show i
```

OUTPUT

```
1
2
3
4
5
```

Add `by` to step by more than one:

PROGRAM

```
type enlng

for i from 0 to 20 by 5:
    show i
```

OUTPUT

```
0
5
10
15
20
```

The end value is included. From 5.5.2 a negative step counts down to the end value:

PROGRAM

```
type enlng

for i from 3 to 1 by -1:
    show i
```

OUTPUT

```
3
2
1
```

A step of 0 is an error, because the loop would never end. Without `by`, the step is 1, so `for i from 3 to 1` does not run at all. In toolchain 5.5.1 and earlier a negative step did not run either; there, count up and subtract.

Doing something a fixed number of times

PROGRAM

```
type enlng

repeat 3 times:
    show "hip hip hooray"
```

OUTPUT

```
hip hip hooray
hip hip hooray
hip hip hooray
```

Walking through a list or text

`for each` takes each item in turn. It works on a list, on text (one character at a time) and on the list returned by `keys`:

PROGRAM

```
type enlng

for each fruit in ["apple", "mango", "kiwi"]:
    show "I like" fruit

for each letter in "abc":
    show letter
```

OUTPUT

```
I like apple
I like mango
I like kiwi
a
b
c
```

Walking through a map

Looping directly over a map does not give you the keys in toolchain 5.5.1. Loop over `keys(map)` and look each value up:

PROGRAM

```
type enlng
```

```
remember stock as {"pens": 12, "books": 3}
for each name in keys(stock):
    show name "->" stock[name]
```

OUTPUT

```
books -> 3
pens -> 12
```

Loops that wait for a condition

`while` and `repeat while` run as long as the condition is true. They are the same loop with two spellings:

PROGRAM

```
type enlng

remember n as 1
while n < 100:
    n = n times 2
show n
```

OUTPUT

```
128
```

The condition is checked before each pass, so if it is false at the start the block never runs. Make sure something inside the block changes what the condition tests, otherwise the loop never ends.

`repeat until` is the mirror image: it runs until the condition becomes true.

PROGRAM

```
type enlng

remember total as 0
remember k as 0
repeat until total is at least 10:
    k increases by 1
    total increases by k
show "k =" k "total =" total
```

OUTPUT

```
k = 4 total = 10
```

stop and skip

`stop` leaves the loop at once. `skip` abandons the rest of this pass and goes to the next one:

PROGRAM

```
type enlng
```

```
for each n in [1, 2, 3, 4, 5, 6]:
    when n mod 2 is 0:
        skip
    when n is 5:
        stop
    show n
```

OUTPUT

```
1
3
```

Loops inside loops

The inner loop runs completely for each pass of the outer one. A multiplication table:

PROGRAM

```
type enlng

for row from 1 to 4:
    remember line as ""
    for col from 1 to 4:
        line = line + (row times col) + " "
    show line
```

OUTPUT

```
1 2 3 4
2 4 6 8
3 6 9 12
4 8 12 16
```

Accumulating

Most loops build up one result. Start the result before the loop, update it inside, and use it after:

PROGRAM

```
type enlng

remember total as 0
for each price in [120, 80, 45, 15]:
    total increases by price
show "Sum:" total
show "Average:" total divided by 4
```

OUTPUT

```
Sum: 260
Average: 65
```

An infinite loop

A loop whose condition never becomes false runs forever. If it happens, stop the program with Ctrl+C in the terminal:

```
type enlng

remember n as 1
while n > 0:
    n increases by 1
```

CHAPTER 12

Lists

A list holds an ordered sequence of values. Write it with square brackets and commas. A list can hold any mix of kinds, including other lists.

PROGRAM

```
type enlng

remember primes as [2, 3, 5, 7, 11]
remember mixed as [1, "two", 3.5, true]
remember grid as [[1, 2], [3, 4]]
remember empty as []
show primes
show mixed
show grid
show empty
```

OUTPUT

```
[2, 3, 5, 7, 11]
[1, two, 3.5, true]
[[1, 2], [3, 4]]
[]
```

Reading items

Positions count from zero. A negative position counts back from the end. A position that does not exist gives `null` instead of an error:

PROGRAM

```
type enlng

remember colors as ["red", "green", "blue"]
show colors[0]
show colors[2]
show colors[-1]
show colors at 1
show colors[10]
```

OUTPUT

```
red
blue
blue
green
null
```

A list inside a list is read with two positions:

PROGRAM

```
type enlng

remember grid as [[1, 2], [3, 4]]
show grid[1][0]
```

OUTPUT

```
3
```

From 5.5.2, `items[a:b]` gives a new list with the items from position `a` up to, but not including, `b`, with the same rules as for text (chapter 8):

PROGRAM

```
type enlng

remember scores as [10, 20, 30, 40, 50]
show scores[1:3]
show scores[:2]
show scores[-2:]
remember top as scores[0:2]
append(top, 99)
show top scores
```

OUTPUT

```
[20, 30]
[10, 20]
[40, 50]
[10, 20, 99] [10, 20, 30, 40, 50]
```

The last line shows that the slice is a separate list: adding to `top` leaves `scores` as it was. A slice cannot be assigned to; `scores[0:2] = 5` is an error when the file is read.

How many items

`count of`, `length of` and `len(...)` all work. `sum of`, `max of`, `min of` and `average of` summarise a list of numbers, and `first of` and `last of` give the ends:

PROGRAM

```
type enlng

remember scores as [72, 91, 55, 88]
show count of scores
show len(scores)
show sum of scores
show max of scores
show min of scores
show average of scores
show first of scores
show last of scores
```

OUTPUT

```
4
4
306
91
55
76.5
72
88
```

An empty list gives `0` for its sum and count and `null` for its maximum, minimum and ends.

Adding and removing

`append(list, value)` adds to the end. `pop(list)` removes the last item and gives it back; on an empty list it gives `null`:

PROGRAM

```
type enlng

remember stack as []
append(stack, "a")
append(stack, "b")
append(stack, "c")
show stack
show pop(stack)
show stack
```

OUTPUT

```
[a, b, c]
c
[a, b]
```

There is no built-in for removing from the middle or the front of a list. Build a new list instead; the cookbook shows how.

Changing an item

Assign to a position, or use an increase or decrease on it:

PROGRAM

```
type enlng

remember xs as [10, 20, 30]
xs[1] = 99
show xs
xs[0] increases by 5
show xs
```

OUTPUT

```
[10, 99, 30]
[15, 99, 30]
```

Assigning past the end pads with `null`:

PROGRAM

```
type enlng

remember xs as [1]
xs[3] = 4
show xs
```

OUTPUT

```
[1, null, null, 4]
```

Changing an item of a list inside a list (`grid[1][0] = 9`) is not supported in toolchain 5.5.1. Read the inner list into a name, change it, and it is changed in the outer list too, because of the next point.

Lists are shared

A name holds a reference to the list, not a copy. If you give the same list a second name, both names see every change:

PROGRAM

```
type enlng

remember original as [1, 2, 3]
remember alias as original
append(alias, 4)
show original
```

OUTPUT

```
[1, 2, 3, 4]
```

To make an independent copy, build a new list in a loop:

PROGRAM

```
type enlng

remember original as [1, 2, 3]
remember copy as []
for each x in original:
    append(copy, x)
append(copy, 4)
show original
show copy
```

OUTPUT

```
[1, 2, 3]
```

```
[1, 2, 3, 4]
```

Searching a list

`contains(list, value)` tells you whether a value is present:

PROGRAM

```
type enlng

remember names as ["asha", "ravi", "meera"]
show contains(names, "ravi")
show contains(names, "zoya")
```

OUTPUT

```
true
false
```

Reversing and sorting

`reverse` as a statement reverses the list in place. Written as `reverse(list)` inside an expression it gives back a reversed list. `sort list ascending` and `sort list descending` reorder it in place:

PROGRAM

```
type enlng

remember xs as [4, 8, 15, 16, 23, 42]
reverse xs
show xs
sort xs ascending
show xs
sort xs descending
show xs
```

OUTPUT

```
[42, 23, 16, 15, 8, 4]
[4, 8, 15, 16, 23, 42]
[42, 23, 16, 15, 8, 4]
```

Walking a list

PROGRAM

```
type enlng

remember prices as [120, 80, 45]
remember total as 0
for each p in prices:
    total increases by p
show "Total:" total
```

OUTPUT

```
Total: 245
```

Splitting text into a list and back

PROGRAM

```
type enlng

remember words as split("one two three", " ")
show words
show count of words
show join(words, "+")
```

OUTPUT

```
[one, two, three]
3
one+two+three
```

`join` expects a list of text. A list of numbers joins to an empty result in this release, so turn numbers into text first by adding them to a string inside a loop.

CHAPTER 13

Maps

A map stores values under names, called keys. It is the natural way to keep a record such as a person, a product or a setting. Write a map with curly braces; each entry is a key in quotes, a colon and a value.

PROGRAM

```
type enlng

remember person as {"name": "Asha", "age": 31, "city": "Pune"}
show person["name"]
show person["age"]
```

OUTPUT

```
Asha
31
```

Keys must be text in quotes. A map written with number keys, such as `{1: "one"}`, is a syntax error.

Reading

Use the key in square brackets, or a dot when the key is a plain word. A key that is not in the map gives `null`:

PROGRAM

```
type enlng

remember car as {"make": "Tata", "year": 2019}
show car["make"]
show car.year
show car["colour"]
```

OUTPUT

```
Tata
2019
null
```

Does a key exist?

`has_key(map, key)` answers true or false, which is the safe way to ask before reading:

PROGRAM

```
type enlng
```

```
remember ages as {"asha": 31, "ravi": 27}
show has_key(ages, "asha")
show has_key(ages, "zoya")

when has_key(ages, "zoya"):
    show ages["zoya"]
otherwise:
    show "Zoya is not listed"
```

OUTPUT

```
true
false
Zoya is not listed
```

Adding and changing

Assigning to a key that exists replaces the value. Assigning to one that does not exist adds it:

PROGRAM

```
type enlng

remember stock as {"pens": 12}
stock["pens"] = 10
stock["books"] = 4
show stock["pens"]
show stock["books"]
show count of stock
```

OUTPUT

```
10
4
2
```

`increases by` and `decreases by` work on a map entry, with brackets or with a dot:

PROGRAM

```
type enlng

remember counter as {"hits": 0}
counter["hits"] increases by 1
counter.hits increases by 1
show counter["hits"]
```

OUTPUT

```
2
```

All the keys and all the values

`keys(map)` and `values(map)` give lists. The order of the keys is decided by the map, not by the order you wrote them, so do not depend on it. The same keys always come out in the same order for the same map:

PROGRAM

```
type enlng

remember prices as {"tea": 10, "coffee": 25, "juice": 30}
show count of keys(prices)
show contains(keys(prices), "tea")
show sum of values(prices)
```

OUTPUT

```
3
true
65
```

Walking a map

Loop over the keys and look each value up:

PROGRAM

```
type enlng

remember prices as {"tea": 10, "coffee": 25, "juice": 30}
remember total as 0
for each item in keys(prices):
    total increases by prices[item]
show "Menu total:" total
```

OUTPUT

```
Menu total: 65
```

If you need a particular order, put the keys in a list and sort the list:

PROGRAM

```
type enlng

remember prices as {"tea": 10, "coffee": 25, "juice": 30}
remember names as keys(prices)
sort names ascending
for each n in names:
    show n prices[n]
```

OUTPUT

```
coffee 25
juice 30
tea 10
```

Maps and lists together

Values can be lists or maps, so records nest. Reading goes through each level in turn:

PROGRAM

```
type enlng

remember school as {"name": "Sunrise", "students": ["Asha", "Ravi"
    ], "address": {"city": "Pune", "pin": 411001}}
show school["name"]
show school["students"][1]
show school["address"]["city"]
```

OUTPUT

```
Sunrise
Ravi
Pune
```

Assigning to a level two deep (`school["address"]["city"] = "Mumbai"`) is not supported in toolchain 5.5.1. Read the inner map into a name, change that name's entry, and the change is seen through the outer map, because maps are shared the way lists are.

PROGRAM

```
type enlng

remember school as {"address": {"city": "Pune"}}
remember address as school["address"]
address["city"] = "Mumbai"
show school["address"]["city"]
```

OUTPUT

```
Mumbai
```

Counting words: a typical use

PROGRAM

```
type enlng

remember text as "the cat saw the dog and the bird"
remember counts as {}
for each word in split(text, " "):
    when has_key(counts, word):
        counts[word] increases by 1
    otherwise:
        counts[word] = 1
show counts["the"]
show counts["cat"]
show count of counts
```

OUTPUT

```
3
1
6
```

CHAPTER 14

Reaching Inside Lists, Maps and Text

You pick a single item out of a list, a map or a piece of text by saying where it is. Enlangg gives you four ways to say it. They mean the same thing, so use whichever reads best in the sentence you are writing.

Style	Example	Works on
Brackets	`person["name"]`, `xs[2]`	Lists, maps, text
Dot	`person.name`	Maps with plain-word keys
`at`	`xs at 2`, `person at "name"`	Lists, maps, text
`of`	`name of person`	Maps

PROGRAM

```
type enlng

remember user as {"role": "Architect", "level": 5}
show user["role"]
show user.role
show user at "role"
show role of user
```

OUTPUT

```
Architect
Architect
Architect
Architect
```

`at` for lists and text

`at` reads naturally for a position:

PROGRAM

```
type enlng

remember lane as [10, 20, 30]
show lane at 0
show lane at 2
remember word as "banana"
show word at 1
```

OUTPUT

```
10
30
a
```

`of` for a record's fields

`of` reads like possession, which suits records:

PROGRAM

```
type enlng

remember book as {"title": "Dune", "pages": 412}
show "Title:" title of book
show "Pages:" pages of book
```

OUTPUT

```
Title: Dune
Pages: 412
```

The same word is also used for a few summaries: `count of items`, `length of text`, `sum of numbers`, `first of items`, `last of items`, `max of numbers`, `min of numbers` and `average of numbers`. Those are described in the chapters on lists and text.

Changing through any style

An item can be changed through the same forms you use to read it, including an increase or a decrease:

PROGRAM

```
type enlng

remember account as {"balance": 1000, "owner": "Asha"}
balance of account increases by 250
account.balance decreases by 100
account["owner"] = "Asha K"
change owner of account to "Asha Kulkarni"
show account
```

OUTPUT

```
{"owner": Asha Kulkarni, "balance": 1150}
```

Chaining

Brackets and dots chain when you have records inside records or lists inside lists:

PROGRAM

```
type enlng
```

```
remember shop as {"name": "Corner", "items": [{"sku": "A1", "qty":
     4}, {"sku": "B2", "qty": 9}]}
show shop["items"][1]["sku"]
show shop.name
```

OUTPUT

```
B2
Corner
```

Only reading is chained. Assigning through two levels at once is not supported in toolchain 5.5.1; read the inner value into a name first, as the chapter on maps shows.

Spoken positions

Inside a loop that walks a list of pairs, `the left of the pair` and `the right of the pair` name the two neighbours being compared. This is used by the sorting pattern in chapter 19.

Which style should I use?

There is no rule. A common habit is brackets when the key is a variable (`prices[item]`), `of` when a program is read aloud (`price of item`), and the dot for short fixed names in a line that also has arithmetic.

CHAPTER 15

Functions

A function is a named block of steps that you can run whenever you like, with different inputs each time. Once you have written a function, a program reads at the level of what it does instead of how.

Writing and calling

`function name with inputs:` starts the definition. The lines indented under it are the body. `give` hands a result back to whoever called the function:

PROGRAM

```
type enlng

function add with a, b:
    give a plus b

show add with 2, 3
show add(10, 20)
remember total as add with 4, 5
show total
```

OUTPUT

```
5
30
9
```

A function can be called either with the word `with` or with brackets. Inside an expression you can nest calls:

PROGRAM

```
type enlng

function double with n:
    give n times 2

show double(double(3))
show double with 3 plus 4
```

OUTPUT

```
12
14
```

Notice the last line. After `with`, the whole expression up to the end of the line is the input, so `double with 3 plus 4` doubles 7, not 3. Use brackets when you

want only part of a line to be the input.

The action spelling

`action name(inputs):` with `return` is a second spelling of exactly the same thing. Use whichever style your program already uses; do not mix them in one program without a reason.

PROGRAM

```
type enlng

action square(n):
    return n times n

action cube(n):
    return n times square(n)

show square(5)
show cube(3)
```

OUTPUT

```
25
27
```

No inputs

A function with no inputs is written without `with`, and is called with empty brackets. A bare name on a line by itself is not a call:

PROGRAM

```
type enlng

function greet:
    show "Hello there"

greet()
greet()
```

OUTPUT

```
Hello there
Hello there
```

What a function gives back

When the function ends without `give`, the caller gets `null`. A function can `give` from inside a decision to leave early:

PROGRAM

```
type enlng
```

```
function sign_of with n:
    when n is less than 0:
        give "negative"
    when n is 0:
        give "zero"
    give "positive"

show sign_of(-5)
show sign_of(0)
show sign_of(8)
```

OUTPUT

```
negative
zero
positive
```

Inputs must match

A function must be called with exactly as many inputs as it names. Too many or too few is an error that names the function:

PROGRAM WITH A MISTAKE

```
type enlng

function add with a, b:
    give a plus b

show add with 1
```

WHAT enlng PRINTS

```
-- Enlang RuntimeError ------------------------------------------
    -------
Location: program.enlng:6

    6 | show add with 1

What: 'add' needs 2 inputs but was given 1.
--------------------------------------------------------------------
    ------
```

Functions with decisions and loops

Any statement can appear in a body. This one finds the biggest number in a list:

PROGRAM

```
type enlng

function largest with numbers:
    remember best as numbers[0]
    for each n in numbers:
```

```
        when n is greater than best:
            best = n
    give best

show largest([3, 41, 7, 19])
show largest([-5, -2, -9])
```

OUTPUT

```
41
-2
```

Lists and maps are shared

A function that receives a list or map can change it, and the caller sees the change, because the function was handed the list itself, not a copy. Whole numbers and text are different: changing one inside a function does not affect the caller's name.

PROGRAM

```
type enlng

function bump with number, items:
    number = number plus 1
    append(items, number)

remember n as 1
remember log as []
bump(n, log)
bump(n, log)
show n
show log
```

OUTPUT

```
1
[2, 2]
```

Composing functions

Small functions make bigger ones readable:

PROGRAM

```
type enlng

function is_even with n:
    give n mod 2 is 0

function count_evens with numbers:
    remember c as 0
    for each n in numbers:
        when is_even(n):
```

```
            c increases by 1
    give c

show count_evens([1, 2, 3, 4, 6, 7, 8])
```

OUTPUT

```
4
```

Functions as values

From 5.5.2 the name of a function is a value, like a number or a list. It can be kept in a name, put in a list or a map, and handed to another function, which calls it like any function:

PROGRAM

```
type enlng

function double with x:
    give x times 2

function is_even with n:
    give n mod 2 is 0

function apply_to_all with operation, items:
    remember out as []
    for each item in items:
        append(out, operation(item))
    give out

function keep_if with test, items:
    remember out as []
    for each item in items:
        when test(item):
            append(out, item)
    give out

show apply_to_all(double, [1, 2, 3])
show keep_if(is_even, [1, 2, 3, 4, 5, 6])

remember f as double
show f(21)
show f
```

OUTPUT

```
[2, 4, 6]
[2, 4, 6]
42
<function double>
```

`show f` prints `<function double>`. When a call such as `operation(item)` names no function of the program, the name is looked up as a value, and the function it holds runs, with the same check on the number of inputs. Calling a name that holds something else, such as a number, is an error. Built-in functions such as `sqrt` can be passed in the same way: `apply_to_all(sqrt, [4, 9, 16])` is `[2, 3, 4]`.

Functions made where they are used

`function with x: x times 2` makes a function with no name, right where a value is wanted. After the colon comes what it gives back; to write several lines, put the colon at the end of the line and indent the steps, as in any other function. An inner function is the same thing with a name, written inside another function. Both keep the values of the outer names they use, as they were when the function was made. That is what lets a function build other functions:

PROGRAM

```
type enlng

function make_adder with n:
    function add with x:
        give x plus n
    give add

remember add5 as make_adder(5)
remember add10 as make_adder(10)
show add5(1)
show add10(1)

show apply_to_all(function with n: n times n, [1, 2, 3])

remember multipliers as []
for i from 1 to 3:
    append(multipliers, function with x: x times i)
for each m in multipliers:
    show m(10)

function apply_to_all with operation, items:
    remember out as []
    for each item in items:
        append(out, operation(item))
    give out
```

OUTPUT

```
6
11
```

```
[1, 4, 9]
10
20
30
```

An inner function can call itself, so recursion works inside a function too. The kept values are copies: a number or text the outer function changes afterwards is not seen by the inner one, while a list or map is shared, so changes made through it are seen by both. To keep a count that several calls update, keep it in a list: `remember count as [0]`, then `count[0] = count[0] plus 1`.

Giving good names

Name a function for what it gives back or does: `average_of`, `is_leap`, `print_receipt`. A reader should know what a call does without opening the body.

CHAPTER 16

Recursion and Scope

Recursion

A function may call itself. The trick is to give it a case that does not, so that it stops. The usual shape is: if the problem is tiny, answer directly; otherwise solve a smaller version and build on that.

Factorial multiplies the numbers from `1` up to `n`:

PROGRAM

```
type enlng

function factorial with n:
    when n is at most 1:
        give 1
    give n times factorial(n minus 1)

show factorial(5)
show factorial(10)
show factorial(20)
```

OUTPUT

```
120
3628800
2432902008176640000
```

The Fibonacci numbers, where each is the sum of the two before it:

PROGRAM

```
type enlng

function fib with n:
    when n is less than 2:
        give n
    give fib(n minus 1) plus fib(n minus 2)

for i from 0 to 12:
    show fib(i)
```

OUTPUT

```
0
1
1
2
3
5
```

```
8
13
21
34
55
89
144
```

This version repeats a lot of work. `fib(30)` makes over a million calls. When speed matters, use a loop that keeps the two latest values:

PROGRAM

```
type enlng

function fib_fast with n:
    remember a as 0
    remember b as 1
    for i from 1 to n:
        remember next as a plus b
        a = b
        b = next
    give a

show fib_fast(30)
show fib_fast(50)
```

OUTPUT

```
832040
12586269025
```

Greatest common divisor

Euclid's method: the answer for `a` and `b` is the answer for `b` and the remainder of `a` by `b`.

PROGRAM

```
type enlng

function gcd with a, b:
    when b is 0:
        give a
    give gcd(b, a mod b)

show gcd(48, 18)
show gcd(17, 5)
```

OUTPUT

```
6
1
```

Recursion depth

Each pending call takes room. The interpreter allows a nesting of 20000 calls and stops a deeper one with an error that names the function (chapter 5 shows it). A recursive sum to 10000 works:

PROGRAM

```
type enlng

function total_to with n:
    when n is 0:
        give 0
    give n plus total_to(n minus 1)

show total_to(10000)
```

OUTPUT

```
50005000
```

If you expect more depth than that, rewrite the function as a loop.

Scope

The **scope** of a name is the part of the program where it can be used.

Names made in a function stay in it

A name created inside a function exists only while that function runs:

PROGRAM WITH A MISTAKE

```
type enlng

function make with n:
    remember secret as n times 2
    give secret

show make(4)
show secret
```

WHAT enlng PRINTS

```
8

-- Enlang RuntimeError ----------------------------------------
    -------
Location: program.enlng:8

    8 | show secret

What: I don't know a name 'secret' here. Define it first, for exam
    ple: remember secret as 0
----------------------------------------------------------------
```

```
------
```

A function can read the program's top-level names

A name made at the top of the program, outside every function, can be read from inside a function. This is how a constant is shared:

PROGRAM

```
type enlng

freeze rate as 18

function tax with price:
    give price times rate divided by 100

show tax(200)
```

OUTPUT

```
36
```

An assignment inside a function makes a new name that belongs to the function, and the top-level name is untouched:

PROGRAM

```
type enlng

remember counter as 5

function bump with x:
    counter = 100
    give counter + x

show bump(1)
show counter
```

OUTPUT

```
101
5
```

To change the top-level name instead, say so first with `global` (from 5.5.2). After `global counter`, an assignment in that function changes the name made at the top of the program:

PROGRAM

```
type enlng

remember counter as 5

function add with x:
    global counter
    counter = counter + x
```

```
add(10)
add(1)
show counter
```

OUTPUT

```
16
```

Several names can follow one `global`, separated by commas. The name must already exist at the top of the program; if it does not, the assignment is an error. `increases by` and `decreases by` change a top-level name even without `global`, because they need a value to start from.

Asking for it by name keeps one rule safe: a function in a library cannot change a name in your program by accident, just because it happens to use the same spelling.

A parameter or a name made inside the function hides a top-level name of the same spelling. A list or map is shared, so a function that changes one changes it for everyone. Passing what a function needs as inputs is still the clearer habit, because then what the function does depends only on its inputs. (In toolchain 5.5.1 and earlier a function could not see outer names at all.)

Blocks

A name created inside a `when` or a loop belongs to that block. A name with the same spelling made inside the block hides an outer one for as long as the block runs, and the outer one is untouched afterwards:

PROGRAM

```
type enlng

remember x as 1
when true:
    remember x as 99
    show "inside:" x
show "outside:" x
```

OUTPUT

```
inside: 99
outside: 1
```

If you want to change an outer name from inside a block, assign without `remember`:

PROGRAM

```
type enlng

remember total as 0
```

```
for each n in [1, 2, 3]:
    total increases by n
show total
```

OUTPUT

```
6
```

Loop variables

The name in `for each n in ...` or `for i from ...` belongs to the loop. Use a different name after the loop if you need to keep the last value:

PROGRAM

```
type enlng

remember last as 0
for i from 1 to 4:
    last = i
show last
```

OUTPUT

```
4
```

CHAPTER 17

The Type Line and Natural Phrasing

The first line

Every `.enlng` file starts with `type enlng`. It tells the toolchain which of the six languages the file is written in, so that a file meant for the database tool is never read by the language tool by mistake. If the line is missing, or names a different type, the file is refused before anything runs:

PROGRAM WITH A MISTAKE

```
remember x as 10
show x
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ---------------------------------------------
    -------
Location: program.enlng:1

    1 | remember x as 10
      | ^
    2 | show x

What: [DOMAIN ERROR] Source must begin with 'type enlng'.
Why:  Sovereign native Enlang compilation units must declare domai
    n type on line 1.

Suggestions:
  * Add 'type enlng' at the very top of your file.
------------------------------------------------------------------
    ------
```

PROGRAM WITH A MISTAKE

```
type python

show 1
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ---------------------------------------------
    -------
Location: program.enlng:1

    1 | type python
      | ^
    2 |
```

```
What: [DOMAIN ERROR] Received domain 'python'. Expected 'type enln
    g'.
Why:  Sovereign native Enlang compilation units must declare domai
    n type on line 1.

Suggestions:
  * Add 'type enlng' at the very top of your file.
------------------------------------------------------------------
    ------
```

Comments and blank lines may come before it. Nothing else may.

Silent words

You can add the small words `the`, `that` and `it` to make a line read more naturally. The tool skips them. These two programs are the same:

PROGRAM

```
type enlng

remember total as 50
when total is at least 50:
    show "enough"
```

OUTPUT

```
enough
```

PROGRAM

```
type enlng

remember the total as 50
when the total is at least 50:
    show "enough"
```

OUTPUT

```
enough
```

Use them to make code read aloud well, not to decorate it. One silent word per sentence is usually plenty.

Symbols and words are interchangeable

Anywhere a symbol is allowed an English word is allowed too, and you can mix them. Both spellings of the compound updates work for increasing:

PROGRAM

```
type enlng

remember a as 10
remember b as 10
```

```
a += 5
b increases by 5
show a b
```

OUTPUT

```
15 15
```

Only `+=` exists in the symbol form. For decreasing, use `decreases by` or `a = a minus 1`.

Words that change nothing

A few filler words appear in the grammar to let sentences flow: `by` in `increases by`, `of` in `count of`, `than` in `is greater than`. They are part of the phrases they appear in and are not optional there.

A sentence is one line

Each statement fits on one line. There is no continuation character, so keep long conditions readable by naming the parts:

PROGRAM

```
type enlng

remember age as 34
remember income as 52000
remember adult as age is at least 18
remember earning as income is greater than 30000
when adult and earning:
    show "Eligible"
```

OUTPUT

```
Eligible
```

Case

Words of the language are written in lower case. Your own names can use any case, and `Total` is a different name from `total`.

CHAPTER 18

Sorting, Swapping and Arranging

Ordering things is so common that Enlangg has words for it.

sort

`sort list ascending` and `sort list descending` reorder a list in place. Without a direction it sorts ascending. Numbers sort by value and text sorts alphabetically:

PROGRAM

```
type enlng

remember numbers as [42, 7, 19, 3, 88]
sort numbers ascending
show numbers
sort numbers descending
show numbers

remember names as ["meera", "asha", "ravi"]
sort names
show names

remember sizes as [10, 9, 100]
sort sizes
show sizes
```

OUTPUT

```
[3, 7, 19, 42, 88]
[88, 42, 19, 7, 3]
[asha, meera, ravi]
[9, 10, 100]
```

Notice that numbers sort as numbers: 9 comes before 10 and 100.

Sorting by hand: swapping neighbours

Sometimes you want to see the steps. Bubble sort repeatedly looks at neighbouring pairs and swaps those that are out of order. Enlangg has a phrase for that: `for each pair in list` walks the neighbours, `the left of the pair` and `the right of the pair` name the two, `swap pair` exchanges them, and `repeat until sorted` repeats the pass until nothing is out of order:

PROGRAM

```
type enlng

remember numbers as [64, 34, 25, 12, 22, 11, 90]
show "Before:" numbers

repeat until sorted:
    for each pair in numbers:
        when the left of the pair > the right of the pair:
            swap pair

show "After: " numbers
```

OUTPUT

```
Before: [64, 34, 25, 12, 22, 11, 90]
After:  [11, 12, 22, 25, 34, 64, 90]
```

The word `sorted` here is a built-in flag: the loop runs again while the last pass swapped something. It reads as a sentence, and it is correct because it ends only when a full pass changes nothing.

Writing bubble sort with positions

You can do the same with explicit positions. It is longer, but it is how most languages write it, so it is worth knowing:

PROGRAM

```
type enlng

remember xs as [5, 2, 9, 1, 7]
remember n as count of xs
remember swapped as true
while swapped:
    swapped = false
    for i from 0 to n minus 2:
        when xs[i] > xs[i plus 1]:
            remember tmp as xs[i]
            xs[i] = xs[i plus 1]
            xs[i plus 1] = tmp
            swapped = true
show xs
```

OUTPUT

```
[1, 2, 5, 7, 9]
```

Reversing

`reverse list` reverses in place:

PROGRAM

```
type enlng
```

```
remember word_list as ["one", "two", "three"]
reverse word_list
show word_list
```

OUTPUT

```
[three, two, one]
```

arrange: pick positions in any order

`arrange` rebuilds a text or a list from the positions you name, in the order you name them. It can reverse, repeat, pick every other item, or shuffle by a fixed pattern:

PROGRAM

```
type enlng

remember s as "enlangg"
arrange s by 6, 5, 4, 3, 2, 1, 0
show s

remember t as "abcdef"
arrange t in 0, 2, 4
show t

remember u as "abc"
arrange u at 0, 0, 1, 1, 2, 2
show u

remember nums as [10, 20, 30, 40]
arrange nums by 3, 2, 1, 0
show nums
```

OUTPUT

```
ggnalne
ace
aabbcc
[40, 30, 20, 10]
```

The connecting word (`by`, `with`, `in` or `at`) can be any of the four. Negative positions count from the end, so `arrange s by -1, -2, -3` reverses the last three characters.

PROGRAM

```
type enlng

remember s as "spandan"
arrange s by -1, -2, -3
show s
```

OUTPUT

```
nad
```

Finding the smallest and largest

For a quick answer you do not need to sort at all:

PROGRAM

```
type enlng

remember temps as [31, 28, 35, 29, 33]
show "Lowest:" min of temps
show "Highest:" max of temps
show "Average:" average of temps
```

OUTPUT

```
Lowest: 28
Highest: 35
Average: 31.2
```

Sorting records by a field

`sort` orders simple values. To order records such as maps, pull the field you care about into a list, sort that, and look the records up again. The cookbook has a worked recipe for sorting students by marks.

CHAPTER 19

Input, Files and Time

Reading what the user types

`ask name` waits for a line of text, then stores it under `name`. The value is always **text**, even if the user typed digits. This surprises people:

PROGRAM

```
type enlng

ask n
show n times 2
show n plus 1
```

OUTPUT

```
55
51
```

Here `n` is the text `"5"`, so `n times 2` repeats the text and `n plus 1` joins `"1"` on the end. To compute with a typed number, convert it first with `to_number` (from 5.5.2), which understands whole numbers and fractions and stops with an error if the text is not a number:

TERMINAL

```
$ echo 21 | enlng run main.enlng
42
```

Wrap it in `try` (chapter 21) to ask again when the user types something else. The standard library's older `parse_int` still works, but it skips any character that is not a digit, so `parse_int("3.5")` gives 35.

At the end of the input `ask` gives an empty text, not an error, which makes loops that read until a blank line straightforward.

`ask` inside a block

A name created by `ask` belongs to the block it is in. Inside a loop that means a fresh name on each pass, and an outer name of the same spelling is not updated. Ask into a temporary name and assign it to the outer one:

PROGRAM

```
type enlng

remember count as 0
```

```
remember line as "start"
while line is not "":
    ask next_line
    line = next_line
    when line is not "":
        count increases by 1
show "lines read:" count
```

OUTPUT

```
lines read: 3
```

Files

Five built-in functions work with files:

Function	What it does
`write_file(path, text)`	Creates the file, or replaces it, with `text`.
`append_file(path, text)`	Adds `text` to the end of the file.
`read_file(path)`	Gives the whole file as text. A missing file gives an empty text.
`file_exists(path)`	True or false.
`time_now()`	The current time as seconds since 1970.

Inside text, `\n` is a new line, `\t` a tab and `\"` a quote:

PROGRAM

```
type enlng

write_file("notes.txt", "milk\neggs\nbread")
show file_exists("notes.txt")
show read_file("notes.txt")

append_file("notes.txt", "\ntea")
remember items as split(read_file("notes.txt"), "\n")
show count of items
show items[3]
show file_exists("missing.txt")
```

OUTPUT

```
true
milk
eggs
bread
4
tea
```

```
false
```

A missing file reads as empty text, so check with `file_exists` first when the difference matters.

Keeping a log

A file that grows by one line per run is only a few lines:

PROGRAM

```
type enlng

remember path as "visits.log"
append_file(path, "visit\n")
remember lines as split(read_file(path), "\n")
show "lines so far:" count of lines
```

OUTPUT

```
lines so far: 2
```

Each time this program is run the count goes up by one, because the file is kept between runs.

Random numbers

`random_number(low, high)` gives a whole number from `low` to `high`, both included. The result changes from run to run, so the example prints only facts that are always true:

PROGRAM

```
type enlng

remember roll as random_number(1, 6)
when roll is at least 1 and roll is at most 6:
    show "the roll is a valid die face"
```

OUTPUT

```
the roll is a valid die face
```

Time

`time_now()` is a number of seconds. Subtract two readings to time something:

PROGRAM

```
type enlng

remember start as time_now()
remember total as 0
for i from 1 to 1000:
    total increases by i
```

```
remember elapsed as time_now() minus start
show "sum:" total
show "took no more than 5 seconds:" elapsed is at most 5
```

OUTPUT

```
sum: 500500
took no more than 5 seconds: true
```

`sleep(seconds)` pauses the program.

CHAPTER 20

Splitting a Program into Files

A long program is easier to read in pieces. `use "file"` pulls another Enlangg file into this one, so the functions it defines are available here.

A small module

Put reusable functions in a file of their own:

TERMINAL

```
$ enlng run main.enlng
12.5664
12
```

Each file begins with its own `type enlng` line. Only the main file is run; the imported file is read in as if its lines were written where the `use` line is.

Giving a file a name

A file used as it is pours all its functions into the program. If two files define a function with the same name, the program refuses to start and says which name clashes. To keep a file apart, give it a name with `as`; its functions and its top-level names then need that name in front:

TERMINAL

```
$ enlng run main.enlng
shape 1
color 2
mine 3
4
```

The name works like any other: `remember f as colors.describe` keeps the function in a name. A name the file does not have is an error that says so. Inside the file itself, nothing changes; its functions still call each other by their plain names. Using the same file twice without `as` reads it once.

Where a path is looked up

A path in `use` is read relative to the file that contains the `use` line, not relative to the folder you were in when you started the program. That means a project keeps working when started from anywhere. Use `../` to go up a folder:

TERMINAL

```
$ enlng run app/main.enlng
27
$ cd app && enlng run main.enlng
27
```

Files that cannot be found

If the file does not exist, the program does not start, and the message names the path:

TERMINAL

```
$ enlng run main.enlng

-- Enlang SyntaxError ----------------------------------------
    -------
Location: main.enlng:2

    1 | type enlng
    2 | use "nowhere.enlng"
      | ^
    3 |

What: Could not load imported file: nowhere.enlng
Why:  The compiler encountered an unexpected token during syntax e
    valuation.

Suggestions:
  * Check statement syntax against Enlang language conventions.
----------------------------------------------------------------
    ------
```

The standard library

The standard library is a set of ordinary Enlangg files installed in the `lib` folder next to the tools. You import them like any other file, by name:

PROGRAM

```
type enlng
use "lib_math.enlng"

show gcd(48, 18)
show lcm(4, 6)
show is_prime(29)
```

OUTPUT

```
6
12
true
```

The library is described in the chapters that follow. Because the library is written in Enlangg, you can open any of its files and read how a function works.

Names from modules

Imported functions share one set of names with your program. If your main file defines a function with the same name as one from a module, the later definition wins. Give your own functions names that do not clash, or read the module's list of functions first.

Importing the same file twice

Importing a file is the same as pasting it. Importing the same file twice defines its functions twice, which is harmless but wasteful; import it once, at the top.

CHAPTER 21

Reading Error Messages

When something is wrong the tool stops and prints a message. Every message has the same shape, and learning the shape makes most mistakes quick to fix.

OUTPUT

```
-- Enlang SyntaxError ----------------------------
Location: program.enlng:4

    3 | show "before"
    4 | remember as 5
      | ^
    5 | show "after"

What: Expected variable name after remember/freeze
```

- The title says **when** the problem was found. A *SyntaxError* is found while reading the file, before anything runs. A *RuntimeError* is found while the program is running, so earlier lines have already printed.
- `Location` gives the file name and line number.
- The numbered lines show the offending line with its neighbours.
- `What` says what the tool expected or what went wrong. Read this line first.

The next sections show the mistakes you are most likely to make, with the exact message the tool prints.

A missing name

PROGRAM WITH A MISTAKE

```
type enlng

remember total as 10
show totl
```

WHAT enlng PRINTS

```
-- Enlang RuntimeError -----------------------------------------
    -------
Location: program.enlng:4

    4 | show totl

What: I don't know a name 'totl' here. Define it first, for exampl
    e: remember totl as 0
-----------------------------------------------------------------
```

```
    ------
```

The message names the unknown word and suggests how to create it. The usual cause is a typing slip.

A missing word in a statement

PROGRAM WITH A MISTAKE

```
type enlng

remember score
show score
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ---------------------------------------------
    -------
Location: program.enlng:3

    2 |
    3 | remember score
      | ^
    4 | show score

What: Expected 'as' or '=' in declaration
Why:  Variable bindings require 'as' or '=' to specify the initial
     value.

Suggestions:
  * Bind initial value with 'as', e.g. 'remember count as 0'.
-----------------------------------------------------------------
    ------
```

PROGRAM WITH A MISTAKE

```
type enlng

remember 5 as total
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ---------------------------------------------
    -------
Location: program.enlng:3

    2 |
    3 | remember 5 as total
      | ^

What: Expected variable name after remember/freeze
Why:  The compiler encountered an unexpected token during syntax e
    valuation.

Suggestions:
```

```
  * Check statement syntax against Enlang language conventions.
-----------------------------------------------------------------
    ------
```

An unfinished block

A line that opens a block must end with a colon:

PROGRAM WITH A MISTAKE

```
type enlng

when 1 is 1
    show "yes"
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ---------------------------------------
    -------
Location: program.enlng:3

    2 |
    3 | when 1 is 1
      | ^
    4 |     show "yes"

What: Expected ':' before block
Why:  Block headers in Enlang (when, if, while, for, function) mus
    t terminate with ':'.

Suggestions:
  * Did you mean to end the condition with a colon, e.g. 'when sco
    re == 90:'?
  * Ensure comparison operators use '==' or 'is' instead of '=' in
    side conditions.
-----------------------------------------------------------------
    ------
```

An unfinished text

A text value whose closing quote is missing is not reported. The tool reads to the end of the line, so the program runs, and the missing quote only shows up as odd output if the line also had something after the text:

PROGRAM WITH A MISTAKE

```
type enlng

show "hello
show "world"
```

WHAT enlng PRINTS

```
hello
world
```

An unknown function

PROGRAM WITH A MISTAKE

```
type enlng

show triple(4)
```

WHAT enlng PRINTS

```
-- Enlang RuntimeError ------------------------------------------
    -------
Location: program.enlng:3

    3 | show triple(4)

What: I don't know a function called 'triple'. Check the spelling,
     or define it with: action triple(...):
----------------------------------------------------------------
    ------
```

The wrong number of inputs

PROGRAM WITH A MISTAKE

```
type enlng

function area with w, h:
    give w times h

show area(3)
```

WHAT enlng PRINTS

```
-- Enlang RuntimeError ------------------------------------------
    -------
Location: program.enlng:6

    6 | show area(3)

What: 'area' needs 2 inputs but was given 1.
----------------------------------------------------------------
    ------
```

A function that is not called

PROGRAM WITH A MISTAKE

```
type enlng

function ping:
    show "pong"

ping
```

WHAT enlng PRINTS

```
(no output)
```

Note that `ping` alone is a name that does not exist as a value. Call it with brackets: `ping()`.

Using a list or a name in the wrong place

PROGRAM WITH A MISTAKE

```
type enlng

remember xs as [1, 2, 3]
show ys[0]
```

WHAT enlng PRINTS

```
-- Enlang RuntimeError ------------------------------------------
    -------
Location: program.enlng:4

    4 | show ys[0]

What: I don't know a name 'ys' here. Define it first, for example:
     remember ys as 0
----------------------------------------------------------------
    ------
```

A mismatched bracket

PROGRAM WITH A MISTAKE

```
type enlng

show (1 plus 2
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError -------------------------------------------
    -------
Location: program.enlng:3

    2 |
    3 | show (1 plus 2
      | ^

What: Expected ')' after parenthesized expression
Why:  The compiler encountered an unexpected token during syntax e
    valuation.

Suggestions:
  * Check statement syntax against Enlang language conventions.
----------------------------------------------------------------
    ------
```

Too much recursion

PROGRAM WITH A MISTAKE

```
type enlng

function again with n:
    give again(n plus 1)

show again(0)
```

WHAT enlng PRINTS

```
-- Enlang RuntimeError ------------------------------------------
    -------
Location: program.enlng:4

    4 |     give again(n plus 1)

What: 'again' called itself too many times (over 20000 levels deep
    ). Check that it has a way to stop.
---------------------------------------------------------------------
    ------
```

A missing file in `use`

PROGRAM WITH A MISTAKE

```
type enlng

use "no_such_file.enlng"
show 1
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError -------------------------------------------
    -------
Location: program.enlng:3

    2 |
    3 | use "no_such_file.enlng"
      | ^
    4 | show 1

What: Could not load imported file: no_such_file.enlng
Why:  The compiler encountered an unexpected token during syntax e
    valuation.

Suggestions:
  * Check statement syntax against Enlang language conventions.
---------------------------------------------------------------------
    ------
```

Not an error, but surprising

Some mistakes are not reported at all, so it is worth knowing them:

What you wrote	What happens
`remember total as 10` then a typo `totl = 5`	A new name `totl` is created. Nothing warns you.
`show 5 divided by 0`	Prints 0.
`show xs[99]` on a short list	Prints `null`.
`change FROZEN to 5` after `freeze FROZEN as 1`	Refused when the file is read (from 5.5.1).
`ask n` then `n times 2`	Repeats the text.
A line indented by an odd amount	Read as outside the block above it.

When a program prints something strange and gives no error, one of these is the usual reason.

What to do when you are stuck

1. Read the `What` line, then look at the numbered source lines above it. 2. Check the line just before the one reported. A missing colon or bracket often makes the *next* line look wrong. 3. Cut the program down until the problem is the only thing left, then add lines back. 4. Print the values in play with `show` just before the failing line.

Catching an error yourself: try and fail

From 5.5.2 a program can catch an error instead of stopping. Put the risky lines under `try:` and the recovery under `on error:`. The message of the error is available in the second block under the name you choose after `as` (it is called `error` when you leave `as` out):

PROGRAM

```
type enlng

function risky with n:
    when n is 3:
        fail "three is not allowed"
    give n times 2

try:
    show risky(1)
    show risky(3)
    show "not reached"
on error as problem:
```

```
    show "caught:" problem
show "the program goes on"
```

OUTPUT

```
2
caught: three is not allowed
the program goes on
```

`fail "message"` raises an error of your own. Outside any `try` it stops the program and prints the message like any other error. A `try` block can sit inside a function and inside a loop, can contain another `try`, and `give`, `stop` and `skip` work inside it. A change to a frozen list made inside a function is caught too. In the built-in interpreter, an error the toolchain finds while running (an unknown name, a call with the wrong number of inputs) is caught the same way; with `--native` such a mistake is refused when the program is built, before it runs.

From 5.5.2, dividing by zero and arithmetic on values that are not numbers (such as `"a" minus 1`) are errors that `try` catches, in both modes. In 5.5.1 and earlier they gave a number without any message.

CHAPTER 22

Built-in Function Reference

These functions are part of the language and need no `use` line. They are the same ones the interpreter and the C backend both provide. Everything else in the standard library is ordinary Enlangg code that you import.

Numbers

Call	Gives
`sqrt(x)`	The square root.
`pow(x, y)`	`x` raised to the power `y`.
`abs(x)`	The absolute value.
`floor(x)`	The largest whole number not above `x`.
`ceil(x)`	The smallest whole number not below `x`.
`round(x)`	`x` rounded to the nearest whole number.
`min(a, b)`	The smaller of two numbers.
`max(a, b)`	The larger of two numbers.
`random_number(low, high)`	A random whole number from `low` to `high`, both included.

Text

Call	Gives
`len(text)`	The number of characters.
`to_number(text)`	The number the text spells: `to_number("42")` is 42, `to_number("3.5")` is 3.5. Text that is not a number is an error.
`to_text(value)`	The value as text, the way `show` prints it.
`split(text, separator)`	A list of the pieces. With `""` as separator, one piece per character.
`join(list, separator)`	The text items of the list glued together with the separator.

Call	Gives
`contains(text, piece)`	True if `piece` appears in `text`. Also works on lists.
`reverse(text)`	The text backwards.
`string_set_at(text, position, character)`	The text with the character at `position` replaced.
`string_remove_at(text, position)`	The text without the character at `position`.
`string_remove(text, piece)`	The text with the first `piece` taken out.

`string_add`, `string_insert` and `string_replace` also exist. They take the text, then the piece, then a position (chapter 8). To replace a word rather than a position, the standard library module `lib_strings.enlng` has `string_replace_first`, `string_replace_all`, and others.

Lists

Call	Gives
`len(list)`	The number of items.
`append(list, value)`	Adds `value` at the end.
`pop(list)`	Removes the last item and gives it back; `null` if the list is empty.
`contains(list, value)`	True if the value is in the list.
`reverse(list)`	A reversed copy.
`join(list, separator)`	See above.

Maps

Call	Gives
`keys(map)`	A list of the keys.
`values(map)`	A list of the values.
`has_key(map, key)`	True if the key is present.
`len(map)`	The number of entries.

Files, time and the machine

Call	Gives
`read_file(path)`	The file as text; empty text if it cannot be read.
`write_file(path, text)`	Writes the file, replacing it.
`append_file(path, text)`	Adds to the end of a file.
`file_exists(path)`	True or false.
`time_now()`	Seconds since 1970.
`sleep(seconds)`	Waits.

More built-ins (5.5.2)

Call	Gives
`arguments()`	The words typed after the file name: `enlng run app.enlng one two` gives `["one", "two"]`.
`env("NAME")`	The environment variable, or `null` if it is not set.
`type_of(x)`	`"number"`, `"decimal"`, `"text"`, `"list"`, `"map"`, `"true/false"`, `"function"` or `"null"`.
`char_code(text)`	The number of the first character (Unicode): `char_code("A")` is 65.
`char_of(number)`	The character with that number: `char_of(65)` is `"A"`.
`unique(list)`	A new list with each value once, in the order first seen.
`insert_at(list, position, value)`	Puts `value` before `position`; a negative position counts from the end.
`remove_at(list, position)`	Removes the item and gives it back.
`remove_key(map, key)`	Removes the key; true if it was there.
`check(condition, message)`	Does nothing when the condition is true; otherwise an error "Check failed: message".
`time_ms()`	A clock in milliseconds, for timing a piece of code.

JSON

Call	Gives
`to_json(value)`	The value as JSON text. Maps, lists, text, numbers, true/false and null can be written; a function cannot.
`to_json_pretty(value)`	The same, spread over indented lines.
`from_json(text)`	The map, list or value the JSON describes. Invalid JSON is an error that says where.

Patterns (regular expressions)

A pattern is written in text, so a backslash is doubled: `"\\d+"` means one or more digits. Supported: `.` `[abc]` `[^a-z]` `\\d` `\\w` `\\s` (and `\\D` `\\W` `\\S`) `\\b` `^` `$` `( )` `(?: )` `|` `*` `+` `?` `{2,3}`, lazy forms such as `*?`, and `(?i)` at the start to ignore capitals.

Call	Gives
`regex_match(pattern, text)`	True if the pattern is found anywhere in the text.
`regex_find(pattern, text)`	The first match, or `null`.
`regex_find_all(pattern, text)`	A list of all matches.
`regex_groups(pattern, text)`	The whole match followed by what each `( )` matched, or `null`.
`regex_replace(pattern, text, with)`	Every match replaced. In `with`, `$1` to `$9` are the groups and `$$` is a dollar sign.
`regex_split(pattern, text)`	The pieces between matches.

Dates and times

Call	Gives
`make_time({"year": 2026, "month": 11, "day": 1, "hour": 9})`	Seconds since 1970 for that local date; missing parts are 0 (month and day 1).
`date_parts(seconds)`	A map with `year`, `month`, `day`, `hour`, `minute`, `second`, `weekday` and `day_of_year`.
`format_time(seconds, pattern)`	The time as text, with `%Y` `%m` `%d` `%H` `%M` `%S` `%A` `%B` and the other C patterns.

Folders and other programs

Call	Gives
`is_folder(path)`	True if the path is a folder.
`list_folder(path)`	The names inside, sorted. A missing folder is an error.
`make_folder(path)`	Makes the folder and any missing ones above it.
`delete_file(path)`	Deletes a file or an empty folder; true if it worked.
`rename_file(from, to)`	Renames or moves; true if it worked.
`run_command(command)`	Runs the command and gives `{"code": 0, "output": "..."}`; the output holds everything it printed.
`http_get(url)`	`{"status": 200, "body": "..."}`. An address that cannot be reached is an error; a 404 is not, check `status`.
`http_post(url, body)`	The same, sending the body. A map or list is sent as JSON, anything else as text.

`http_get` and `http_post` use the `curl` program, which comes with Windows 10 and later, macOS and most Linux systems. They need an address that starts with `http://` or `https://`.

Phrases that act like functions

Some summaries are written as phrases instead of calls:

Phrase	Gives
`count of x`	The number of items or characters.
`length of x`	The same.
`sum of list`	The total of a list of numbers.
`average of list` (or `avg of list`)	The mean.
`max of list`, `min of list`	The largest and smallest item.
`first of list`, `last of list`	The ends of the list.
`item at position`	One item.
`key of map`	One entry.

A sampler

PROGRAM

```
type enlng

remember xs as [5, 3, 9]
remember m as {"k": 1}

show sqrt(81)
show pow(3, 4)
show abs(-2)
show floor(2.9) ceil(2.1) round(2.5)
show min(4, 6) max(4, 6)
show len("abc") len(xs) len(m)
show split("a-b-c", "-")
show join(["x", "y"], "/")
show contains("seashell", "shell")
show contains(xs, 9)
show reverse("abc")
append(xs, 1)
show pop(xs)
show keys(m) values(m) has_key(m, "k")
show count of xs sum of xs max of xs first of xs
```

OUTPUT

```
9
81
2
2 3 3
4 6
3 3 1
[a, b, c]
x/y
true
true
cba
1
[k] [1] true
3 17 9 5
```

Where each function lives

Every function listed here is available in both ways of running a program. If a name you expected is missing, such as `upper`, `trim` or `replace`, it is not a built-in; look in `lib_strings.enlng` and `lib_std.enlng`, which are covered in the next part.

CHAPTER 23

The Standard Library: Overview, lib_std and lib_math

The standard library is eleven ordinary Enlangg files in the `lib` folder of the installation. You import one the way you import any file, by its name:

PROGRAM

```
type enlng
use "lib_math.enlng"

show square(9)
```

OUTPUT

```
81
```

Because they are plain Enlangg, you can open them in an editor to see exactly how a function works. They are also a good source of examples of how to write functions.

Module	What it holds
`lib_std.enlng`	Reading numbers, small helpers, and the sort and arrange functions.
`lib_math.enlng`	Maths, number theory, geometry and statistics.
`lib_strings.enlng`	Text helpers: trimming, case, padding, searching, formatting.
`lib_ds.enlng`	Stacks, queues, deques, priority queues, trees, graphs, linked lists, sets.
`lib_file.enlng`	Files, paths, CSV and key-value text.
`lib_fs.enlng`	Short names for file operations.
`lib_db.enlng`	A small in-memory table store with filtering, ordering and joins.
`lib_enlngdb.enlng`	The same shape of API for use with the EnlngDB tool.
`lib_ml.enlng`	Vector arithmetic, activation functions and a few simple predictors.
`lib_net.enlng`	Building and parsing URLs, query strings, headers and request text.

Module	What it holds
`lib_concurrency.enlng`	Task queues, channels and other coordination structures, simulated in plain code.

Two things to keep in mind when you read the next chapters:

- These modules are written in Enlangg, so they run at the speed of the interpreter or the C backend like any other program. They are meant to be clear, not tuned.
- Each function listed here was called by a program in this book and the output printed beside it is real. A function that is not listed is not documented here, and may behave in ways this book has not checked.

lib_std: small helpers

Reading numbers

`ask` gives text. These turn it into numbers.

Function	Gives
`parse_int with text`	The whole number written in the text. A leading `-` makes it negative.
`read_number`	Reads one line and gives it as a number.
`read_numbers_line`	Reads one line of numbers separated by spaces and gives a list.

PROGRAM

```
type enlng
use "lib_std.enlng"

show parse_int with "407"
show parse_int with "-12"
show (parse_int with "21") times 2
```

OUTPUT

```
407
-12
42
```

`parse_int` understands digits only. It does not understand decimals, so `parse_int with "3.5"` is the digits 3 and 5 read as 35.

PROGRAM

```
type enlng
```

```
use "lib_std.enlng"

show parse_int with "3.5"
```

OUTPUT

```
35
```

With input supplied:

PROGRAM

```
type enlng
use "lib_std.enlng"

remember nums as read_numbers_line()
show nums
show sum of nums
```

OUTPUT

```
[4, 8, 15, 16]
43
```

Number helpers

Function	Gives
`abs_val with n`	The absolute value.
`difference with a, b`	How far apart `a` and `b` are, never negative.
`max_of with a, b`	The larger.
`min_of with a, b`	The smaller.
`is_even with n`	True when `n` is even.
`is_odd with n`	True when `n` is odd.
`sum_of_list with items`	The total of a list.

PROGRAM

```
type enlng
use "lib_std.enlng"

show abs_val with -9
show difference with 3, 10
show max_of with 4, 7
show min_of with 4, 7
show is_even with 10
show is_odd with 10
show sum_of_list with [1, 2, 3, 4]
```

OUTPUT

```
9
7
```

```
7
4
true
false
10
```

Sorting and arranging as functions

`sort_list` sorts a list with the swap-pair method and gives it back:

PROGRAM

```
type enlng
use "lib_std.enlng"

show sort_list with [5, 2, 8, 1]
```

OUTPUT

```
[1, 2, 5, 8]
```

The file also has `arrange_text(text, positions)` and `arrange_items(list, positions)`, the function forms of the `arrange` statement from chapter 18. They build a new text or list from the positions you name, and a negative position counts from the end. (In 5.5.0 they were named `arrange` and `arrange_list`, which clashed with the language word and could not be called.)

PROGRAM

```
type enlng
use "lib_std.enlng"

show arrange_text("abcdef", [5, 4, 3])
show arrange_items([10, 20, 30], [2, 0])
show arrange_text("abcdef", [-1, 0])
```

OUTPUT

```
fed
[30, 10]
fa
```

Cells and references

Lists and maps are already shared when you pass them, so you rarely need these. They exist for programs that want an explicit box around one value, so a function can change it:

Function	Gives
`make_cell with val`	A box holding `val`.
`get_cell with cell`	The value in the box.
`set_cell with cell, new_val`	Puts a new value in the box.

Function	Gives
`swap_cells with a, b`	Swaps the contents of two boxes.
`pointer with val`, `get_pointer`, `set_pointer`, `swap_pointers`	The same idea with a named box.

PROGRAM

```
type enlng
use "lib_std.enlng"

remember a as make_cell(1)
remember b as make_cell(2)
swap_cells(a, b)
show get_cell(a) get_cell(b)

remember p as pointer(10)
set_pointer(p, 25)
show get_pointer(p)
```

OUTPUT

```
2 1
25
```

lib_math: maths and statistics

Function	Gives
`square(x)`, `cube(x)`	`x` squared and cubed.
`pythagoras(a, b)`, `hypotenuse(a, b)`	The long side of a right triangle.
`math_power(base, exp)`, `math_sqrt(v)`, `math_abs(v)`	Names for the built-ins, for programs that prefer the `math_` prefix.
`math_floor(v)`, `math_ceil(v)`, `math_round(v)`	Likewise.
`clamp(val, low, high)`	`val`, but no lower than `low` and no higher than `high`.
`factorial(n)`	`n` factorial, by a loop.
`gcd(a, b)`, `lcm(a, b)`	Greatest common divisor and least common multiple.
`is_prime(n)`	True when `n` is prime.
`distance_2d(x1, y1, x2, y2)`	Straight-line distance between two points.

Function	Gives
`polygon_area(xs, ys)`	Area of a polygon from its corner coordinates.
`math_sum`, `math_mean`, `math_variance`, `math_std_dev`	Statistics of a list.
`math_min_list`, `math_max_list`	Smallest and largest of a list; 0 for an empty list.

PROGRAM

```
type enlng
use "lib_math.enlng"

show square(12) cube(3)
show hypotenuse(3, 4)
show clamp(15, 0, 10) clamp(-5, 0, 10) clamp(4, 0, 10)
show factorial(6)
show gcd(84, 36) lcm(4, 6)
show is_prime(97) is_prime(91)
show distance_2d(0, 0, 6, 8)
```

OUTPUT

```
144 27
5
10 0 4
720
12 12
true false
10
```

Statistics treat the list as the whole population. The variance is divided by the number of items, not by one less:

PROGRAM

```
type enlng
use "lib_math.enlng"

remember data as [2, 4, 4, 4, 5, 5, 7, 9]
show math_sum(data)
show math_mean(data)
show math_variance(data)
show math_std_dev(data)
show math_min_list(data) math_max_list(data)
```

OUTPUT

```
40
5
4
2
```

```
2 9
```

The area of a polygon comes from its corners, listed in order around the shape. A 4 by 3 rectangle:

PROGRAM

```
type enlng
use "lib_math.enlng"

show polygon_area([0, 4, 4, 0], [0, 0, 3, 3])
```

OUTPUT

```
12
```

A caution about `lcm`

`lcm` computes `a * b / gcd`. With whole numbers the division is whole-number division, which is exact here because the gcd always divides the product, so the result is correct for numbers whose product fits in 64 bits.

CHAPTER 24

The Standard Library: lib_strings

`lib_strings.enlng` fills the gaps left by the built-in text functions: trimming, case conversion, padding, searching by position, and formatting. Import it at the top of a program:

```
use "lib_strings.enlng"
```

Positions count from zero everywhere in this module. A "range" is written as a start position and an end position, where the end is not included (so `0, 5` is the first five characters), except where the table says it is a start and a count.

Measuring and testing

Function	Gives
`string_length(text)`	The number of characters.
`string_contains(text, piece)`	True when `piece` is somewhere in `text`.
`string_starts_with(text, prefix)`	True when the text begins with `prefix`.
`string_ends_with(text, suffix)`	True when it ends with `suffix`.
`string_index_of(text, piece)`	The position of the first match, or -1.
`string_count_char(text, ch)`	How many times one character appears.
`count_occurrences(text, piece)`	How many times a piece appears.
`is_palindrome(text)`	True when the text reads the same backwards.

PROGRAM

```
type enlng
use "lib_strings.enlng"

show string_length("hello")
show string_contains("hello", "ell")
show string_starts_with("hello", "he") string_ends_with("hello", "
    lo")
show string_index_of("hello", "l")
show string_index_of("hello", "z")
show string_count_char("banana", "a")
show count_occurrences("banana", "an")
show is_palindrome("level") is_palindrome("hello")
```

OUTPUT

```
5
true
true true
2
-1
3
2
true false
```

Taking pieces

Function	Gives
`char_at(text, position)`	One character.
`sub_string(text, start, end)`	The characters from `start` up to but not including `end`.
`string_substring(text, start, end)`	The same.
`string_slice(text, start, count)`	`count` characters starting at `start`.
`string_split(text, separator)`	The same as the built-in `split`.
`string_join(list, separator)`	The same as the built-in `join`.

PROGRAM

```
type enlng
use "lib_strings.enlng"

remember s as "hello world"
show char_at(s, 1)
show sub_string(s, 0, 5)
show string_substring(s, 6, 11)
show string_slice(s, 6, 5)
```

OUTPUT

```
e
hello
world
world
```

From 5.5.2 the language itself has slices: `s[0:5]` is the same as `sub_string(s, 0, 5)` (chapter 8).

Trimming

Function	Gives
`trim(text)`	The text without spaces at both ends.
`trim_left(text)`, `trim_right(text)`	Without spaces at one end.
`strip_prefix(text, prefix)`	Without `prefix` if it begins the text.
`strip_suffix(text, suffix)`	Without `suffix` if it ends the text.

PROGRAM

```
type enlng
use "lib_strings.enlng"

show "[" + trim("   padded   ") + "]"
show "[" + trim_left("   padded") + "]"
show "[" + trim_right("padded   ") + "]"
show strip_prefix("unhappy", "un")
show strip_suffix("report.txt", ".txt")
```

OUTPUT

```
[padded]
[padded]
[padded]
happy
report
```

Case

Function	Gives
`to_upper(text)`, `to_lower(text)`	All capitals, all small letters.
`capitalize(text)`	The first letter in capitals, the rest as it was.
`title_case(text)`	Each word starting with a capital.
`to_snake_case(text)`	`words_joined_by_underscores`.
`to_kebab_case(text)`	`words-joined-by-hyphens`.
`to_camel_case(text)`	`wordsJoinedWithCapitals`.
`char_to_upper(ch)`, `char_to_lower(ch)`	Case of one character.

PROGRAM

```
type enlng
use "lib_strings.enlng"

show to_upper("Hello") to_lower("Hello")
show capitalize("hello world")
show title_case("hello big world")
```

```
show to_snake_case("Hello Big World")
show to_kebab_case("Hello Big World")
show to_camel_case("hello big world")
```

OUTPUT

```
HELLO hello
Hello world
Hello Big World
hello_big_world
hello-big-world
helloBigWorld
```

Padding and repeating

`pad_left` and `pad_right` make text a fixed width, which is how you line up columns:

PROGRAM

```
type enlng
use "lib_strings.enlng"

show repeat_char("ab", 3)
show pad_left("7", 3, "0")
show pad_right("7", 3, ".")
show "Item " + pad_right("tea", 8, ".") + pad_left("25", 5, " ")
show "Item " + pad_right("coffee", 8, ".") + pad_left("120", 5, "
    ")
```

OUTPUT

```
ababab
007
7..
Item tea.....   25
Item coffee..  120
```

Replacing

Function	Gives
`string_replace_first(text, target, replacement)`	The text with the first `target` replaced.
`string_replace_all(text, target, replacement)`	The text with every `target` replaced.

PROGRAM

```
type enlng
use "lib_strings.enlng"

show string_replace_first("a-b-c", "-", "+")
show string_replace_all("a-b-c", "-", "+")
```

```
show string_replace_all("one  two", "  ", " ")
```

OUTPUT

```
a+b-c
a+b+c
one two
```

Filling in a template

`string_format_1`, `string_format_2` and `string_format_3` fill numbered holes `{0}`, `{1}` and `{2}` in a template:

PROGRAM

```
type enlng
use "lib_strings.enlng"

show string_format_1("Hello, {0}!", "Asha")
show string_format_2("{0} + {1} = 3", 1, 2)
show string_format_3("{2} {1} {0}", "a", "b", "c")
```

OUTPUT

```
Hello, Asha!
1 + 2 = 3
c b a
```

A hole written `{}` without a number is not filled in.

Replacing at the ends of a string

These functions, and `string_replace_all`, are built on the built-in `split`. In 5.5.0 `split` left out empty pieces, and a replacement at the very start or end of the text, or two in a row, was lost. From 5.5.1 `split` keeps them and all of these come out right:

PROGRAM

```
type enlng
use "lib_strings.enlng"

show string_replace_all("a-b-c", "-", "+")
show string_replace_all("-a-", "-", "+")
show string_replace_all("a--b", "-", "+")
show string_format_2("{0} + {1}", 1, 2)
show string_format_2("x {0} y {1}", 1, 2)
```

OUTPUT

```
a+b+c
+a+
a++b
1 + 2
x 1 y 2
```

`string_replace_first` never used `split` and was not affected.

Small helpers for output

`banner(title)` makes a heading line and `csv_row` joins three or four columns with commas:

PROGRAM

```
type enlng
use "lib_strings.enlng"

show banner("Report")
show csv_row("name", "age", "city")
show csv_row_4("a", "b", "c", "d")
```

OUTPUT

```
=== Report ===
name,age,city
a,b,c,d
```

rot13 and slugs

`rot13` swaps each letter with the one thirteen places along, so applying it twice gives back the original. It is a toy cipher and is not secret in any real sense. `slugify` is another name for `to_kebab_case`; it keeps punctuation, so clean the text first if you need a web-safe slug.

PROGRAM

```
type enlng
use "lib_strings.enlng"

show rot13("Hello")
show rot13(rot13("Hello"))
show slugify("Hello World Again")
```

OUTPUT

```
Uryyb
Hello
hello-world-again
```

Predictable names for the built-in helpers

The module also defines `string_arrange(text, positions)`, a function version of the `arrange` statement that does work as a function, and wrappers for the position-based built-ins. Use the functions in this chapter when you want to work with words; the built-ins `string_add`, `string_insert` and `string_replace` (chapter 8) work by position.

PROGRAM

```
type enlng
use "lib_strings.enlng"

show string_arrange("abcdef", [5, 4, 3])
```

OUTPUT

```
fed
```

CHAPTER 25

The Standard Library: lib_ds (Data Structures)

`lib_ds.enlng` builds the classic data structures out of lists and maps. Each structure is created with a `create_` function, and you pass it to the other functions of that structure. The structures are ordinary values, so you can keep several at once.

```
use "lib_ds.enlng"
```

Stack: last in, first out

A stack hands things back in the reverse of the order they went in, like a pile of plates.

Function	Does
`create_stack()`	Makes an empty stack.
`stack_push(s, val)`	Puts `val` on top.
`stack_pop(s)`	Removes and gives back the top; `null` when empty.
`stack_peek(s)`	The top, without removing it; `null` when empty.
`stack_is_empty(s)`, `stack_size(s)`	Whether it is empty, and how many items.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember plates as create_stack()
stack_push(plates, "blue")
stack_push(plates, "red")
show stack_peek(plates) stack_size(plates)
show stack_pop(plates)
show stack_pop(plates)
show stack_is_empty(plates)
show stack_pop(plates)
```

OUTPUT

```
red 2
```

```
red
blue
true
null
```

A classic use of a stack is checking that brackets match. Each opening bracket is pushed, each closing one pops:

PROGRAM

```
type enlng
use "lib_ds.enlng"

function balanced with text:
    remember s as create_stack()
    for each ch in split(text, ""):
        when ch is "(":
            stack_push(s, ch)
        when ch is ")":
            when stack_is_empty(s):
                give false
            stack_pop(s)
    give stack_is_empty(s)

show balanced("(a (b) c)")
show balanced("(a (b c)")
show balanced("a) (b")
```

OUTPUT

```
true
false
false
```

Queue: first in, first out

A queue serves things in the order they arrived, like a line at a counter.

Function	Does
`create_queue()`	Makes an empty queue.
`queue_enqueue(q, val)`	Joins the back of the line.
`queue_dequeue(q)`	Leaves from the front and gives the item; `null` when empty.
`queue_peek(q)`	The front item, without removing it.
`queue_is_empty(q)`, `queue_size(q)`	Emptiness and size.

PROGRAM

```
type enlng
```

```
use "lib_ds.enlng"

remember line as create_queue()
queue_enqueue(line, "Asha")
queue_enqueue(line, "Ravi")
queue_enqueue(line, "Meera")
show queue_peek(line) queue_size(line)
show queue_dequeue(line)
show queue_dequeue(line)
show queue_size(line)
```

OUTPUT

```
Asha 3
Asha
Ravi
1
```

Deque: both ends

A deque (pronounced "deck") is a list you can add to and remove from at either end.

Function	Does
`create_deque()`	Makes an empty deque.
`deque_push_back(d, v)`, `deque_push_front(d, v)`	Adds at the back or the front.
`deque_pop_back(d)`, `deque_pop_front(d)`	Removes from the back or the front.
`deque_peek_back(d)`, `deque_peek_front(d)`	Looks at either end.
`deque_is_empty(d)`, `deque_size(d)`	Emptiness and size.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember d as create_deque()
deque_push_back(d, 1)
deque_push_front(d, 0)
deque_push_back(d, 2)
show deque_peek_front(d) deque_peek_back(d) deque_size(d)
show deque_pop_front(d) deque_pop_back(d)
show deque_size(d)
```

OUTPUT

```
0 2 3
0 2
1
```

Priority queue: smallest first

A priority queue always gives you the smallest value next, whatever order they were added in.

Function	Does
`create_priority_queue()`	Makes an empty one.
`pq_insert(pq, val)`	Adds a value.
`pq_peek_min(pq)`	The smallest value, without removing it.
`pq_extract_min(pq)`	Removes and gives the smallest; `null` when empty.
`pq_size(pq)`, `pq_is_empty(pq)`	Size and emptiness.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember tasks as create_priority_queue()
pq_insert(tasks, 5)
pq_insert(tasks, 1)
pq_insert(tasks, 3)
show pq_peek_min(tasks)
show pq_extract_min(tasks) pq_extract_min(tasks) pq_size(tasks)
```

OUTPUT

```
1
1 3 1
```

Binary search tree

A binary search tree keeps numbers so that the smaller ones hang to the left of a node and the larger to the right, which makes searching quick.

Function	Does
`bst_node(val)`	Makes a tree with one value; it is the root.
`bst_insert(root, val)`	Adds a value below the root.
`bst_search(root, target)`	True when the value is in the tree.
`bst_min(root)`, `bst_max(root)`	The smallest and largest values.

PROGRAM

```
type enlng
use "lib_ds.enlng"
```

```
remember root as bst_node(50)
for each v in [30, 70, 20, 40, 60, 80]:
    bst_insert(root, v)
show bst_search(root, 40) bst_search(root, 45)
show bst_min(root) bst_max(root)
```

OUTPUT

```
true false
20 80
```

Graph

A graph stores connections between named points. `graph_bfs` and `graph_dfs` list the points reachable from a start, breadth-first and depth-first.

Function	Does
`create_graph()`	Makes an empty graph.
`graph_add_node(g, id)`	Adds a point.
`graph_add_edge(g, u, v)`	Connects two points.
`graph_neighbors(g, u)`	The points directly connected to `u`.
`graph_has_edge(g, u, v)`	True when `u` and `v` are connected.
`graph_bfs(g, start)`, `graph_dfs(g, start)`	Points reachable from `start`, in the order visited.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember map as create_graph()
for each city in ["Pune", "Mumbai", "Nashik", "Goa"]:
    graph_add_node(map, city)
graph_add_edge(map, "Pune", "Mumbai")
graph_add_edge(map, "Mumbai", "Nashik")
graph_add_edge(map, "Mumbai", "Goa")
show graph_neighbors(map, "Mumbai")
show graph_has_edge(map, "Pune", "Mumbai")
show graph_has_edge(map, "Pune", "Goa")
show graph_bfs(map, "Pune")
show graph_dfs(map, "Pune")
```

OUTPUT

```
[Nashik, Goa]
true
false
[Pune, Mumbai, Nashik, Goa]
[Pune, Mumbai, Goa, Nashik]
```

Linked lists

A linked list chains items together so that adding at the front is as cheap as adding at the back. A doubly linked list can also be read backwards.

Function	Does
`create_linked_list()`	Makes an empty list.
`ll_append(l, v)`, `ll_prepend(l, v)`	Adds at the end or the front.
`ll_find(l, v)`	True when the value is present.
`ll_to_list(l)`, `ll_size(l)`	The items as an ordinary list, and the count.
`create_doubly_linked_list()`	The doubly linked kind.
`dll_append`, `dll_prepend`, `dll_to_list`, `dll_to_list_reverse`, `dll_size`	The same operations, plus reading backwards.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember chain as create_linked_list()
ll_append(chain, 1)
ll_append(chain, 2)
ll_prepend(chain, 0)
show ll_to_list(chain) ll_size(chain)
show ll_find(chain, 2) ll_find(chain, 9)

remember both as create_doubly_linked_list()
dll_append(both, "x")
dll_append(both, "y")
dll_prepend(both, "w")
show dll_to_list(both)
show dll_to_list_reverse(both)
```

OUTPUT

```
[0, 1, 2] 3
true false
[w, x, y]
[y, x, w]
```

Sets

A set holds each value once, however many times you add it.

Function	Does
`create_set()`	Makes an empty set.

Function	Does
`set_add(s, item)`	Adds an item (ignored if already present).
`set_has(s, item)`	True when the item is present.
`set_size(s)`, `set_to_list(s)`	Size, and the items as a list.
`set_union(a, b)`, `set_intersection(a, b)`, `set_difference(a, b)`	New sets from two sets.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember a as create_set()
set_add(a, 1)
set_add(a, 2)
set_add(a, 2)
remember b as create_set()
set_add(b, 2)
set_add(b, 3)
show set_size(a) set_has(a, 2) set_has(a, 9)
show set_to_list(set_union(a, b))
show set_to_list(set_intersection(a, b))
show set_to_list(set_difference(a, b))
```

OUTPUT

```
2 true false
[1, 2, 3]
[2]
[1]
```

`set_remove(s, item)` takes an item out. (In 5.5.0 it left a `null` in its place, which then showed up in `set_to_list`.)

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember s as create_set()
set_add(s, 1)
set_add(s, 2)
set_add(s, 4)
set_remove(s, 2)
show set_to_list(s)
show set_size(s) set_has(s, 2) set_has(s, 4)
```

OUTPUT

```
[1, 4]
2 false true
```

Trie

A trie stores words so that you can ask which words start with some letters.

Function	Does
`create_trie()`	Makes an empty trie.
`trie_insert(t, word)`	Adds a word.
`trie_search(t, word)`	True when exactly this word was added.
`trie_starts_with(t, prefix)`	True when some word begins with `prefix`.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember words as create_trie()
trie_insert(words, "car")
trie_insert(words, "cat")
show trie_search(words, "car")
show trie_search(words, "ca")
show trie_starts_with(words, "ca")
show trie_starts_with(words, "d")
```

OUTPUT

```
true
false
true
false
```

Ring buffer

A ring buffer holds a fixed number of items. Once it is full, new pushes are ignored in this version.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember rb as create_ring_buffer(2)
ring_push(rb, 1)
ring_push(rb, 2)
show ring_is_full(rb) ring_size(rb)
ring_push(rb, 3)
show ring_pop(rb) ring_pop(rb) ring_is_empty(rb)
```

OUTPUT

```
true 2
1 2 true
```

A note on the cache

The module also has `create_lru_cache`, `lru_get`, `lru_put` and `lru_size`. A least-recently-used cache should forget its oldest entry when it is full. It does that in 5.5.1, but `lru_size` reports a count that includes forgotten entries, so use `lru_get` to ask about a key instead of counting.

PROGRAM

```
type enlng
use "lib_ds.enlng"

remember cache as create_lru_cache(2)
lru_put(cache, "a", 1)
lru_put(cache, "b", 2)
lru_put(cache, "c", 3)
show lru_get(cache, "a")
show lru_get(cache, "b")
show lru_get(cache, "c")
```

OUTPUT

```
null
2
3
```

CHAPTER 26

The Standard Library: lib_file and lib_fs (Files)

The built-in file functions (`read_file`, `write_file`, `append_file`, `file_exists`) are in chapter 22. `lib_file.enlng` adds line-based reading, path helpers, CSV and key=value text on top of them. `lib_fs.enlng` is a set of short names for the same basics.

```
use "lib_file.enlng"
use "lib_fs.enlng"
```

Files are written to the folder you run the program from unless you give a path. The examples below write small files next to the program.

Reading and writing

Function	Does
`file_read(path)`	The whole file as text.
`file_write(path, text)`	Replaces the file with `text`.
`file_append(path, text)`	Adds `text` at the end.
`file_append_line(path, line)`	Adds a line at the end.
`file_write_lines(path, lines)`	Writes a list of lines, one per line.
`file_read_lines(path)`	The lines as a list.
`file_exists_check(path)`	True when the file exists.
`file_copy(source, destination)`	Copies a file.
`file_line_count(path)`	The number of lines.

PROGRAM

```
type enlng
use "lib_file.enlng"

file_write("demo.txt", "alpha\nbeta\ngamma\n")
file_append_line("demo.txt", "delta")
show file_read_lines("demo.txt")
show file_line_count("demo.txt")
file_copy("demo.txt", "demo_copy.txt")
show file_exists_check("demo_copy.txt")
```

```
show file_exists_check("never_made.txt")
```

OUTPUT

```
[alpha, beta, gamma, delta]
4
true
false
```

`file_read_lines` drops the trailing `\r` that Windows adds at the end of lines, so files made on either system read the same.

Counting words

`file_word_count` counts the words in a file. A word ends at a space or at the end of a line:

PROGRAM

```
type enlng
use "lib_file.enlng"

file_write("w.txt", "one two three\nfour five\n")
show file_word_count("w.txt")
show file_line_count("w.txt")
```

OUTPUT

```
5
2
```

In 5.5.0 only spaces ended a word, so the last word of one line and the first of the next were counted as one and this file gave 4. A file that ends with a line break also has no extra empty line at the end in `file_read_lines` and `file_line_count`.

Paths

These work on the text of a path. They do not look at the disk.

Function	Gives
`path_basename(path)`	The file name at the end.
`path_dirname(path)`	Everything before the file name.
`path_extension(path)`	The text after the last dot, without the dot.
`path_join(folder, name)`	The two joined with a slash.

PROGRAM

```
type enlng
use "lib_file.enlng"
```

```
show path_basename("a/b/report.txt")
show path_dirname("a/b/report.txt")
show path_extension("a/b/report.txt")
show path_join("a/b", "x.txt")
```

OUTPUT

```
report.txt
a/b
txt
a/b/x.txt
```

CSV text

`csv_parse_records(text)` turns comma-separated text with a header row into a list of maps, one per row. Values are read as numbers where they look like numbers. `csv_serialize_records(headers, records)` does the reverse and needs the records as maps:

PROGRAM

```
type enlng
use "lib_file.enlng"

remember rows as csv_parse_records("name,age\nAsha,30\nRavi,25\n")
show len(rows)
show rows[0]["name"] rows[1]["age"]

remember out as csv_serialize_records(["name", "age"], [{"name": "
    Asha", "age": 30}, {"name": "Ravi", "age": 25}])
show out
```

OUTPUT

```
2
Asha 25
name,age
Asha,30
Ravi,25
```

Giving `csv_serialize_records` rows as plain lists instead of maps produces empty cells, so use maps. The functions do not handle quoted fields or commas inside a value.

Key=value text

Settings files are often one `key=value` per line. `kv_parse_text` reads that into a map and `kv_serialize_map` writes a map back:

PROGRAM

```
type enlng
use "lib_file.enlng"
```

```
remember settings as kv_parse_text("volume=7\nname=lamp\n")
show settings["volume"]
show settings["name"]

show kv_serialize_map({"a": 1, "b": 2})
```

OUTPUT

```
7
lamp
a=1
b=2
```

lib_fs: shorter names

Function	Same as
`fs_read(path)`	`file_read`
`fs_write(path, data)`	`file_write`
`fs_append(path, data)`	`file_append`
`fs_exists(path)`	`file_exists_check`
`fs_copy(source, destination)`	`file_copy`

PROGRAM

```
type enlng
use "lib_fs.enlng"

fs_write("z.txt", "hi")
fs_append("z.txt", " there")
show fs_read("z.txt")
show fs_exists("z.txt")
```

OUTPUT

```
-- Enlang SyntaxError -------------------------------------------
    -------
Location: program.enlng:10


What: There are two functions called 'file_read_lines_str'. Rename
     one of them. If one comes from a file you used, give that fil
    e a name instead: use "file.enlng" as tools, then call tools.f
    ile_read_lines_str(...).
Why:  The compiler encountered an unexpected token during syntax e
    valuation.

Suggestions:
  * Check statement syntax against Enlang language conventions.
------------------------------------------------------------------
    ------
```

A small program: a notes file

PROGRAM

```
type enlng
use "lib_file.enlng"

remember notes_file as "notes.txt"
file_write(notes_file, "")
for each note in ["buy milk", "call Asha", "pay rent"]:
    file_append_line(notes_file, note)

show "Saved" file_line_count(notes_file) "notes:"
remember n as 1
for each line in file_read_lines(notes_file):
    show n + "." line
    n increases by 1
```

OUTPUT

```
Saved 3 notes:
1. buy milk
2. call Asha
3. pay rent
```

CHAPTER 27

The Standard Library: lib_db (A Small Table Store)

`lib_db.enlng` is a tiny database that lives in memory while your program runs. A database holds named tables, a table holds rows, and a row is a map from column name to value. Nothing is saved to disk: when the program ends, the data is gone (use `lib_file` to save it, or the separate EnlngDB tool described in a later chapter).

```
use "lib_db.enlng"
```

Tables and rows

Function	Does
`db_connect(name)`	Makes a new empty database.
`db_create_table(db, table, columns)`	Adds a table with the given column names.
`db_drop_table(db, table)`	Removes a table.
`db_list_tables(db)`	The table names.
`db_insert(db, table, row)`	Adds a row (a map). It gets an `id` column automatically, starting at 1.
`db_count(db, table)`	The number of rows.
`db_find_all(db, table)`	All rows as a list of maps.

Rows are maps, and maps have no fixed order, so when you print a whole row the keys can come out in any order. The examples here print individual fields to keep the output steady.

PROGRAM

```
type enlng
use "lib_db.enlng"

remember db as db_connect("shop")
db_create_table(db, "items", ["name", "price", "qty"])
db_insert(db, "items", {"name": "tea", "price": 25, "qty": 10})
db_insert(db, "items", {"name": "coffee", "price": 120, "qty": 3})
db_insert(db, "items", {"name": "milk", "price": 30, "qty": 0})
```

```
show db_list_tables(db)
show db_count(db, "items")
for each row in db_find_all(db, "items"):
    show row["id"] row["name"] row["price"] row["qty"]
```

OUTPUT

```
[items]
3
1 tea 25 10
2 coffee 120 3
3 milk 30 0
```

Searching

`db_find_where(db, table, column, operator, value)` gives the matching rows. The operator is written as text: `"=="`, `"!="`, `"<"`, `">"`, `"<="`, `">="`, or `"contains"` (true when the column's text includes the value's text). A single `"="` also works as `"=="`. `db_find_one` gives the first row where a column equals a value.

PROGRAM

```
type enlng
use "lib_db.enlng"

remember db as db_connect("shop")
db_create_table(db, "items", ["name", "price", "qty"])
db_insert(db, "items", {"name": "tea", "price": 25, "qty": 10})
db_insert(db, "items", {"name": "coffee", "price": 120, "qty": 3})
db_insert(db, "items", {"name": "milk", "price": 30, "qty": 0})

for each row in db_find_where(db, "items", "price", ">", 26):
    show row["name"]
show "out of stock:"
for each row in db_find_where(db, "items", "qty", "==", 0):
    show row["name"]
remember tea as db_find_one(db, "items", "name", "tea")
show tea["price"]
```

OUTPUT

```
coffee
milk
out of stock:
milk
25
```

Changing and removing

Function	Does
`db_update_where(db, table, column, op, value, set_column, new_value)`	Sets a column on every matching row.
`db_delete_where(db, table, column, op, value)`	Removes every matching row.

PROGRAM

```
type enlng
use "lib_db.enlng"

remember db as db_connect("shop")
db_create_table(db, "items", ["name", "price", "qty"])
db_insert(db, "items", {"name": "tea", "price": 25, "qty": 10})
db_insert(db, "items", {"name": "coffee", "price": 120, "qty": 3})
db_insert(db, "items", {"name": "milk", "price": 30, "qty": 0})

db_update_where(db, "items", "name", "==", "milk", "qty", 12)
show db_find_one(db, "items", "name", "milk")["qty"]

db_delete_where(db, "items", "qty", "<", 5)
show db_count(db, "items")
```

OUTPUT

```
12
2
```

Ordering, trimming and picking columns

These work on a list of rows, usually the result of one of the finders.

Function	Does
`db_order_by(rows, column, ascending)`	Sorted rows. Pass `true` for smallest first, `false` for largest first.
`db_limit(rows, count)`	The first `count` rows.
`db_select_columns(rows, columns)`	Rows reduced to the listed columns.

PROGRAM

```
type enlng
use "lib_db.enlng"

remember db as db_connect("shop")
db_create_table(db, "items", ["name", "price"])
db_insert(db, "items", {"name": "tea", "price": 25})
db_insert(db, "items", {"name": "coffee", "price": 120})
db_insert(db, "items", {"name": "milk", "price": 30})
```

```
show "dearest first:"
for each row in db_order_by(db_find_all(db, "items"), "price", fal
    se):
    show row["name"] row["price"]

show "two cheapest:"
for each row in db_limit(db_order_by(db_find_all(db, "items"), "pr
    ice", true), 2):
    show row["name"]

show db_select_columns(db_find_all(db, "items"), ["name"])
```

OUTPUT

```
dearest first:
coffee 120
milk 30
tea 25
two cheapest:
tea
milk
[{"name": tea}, {"name": coffee}, {"name": milk}]
```

Joining two tables

`db_inner_join(rows_a, rows_b, key_a, key_b)` pairs up rows whose key columns are equal and merges them:

PROGRAM

```
type enlng
use "lib_db.enlng"

remember db as db_connect("shop")
db_create_table(db, "items", ["name", "price"])
db_insert(db, "items", {"name": "tea", "price": 25})
db_insert(db, "items", {"name": "coffee", "price": 120})
db_create_table(db, "makers", ["item", "maker"])
db_insert(db, "makers", {"item": "tea", "maker": "Asha"})

for each row in db_inner_join(db_find_all(db, "items"), db_find_al
    l(db, "makers"), "name", "item"):
    show row["name"] row["price"] row["maker"]
```

OUTPUT

```
tea 25 Asha
```

Only coffee is missing from the output, because no row in `makers` mentions it.

Transactions

`db_transaction_begin`, `db_transaction_commit` and `db_transaction_rollback` let you try changes and take them back:

PROGRAM

```
type enlng
use "lib_db.enlng"

remember db as db_connect("shop")
db_create_table(db, "items", ["name"])
db_insert(db, "items", {"name": "tea"})

db_transaction_begin(db)
db_insert(db, "items", {"name": "extra"})
show "inside:" db_count(db, "items")
db_transaction_rollback(db)
show "after rollback:" db_count(db, "items")
```

OUTPUT

```
inside: 2
after rollback: 1
```

Exporting to CSV

`db_export_csv(db, table)` gives the table as comma-separated text, header row first:

PROGRAM

```
type enlng
use "lib_db.enlng"

remember db as db_connect("shop")
db_create_table(db, "items", ["name", "price", "qty"])
db_insert(db, "items", {"name": "tea", "price": 25, "qty": 10})
db_insert(db, "items", {"name": "milk", "price": 30, "qty": 12})
show db_export_csv(db, "items")
```

OUTPUT

```
name,price,qty
tea,25,10
milk,30,12
```

Pair it with `file_write` from chapter 26 to keep the data between runs.

What this is and is not

It is a convenient way to hold and query rows in a program. It is not a server, it does not write to disk by itself, it has no indexes, and every search looks at every row. For a few thousand rows that is fine.

CHAPTER 28

The Standard Library: lib_ml and lib_net

lib_ml: vectors and small predictors

`lib_ml.enlng` holds the arithmetic that machine-learning code is made of. It is not a training framework: there is no model fitting, no tensors and no GPU. It is a handful of plain functions on lists of numbers, useful for teaching and for small calculations.

```
use "lib_ml.enlng"
```

A vector is just a list of numbers.

Function	Gives
`vec_add(a, b)`, `vec_sub(a, b)`	The sum and difference, item by item.
`vec_scale(v, k)`	Every item multiplied by `k`.
`vec_dot(a, b)`	The dot product: multiply item by item, then add up.
`vec_norm(v)`	The length of the vector.
`euclidean_distance(a, b)`	The straight-line distance between two points.
`cosine_similarity(a, b)`	1 when they point the same way, 0 when at right angles.

PROGRAM

```
type enlng
use "lib_ml.enlng"

show vec_add([1, 2], [3, 4])
show vec_sub([5, 5], [1, 2])
show vec_scale([1, 2, 3], 2)
show vec_dot([1, 2, 3], [4, 5, 6])
show vec_norm([3, 4])
show euclidean_distance([0, 0], [3, 4])
show cosine_similarity([1, 0], [1, 0])
show cosine_similarity([1, 0], [0, 1])
```

OUTPUT

```
[4, 6]
```

```
[4, 3]
[2, 4, 6]
32
5
5
1
0
```

Activation functions and scaling

Function	Gives
`sigmoid(z)`	A smooth curve from 0 to 1; `sigmoid(0)` is 0.5.
`relu(z)`	`z` when positive, otherwise 0.
`leaky_relu(z, alpha)`	`z` when positive, otherwise `alpha` times `z`.
`min_max_normalize(values)`	The list rescaled so the smallest is 0 and the largest is 1.

PROGRAM

```
type enlng
use "lib_ml.enlng"

show sigmoid(0)
show relu(-3) relu(4)
show leaky_relu(-10, 0.1)
show min_max_normalize([10, 20, 30])
```

OUTPUT

```
0.5
0 4
-1
[0, 0.5, 1]
```

Three small predictors

`linear_regression_predict(features, weights, bias)` multiplies each feature by its weight, adds them up, then adds the bias. You supply the weights; it does not learn them:

PROGRAM

```
type enlng
use "lib_ml.enlng"

show linear_regression_predict([2, 3], [0.5, 1.5], 1)
```

OUTPUT

```
6.5
```

That is 2 times 0.5, plus 3 times 1.5, plus 1.

`gini_impurity_binary(p1, p2)` measures how mixed two classes are: 0 for a pure group, 0.5 for an even split.

`predict_knn1(points, x, y)` finds the nearest point and returns its label. Each point is a map with `x`, `y` and `label`:

PROGRAM

```
type enlng
use "lib_ml.enlng"

remember points as [{"x": 0, "y": 0, "label": "cat"}, {"x": 10, "y
    ": 10, "label": "dog"}]
show predict_knn1(points, 9, 8)
show predict_knn1(points, 1, 2)
show gini_impurity_binary(0.5, 0.5)
```

OUTPUT

```
dog
cat
0.5
```

Points given as lists such as `[0, 0, "cat"]` do not work here; the function reads the names `x`, `y` and `label`.

lib_net: building and reading web text

`lib_net.enlng` works on the text that web programs exchange: addresses, query strings, headers and request lines. It does not open network connections itself. To send a request, use the built-in `http_get` and `http_post` (chapter 22); this module prepares and reads the text around them.

```
use "lib_net.enlng"
```

Addresses

`parse_url` breaks an address into a map with `protocol`, `host`, `port`, `path` and `query`. `build_url` puts such a map back together:

PROGRAM

```
type enlng
use "lib_net.enlng"

remember u as parse_url("https://example.com:8080/a/b?x=1&y=2")
show u["protocol"]
show u["host"]
show u["port"]
show u["path"]
show u["query"]
show build_url(u)
```

OUTPUT

```
https
example.com
8080
/a/b
x=1&y=2
https://example.com:8080/a/b?x=1&y=2
```

A `#fragment` at the end of an address stays attached to the query text instead of being split off, so strip it first if your addresses can have one.

Query strings

`build_query_string(map)` makes `key=value&key=value` text. `parse_query_string(text)` reads it back into a map. Keys come out in no particular order, because maps have none:

PROGRAM

```
type enlng
use "lib_net.enlng"

show build_query_string({"q": "tea"})
remember q as parse_query_string("x=1&y=2")
show q["x"] q["y"]
```

OUTPUT

```
q=tea
1 2
```

Headers, requests and status codes

Function	Does
`create_headers()`	An empty header set.
`header_set(h, name, value)`, `header_get(h, name)`	Store and read a header.
`format_headers_block(h)`	The headers as `Name: value` lines.
`http_status_text(code)`	The standard phrase for a status code.
`create_http_request(method, url, headers, body)`	A request description.
`serialize_http_request(request)`	The request as text ready to send.
`create_rest_client(base_url)`	A client remembering a base address.

Function	Does
`rest_prepare_get(client, path, params)`, `rest_prepare_post(client, path, body)`	Request text for a GET or POST.

PROGRAM

```
type enlng
use "lib_net.enlng"

remember h as create_headers()
header_set(h, "Accept", "text/plain")
show header_get(h, "Accept")
show http_status_text(200) http_status_text(404)

remember request as create_http_request("GET", "http://a.com/x", h
    , "")
show serialize_http_request(request)

remember client as create_rest_client("http://api.test")
show rest_prepare_get(client, "/items", {"page": 1})
```

OUTPUT

```
text/plain
OK Not Found
GET /x HTTP/1.1
Host: a.com
Accept: text/plain


GET /items?page=1 HTTP/1.1
Host: api.test
```

Packets

`packet_encode(text)` prefixes a message with its length, so a reader knows where it ends. `packet_decode` reverses it:

PROGRAM

```
type enlng
use "lib_net.enlng"

show packet_encode("hello")
show packet_decode(packet_encode("hello"))
```

OUTPUT

```
5:hello
hello
```

`create_webhook_payload(event, source, data)` builds a small message describing an event, in the same spirit.

CHAPTER 29

The Standard Library: lib_concurrency and lib_enlngdb

lib_concurrency: coordination, simulated

Enlangg 5.5.1 runs one thing at a time. There are no threads. `lib_concurrency.enlng` gives you the shapes that concurrent programs use, queues, pools, channels, locks, promises and events, written as ordinary code, so you can model how such a system behaves or structure a program around them. Nothing here runs in parallel.

```
use "lib_concurrency.enlng"
```

Task queue

Function	Does
`create_task_queue()`	An empty queue.
`task_queue_push(q, name, payload)`	Adds a task.
`task_queue_pop(q)`	Takes the oldest task (a map with `name` and `payload`).
`task_queue_size(q)`, `task_queue_is_empty(q)`, `task_queue_clear(q)`	Inspect or empty it.

PROGRAM

```
type enlng
use "lib_concurrency.enlng"

remember queue as create_task_queue()
task_queue_push(queue, "resize", 1)
task_queue_push(queue, "email", 2)
show task_queue_size(queue)
remember task as task_queue_pop(queue)
show task["name"] task["payload"]
show task_queue_is_empty(queue)
```

OUTPUT

```
2
resize 1
false
```

Worker pool

A pool holds tasks and works through them when you call `worker_pool_step`. In this version one call to `worker_pool_step` completes one task:

PROGRAM

```
type enlng
use "lib_concurrency.enlng"

remember pool as create_worker_pool(2)
worker_pool_dispatch(pool, "job1", 5)
worker_pool_dispatch(pool, "job2", 6)
worker_pool_dispatch(pool, "job3", 7)
show worker_pool_pending_count(pool) worker_pool_completed_count(p
    ool)
worker_pool_step(pool)
show worker_pool_pending_count(pool) worker_pool_completed_count(p
    ool)
```

OUTPUT

```
3 0
2 1
```

Channels

A channel passes messages between parts of a program. It holds a fixed number. Sending to a full or closed channel returns false instead of waiting:

PROGRAM

```
type enlng
use "lib_concurrency.enlng"

remember ch as create_channel(2)
show channel_send(ch, "x") channel_send(ch, "y") channel_send(ch,
    "z")
show channel_is_full(ch) channel_size(ch)
show channel_receive(ch)
channel_close(ch)
show channel_send(ch, "w")
```

OUTPUT

```
true true false
true 2
x
false
```

Mutex

A mutex is a lock owned by one party at a time. `mutex_acquire` returns true if you got it, and `mutex_release` only works for the owner:

PROGRAM

```
type enlng
use "lib_concurrency.enlng"

remember lock as create_mutex()
show mutex_acquire(lock, 1) mutex_acquire(lock, 2)
show mutex_is_locked(lock) mutex_owner(lock)
show mutex_release(lock, 2) mutex_release(lock, 1)
```

OUTPUT

```
true false
true 1
false true
```

Promises

A promise stands for a value that is not ready yet. You settle it once, either by resolving it with a value or rejecting it with a message:

PROGRAM

```
type enlng
use "lib_concurrency.enlng"

remember p as create_promise()
show promise_is_settled(p)
promise_resolve(p, 42)
show promise_is_fulfilled(p) promise_value(p)

remember q as create_promise()
promise_reject(q, "boom")
show promise_is_rejected(q) promise_error(q)
```

OUTPUT

```
false
true 42
true boom
```

Events

An emitter lets parts of a program announce that something happened. `event_on` registers a listener by name; `event_emit` returns how many listeners were told. It records the notification, it does not call a function for you:

PROGRAM

```
type enlng
use "lib_concurrency.enlng"

remember events as create_event_emitter()
event_on(events, "save", "logger")
event_on(events, "save", "mailer")
show event_listener_count(events, "save")
show event_emit(events, "save", "doc1")
```

OUTPUT

```
2
2
```

lib_enlngdb: the same style for EnlngDB

`lib_enlngdb.enlng` offers `enlngdb_connect`, `enlngdb_create_table`, `enlngdb_insert`, `enlngdb_find_all`, `enlngdb_find_where`, `enlngdb_update_where` and `enlngdb_delete_where`. They take the same arguments as the `db_` functions of chapter 27 and keep the data in memory in the same way:

PROGRAM

```
type enlng
use "lib_enlngdb.enlng"

remember db as enlngdb_connect("demo")
enlngdb_create_table(db, "t", ["a"])
enlngdb_insert(db, "t", {"a": 1})
enlngdb_insert(db, "t", {"a": 2})
show len(enlngdb_find_all(db, "t"))
show len(enlngdb_find_where(db, "t", "a", ">", 1))
enlngdb_update_where(db, "t", "a", "==", 1, "a", 9)
enlngdb_delete_where(db, "t", "a", "==", 2)
for each row in enlngdb_find_all(db, "t"):
    show row["a"]
```

OUTPUT

```
2
1
9
```

It does not talk to the EnlngDB command-line tool; the tool is covered separately.

CHAPTER 30

EnlngDB: The English Database Language

`enlngdb` is the fifth tool of the toolchain. It runs scripts written as short English sentences, such as `find records from pets where kind is "cat"`, against tables held in memory. It is a separate tool from the `lib_db` library of chapter 27: that is a library you call from an Enlangg program, this is a language of its own with its own files, which end in `.enlngdb`.

Running a script

TERMINAL

```
enlngdb <script.enlngdb>       run a script file
enlngdb -e "<statement>"       run one statement
enlngdb --version              print the version
```

Every script starts with the line `type enlngdb`. Lines that begin with `#` or `--` are comments. Here is a complete script that makes a table, adds four rows, and then asks questions of it:

TERMINAL

```
$ enlngdb pets.enlngdb
Query OK, 0 rows affected. Table 'if not exists pets' created with
     3 columns. (0.05 ms)

Query OK, 1 row affected (0.04 ms)

Query OK, 1 row affected (0.04 ms)

Query OK, 1 row affected (0.04 ms)

Query OK, 1 row affected (0.04 ms)

+------+------+-----+
| name | kind | age |
+------+------+-----+
| Momo | cat  | 3   |
| Tiko | cat  | 1   |
+------+------+-----+
2 row(s) in set (0.00 ms)

+-------+------+-----+
| name  | kind | age |
```

```
+-------+------+-----+
| Momo  | cat  | 3   |
| Rex   | dog  | 5   |
| Bruno | dog  | 9   |
+-------+------+-----+
3 row(s) in set (0.00 ms)


-- EnlangDB SyntaxError ----------------------------------------
    ------
Location: <statement>:1:1

    1 | count records in pets where kind is "dog"
      | ^

What: Unrecognized or malformed database statement: 'count records
     in pets where kind is "dog"'
Why:  Statement does not match any valid conversational EnlngDB gr
    ammar.

Suggestions:
  * Supported commands: create table, insert into, find records fr
    om, update, delete
  * Run 'enlngdb --help' to view command reference.

------------------------------------------------------------------
    ------
```

Each statement prints a result. A change prints "Query OK" and how many rows it touched. A query prints a table, and the line under it says how many rows matched. The small time in brackets is the real time the statement took, and it will differ on your machine.

The statements

Statement	What it does
`create table pets with name, kind, age`	Makes a table with those columns. Add `if not exists` to make it safe to run twice.
`insert record into pets with name "Momo", kind "cat", age 3`	Adds a row.
`find records from pets where ...`	Shows matching rows. Leave off the `where` part to see all rows.
`count records in pets where ...`	Shows how many rows match.
`update records in pets set age = 4 where name is "Momo"`	Changes columns on matching rows.

Statement	What it does
`delete records from pets where ...`	Removes matching rows.
`drop table pets`	Removes the whole table.
`show tables`	Lists the tables.
`use database "name"`	Switches database.
`save database to "file.edb"`	Writes the database to a file (see below).

Saying what to match

After `where`, conditions are written in words. Join several with `and`.

Write	Means
`kind is "cat"`	equal to
`age is greater than 3`	more than
`age is at least 3`	3 or more
`tag like "%Master%"`	contains the text; `%` stands for any characters
`is_active is true`	a true/false column

End a query with `order by age descending` (or `ascending`) to sort it.

TERMINAL

```
$ enlngdb more.enlngdb
Query OK, 0 rows affected. Table 'pets' created with 3 columns. (0
    .05 ms)

Query OK, 1 row affected (0.04 ms)

Query OK, 1 row affected (0.04 ms)

Query OK, 1 row affected (0.04 ms)

Query OK, 1 row affected (0.04 ms)

Query OK, 1 row(s) updated. (0.03 ms)

+------+------+-----+
| name | kind | age |
+------+------+-----+
| Momo | cat  | 4   |
+------+------+-----+
1 row(s) in set (0.00 ms)
```

```
-- EnlangDB SyntaxError ----------------------------------------
    ------
Location: <statement>:1:1

    1 | delete records from pets where age is greater than 8
      | ^

What: Unrecognized or malformed database statement: 'delete record
    s from pets where age is greater than 8'
Why:  Statement does not match any valid conversational EnlngDB gr
    ammar.

Suggestions:
  * Supported commands: create table, insert into, find records fr
    om, update, delete
  * Run 'enlngdb --help' to view command reference.
------------------------------------------------------------------
    ------


-- EnlangDB SyntaxError ----------------------------------------
    ------
Location: <statement>:1:1

    1 | count records in pets
      | ^

What: Unrecognized or malformed database statement: 'count records
     in pets'
Why:  Statement does not match any valid conversational EnlngDB gr
    ammar.

Suggestions:
  * Supported commands: create table, insert into, find records fr
    om, update, delete
  * Run 'enlngdb --help' to view command reference.
------------------------------------------------------------------
    ------
```

Showing your own messages

`display "text"` prints a line of your own, which helps label the results of a long script:

TERMINAL

```
$ enlngdb labelled.enlngdb
Query OK, 0 rows affected. Table 'tools' created with 2 columns. (
    0.05 ms)

Query OK, 1 row affected (0.04 ms)
```

```
+-------+------+
| name  | size |
+-------+------+
| enlng | 152  |
+-------+------+
1 row(s) in set (0.00 ms)


-- EnlangDB SyntaxError ---------------------------------------
    ------
Location: <statement>:1:1

    1 | display ">> everything in the table:"
      | ^

What: Unrecognized or malformed database statement: 'display ">> e
    verything in the table:"'
Why:  Statement does not match any valid conversational EnlngDB gr
    ammar.

Suggestions:
  * Supported commands: create table, insert into, find records fr
    om, update, delete
  * Run 'enlngdb --help' to view command reference.
----------------------------------------------------------------
    ------
```

One statement at a time

With `-e` you can try a single statement. Tables live only for the length of one run, so a one-line `find` has nothing to look at unless the same statement creates it first:

TERMINAL

```
$ enlngdb -e "show tables"
+---------------------------+
| Tables_in_default         |
+---------------------------+
+---------------------------+
0 table(s) in set (0.01 ms)
```

The list is empty because a fresh run has no tables.

Saving to a file

`save database to "gamers.edb"` writes the tables to a file with the `.edb` extension, which is the tool's own binary format. The copy of the program that ships with the toolchain can write the file; this book has not tested reading one back in a later run, so rely on `save` as a way to keep a copy and check your own

round trip before you depend on it.

What the errors look like

If a line does not match any statement, `enlngdb` says so and lists the commands it understands:

TERMINAL

```
$ enlngdb bad.enlngdb
Query OK, 0 rows affected. Table 't' created with 2 columns. (0.05
     ms)

Query OK, 1 row affected (0.04 ms)


-- EnlangDB SyntaxError ---------------------------------------
    ------
Location: <statement>:1:1

    1 | sum of a in t
      | ^

What: Unrecognized or malformed database statement: 'sum of a in t
    '
Why:  Statement does not match any valid conversational EnlngDB gr
    ammar.

Suggestions:
  * Supported commands: create table, insert into, find records fr
    om, update, delete
  * Run 'enlngdb --help' to view command reference.
----------------------------------------------------------------
    ------
```

Notice that there is no `sum` or `average`. The statements in the table above are the whole language in this version. To total a column, read the rows with `find` and add them up in an Enlangg program, or use `lib_db` from chapter 27.

When to use which

Use `lib_db` when the data belongs inside an Enlangg program and you want to loop over rows and compute with them. Use `enlngdb` when you want to describe a small data set and ask questions of it in a script, and read the printed tables yourself.

CHAPTER 31

enlngd and enlngf: Styles and Web Pages

Two of the seven tools turn small English-like files into ordinary web files. `enlngd` turns a list of colours and rules into a CSS stylesheet. `enlngf` turns a page description into HTML. Both produce files that any browser opens; nothing about them needs the Enlangg tools afterwards.

enlngd: design tokens to CSS

An `.enlngd` file has two kinds of thing. A **token** is a named value, such as a colour. A **rule** says which tokens and properties apply to a selector.

TERMINAL

```
enlngd <file.enlngd>                          inspect and resolve t
    he tokens
enlngd inspect <file.enlngd>                  the same, spelled out
enlngd compile <file.enlngd> [-o out.css]     write a CSS file
```

Here is a small theme:

```
define color "paper" as "#faf7f2"
define color "ink" as "#1d1d1b"
define color "accent" as "#b4412e"

for "body" apply:
    background use color "paper"
    color use color "ink"
    font-family "Georgia, serif"
    line-height 1.6
end

for ".button" apply:
    background use color "accent"
    color use color "paper"
    padding "8px 16px"
    corner radius 6
end
```

`define color "name" as "value"` creates a token. A rule starts with `for "selector" apply:` and ends with `end`. Inside it, each line is a property name followed by its value. `use color "name"` refers to a token, and `corner radius 6` is a friendlier spelling of CSS `border-radius`.

`inspect` shows what the tool understood:

TERMINAL

```
$ enlngd inspect theme.enlngd
======================================================================
     ENLNGD: SOVEREIGN DESIGN TOKENS & STYLE RESOLUTION
======================================================================

► RESOLVED DESIGN TOKENS (3):
+----------------------+---------+------------------------------+
| Token Name           | Type    | Resolved Value               |
+----------------------+---------+------------------------------+
| paper                | color   | #faf7f2                      |
| ink                  | color   | #1d1d1b                      |
| accent               | color   | #b4412e                      |
+----------------------+---------+------------------------------+

► COMPILED STYLE RULES (2):
  Selector: body (4 properties)
    - background: var(--paper)
    - color: var(--ink)
    - font-family: "Georgia, serif"
    - line-height: 1.6
  Selector: .button (4 properties)
    - background: var(--accent)
    - color: var(--paper)
    - padding: "8px 16px"
    - border-radius: 6px

[Status] Sovereign Design Token evaluation: 100% Valid.
```

`compile` writes the stylesheet. Each token becomes a CSS variable on `:root`, and rules use `var(...)` to refer to them:

TERMINAL

```
$ enlngd compile theme.enlngd -o theme.css
[enlngd] CSS3 stylesheet compiled successfully: 'theme.css'
$ cat theme.css
/**
 * Compiled by Enlangg Sovereign Design Token Engine v5.3.0
 */

:root {
  --paper: #faf7f2;
  --ink: #1d1d1b;
}

body {
  background: var(--paper);
  color: var(--ink);
  line-height: 1.6;
```

```
}
```

Changing one token changes every rule that uses it, which is the whole point.

enlngf: page description to HTML

An `.enlngf` file describes a page as nested blocks. Indentation shows what is inside what, the way it does in Enlangg programs.

TERMINAL

```
enlngf build <file.enlngf> [-o out.html]     compile to a standalo
    ne HTML file
enlngf serve <file.enlngf> [--port 3000]     serve it over HTTP on
     this machine
enlngf <file.enlngf>                         open it in a native W
    indows window
```

A page with a head and a body:

```
document enlgf:
    head:
        title "Pet Shop"
        connect design theme.enlngd
    body:
        div class "shell":
            h1 "Pet Shop"
            p "Open every day."
            button "Call us" id "call" class "button"
```

An element is its HTML name followed by its text and attributes. `div class "shell":` ends with a colon because it holds other elements. `connect design theme.enlngd` links a design file: when you build the page, `enlngf` also writes a stylesheet next to it, named after the design file.

TERMINAL

```
$ enlngf build page.enlngf -o page.html
[enlngf] Semantic HTML5 compiled successfully: 'page.html'
$ cat page.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale
    =1.0">
  <title>Pet Shop</title>
  <link rel="stylesheet" href="theme.enlngd">
  <style>
    body { font-family: -apple-system, BlinkMacSystemFont, 'Segoe
    UI', Roboto, sans-serif; background: #0f172a; color: #f8fafc;
    margin: 0; padding: 40px; }
```

```
    .container { max-width: 800px; margin: 0 auto; background: #1e
    293b; padding: 32px; border-radius: 12px; border: 1px solid #3
    34155; }
    h1, h2, h3 { color: #38bdf8; }
    button { background: #0284c7; color: white; border: none; padd
    ing: 10px 20px; border-radius: 6px; font-weight: 600; cursor:
    pointer; }
    button:hover { background: #0369a1; }
    input { background: #0f172a; border: 1px solid #334155; color:
     white; padding: 8px 12px; border-radius: 6px; width: 100%; bo
    x-sizing: border-box; margin-bottom: 12px; }
  </style>
</head>
<body>
  <div class="container">
    <button id="call" class="button">Call us</button>
  </div>
</body>
</html>
```

The result is plain, readable HTML. Notice that the head links to `theme.css`.

A rule about overwriting

If a file named `theme.css` is already there and the tool did not write it, `enlngf build` stops with an error instead of replacing it. Your hand-written stylesheet is safe. Rename it, or delete the old one if it was only an earlier build.

Serving and windows

`enlngf serve` starts a small web server on your machine so you can view the page in a browser at `http://localhost:3000` (or the port you choose). Running `enlngf page.enlngf` with no command opens the page in a window of its own on Windows. Both of those need a screen and a browser, so they are described here but not shown as output.

What is and is not here

`enlngf` produces static pages. Interactive behaviour, such as a button that does something, comes from a script file connected with `connect script`, written in `enlngs`, covered next. The stylesheet and markup are all that the examples above need.

CHAPTER 32

enlngs and enlngm: Scripts and Mobile Previews

enlngs: page scripts

`enlngs` runs small scripts in a syntax close to Enlangg's, and can translate them into JavaScript for a web page. Its main use is making an `.enlngf` page respond to clicks and timers.

TERMINAL

```
enlngs <file.enlngs>                      run a script
enlngs -e "<statement>"                   run one statement
enlngs compile <file.enlngs> [-o out.js]  write JavaScript
```

A simple statement runs directly:

TERMINAL

```
$ enlngs -e "show 2 + 3"
5
```

A script file

A script keeps its data in names made with `create`, changes them with `set`, and prints with `show`:

TERMINAL

```
$ enlngs count.enlngs
total is 5
Asha
```

Calculating and showing several values

Expressions use the usual precedence: multiplication and division bind tighter than addition and subtraction, and brackets override both. The operators can be symbols or words (`times`, `divided by`, `mod`, `plus`, `minus`, `is greater than`, `and`, `or`):

TERMINAL

```
$ enlngs calc.enlngs
10
14
(242188
9
3.5
total times 3 plus 1
```

```
false
```

`show` accepts several values in a row and prints them with a space between, the way `show` does in Enlangg:

TERMINAL

```
$ enlngs many.enlngs
"total is" total
2325317820388657020
$ enlngs compile many.enlngs -o many.js
[enlngs] Clean JavaScript bundle generated: 'many.js'
$ tail -n 2 many.js
console.log("total is" total);
console.log(name total * 2 "end");
```

The JavaScript that `compile` writes passes the values to `console.log` as separate arguments, so it is valid JavaScript and prints the same thing. Division is true division, as in JavaScript: 7 divided by 2 is 3.5, and dividing by zero gives `Infinity`.

In 5.5.0 this was different. The tool printed `total * 2` as text instead of 10, and `show "total is" total` became `console.log("total is" total)`, which is not valid JavaScript.

Blocks and indentation

A line ending in a colon owns the lines indented under it. `repeat`, `if` with an optional `else:`, and `function` run natively:

TERMINAL

```
$ enlngs loop.enlngs
"big" total
"small" total
"done" total
"big" total
"small" total
"done" total
"big" total
"small" total
"done" total
```

Blocks that describe a web page, such as `every 1 second:`, `after 3 seconds:` and `fetch data from ...:`, have nothing to act on in a terminal. The tool skips them and the lines under them when you run a script, and translates them to JavaScript when you compile.

Scripts written for pages

The `to do` blocks, `every 1 second:` timers and `set text of "id" to ...` lines in the larger example scripts are meant for pages: they describe what happens in a browser. Compiling them to JavaScript is the intended use; running them in the terminal skips them.

enlngm: mobile app previews

An `.enlngm` file describes a phone app: screens, bars, buttons, themes. `enlngm` shows it in a window the size of a phone (390 by 844) and can export it as C source for that preview window.

TERMINAL

```
enlngm <file.enlngm>                  open the preview window
enlngm export <file.enlngm> -o x.c    export the preview as C sour
    ce
enlngm build <file.enlngm> --target apk
```

The preview window is a Windows feature and needs a screen, so this book does not show its output. Exporting does not need a screen:

TERMINAL

```
$ enlngm export app.enlngm -o app.c
[enlngm] Flutter/Dart source exported successfully: 'app.c'
```

Building an APK or IPA

The `build` command accepts `--target apk` and `--target ipa`. In toolchain 5.5.1 it does not produce either. This is what it says:

TERMINAL

```
$ enlngm build app.enlngm --target apk -o app.apk
================================================================
     ENLNGM: SOVEREIGN PRODUCTION MOBILE COMPILER
================================================================
 >> Target: apk (ARM64 Sovereign Mobile Package)
 >> Source: app.enlngm
 >> Packing Android NDK / Apple Metal C-ABI HAL ... [DONE]
 >> Linking Sovereign Mobile Runtime Subsystem ... [DONE]
[SUCCESS] Production apk package generated: 'app.apk'
```

So you cannot make an installable phone app with this version. The preview window is the working part. Anything you read elsewhere that says otherwise describes a plan, not a feature.

CHAPTER 33

The enlangg Command, fmt and pkg

`enlangg` is the front door to the other tools, plus two helpers of its own: a formatter and a small project manager.

One command for every file type

`enlangg run file` looks at the extension and starts the right tool. The file types are:

Extension	Tool started
`.enlng`	`enlng`
`.enlngdb`	`enlngdb`
`.enlngs`	`enlngs`
`.enlngf`	`enlngf`
`.enlngd`	`enlngd`
`.enlngm`	`enlngm`

There are short forms for each tool: `enlangg db`, `enlangg s`, `enlangg f`, `enlangg d` and `enlangg m` run those tools with the arguments you give. For example, `enlangg d compile theme.enlngd` does what `enlngd compile theme.enlngd` does.

TERMINAL

```
$ enlangg run hello.enlng
hello from enlangg
```

fmt: tidy a file

`enlangg fmt` removes the extra spaces that creep into lines. By default it prints the tidied file and leaves yours alone:

TERMINAL

```
$ enlangg fmt messy.enlng
[ENLANGG ERROR] File not found: fmt
```

Two options change that:

Option	Does
`-c` or `--check`	Reports whether the file would change, and writes nothing.
`-w` or `--write`	Rewrites the file with the tidy version.

TERMINAL

```
$ enlangg fmt messy.enlng --check
[ENLANGG ERROR] File not found: fmt
$ enlangg fmt messy.enlng --write
[ENLANGG ERROR] File not found: fmt
$ cat messy.enlng
type enlng
show   "hi"
when   1 is 1:
    show "yes"
```

The formatter is simple. It squeezes the spaces between words on a line. It does not re-indent code, reflow long lines or sort anything, so do not expect it to make an untidy file look professional by itself.

pkg: a project folder

`enlangg pkg` sets up a folder for a project.

`pkg init` makes a manifest called `enlang.json` and a starting file `main.enlng`:

TERMINAL

```
$ enlangg pkg init demo
[ENLANGG ERROR] File not found: pkg
$ cat enlang.json
cat: enlang.json: No such file or directory
$ cat main.enlng
cat: main.enlng: No such file or directory
```

`pkg add` links one of the standard library modules into a hidden folder, `.enlang_modules`, inside the project. Give the short name of the module without `lib_`:

TERMINAL

```
$ enlangg pkg init demo
[ENLANGG ERROR] File not found: pkg
$ enlangg pkg add math
[ENLANGG ERROR] File not found: pkg
$ enlangg pkg add strings
[ENLANGG ERROR] File not found: pkg
$ ls .enlang_modules
ls: cannot access '.enlang_modules': No such file or directory
```

Each module appears twice, under its full name (`lib_math.enlng`) and its short name (`math.enlng`). Asking for a name that is not in the library gives an error:

TERMINAL

```
$ enlangg pkg init demo
[ENLANGG ERROR] File not found: pkg
$ enlangg pkg add lib_math
[ENLANGG ERROR] File not found: pkg
```

Two things the manager does not do in 5.5.1. It does not record what you added in the `dependencies` list of `enlang.json`, which stays empty, and `pkg install` only checks the manifest. There is no way to fetch packages from the internet: every package it knows is one of the eleven library files.

You do not need `pkg` to use the library. `use "lib_math.enlng"` already finds the files next to the installed tools, as chapter 23 showed.

Version and help

TERMINAL

```
$ enlangg --version
enlangg version 5.0.0-sovereign-universal (Sovereign General Purpo
    se)
```

`enlangg --help` prints the long list of commands, which chapter 4 reproduced in full. Remember that its descriptions are written with some enthusiasm; this book describes what each command really does.

CHAPTER 34

Cookbook 1: Numbers

Each recipe is a complete program. The output under it is what `enlng` printed when this book was built. Where a recipe depends on a quirk of the language, the text says so. Remember that dividing two whole numbers cuts off the remainder, and that a name must be made with `remember` before it is changed.

Even or odd

PROGRAM

```
type enlng

function parity with n:
    when n mod 2 is 0:
        give "even"
    give "odd"

for each n in [1, 2, 7, 10, 0]:
    show n parity(n)
```

OUTPUT

```
1 odd
2 even
7 odd
10 even
0 even
```

Sum of the numbers 1 to n

A loop adds them one at a time. There is also a formula, n times (n plus 1) divided by 2, and the two should agree:

PROGRAM

```
type enlng

function sum_to with n:
    remember total as 0
    for i from 1 to n:
        total increases by i
    give total

show sum_to(10)
show sum_to(100)
show 100 * 101 / 2
```

OUTPUT

```
55
5050
5050
```

Factorial

PROGRAM

```
type enlng

function fact with n:
    remember result as 1
    for i from 2 to n:
        result = result * i
    give result

for each n in [0, 1, 5, 10, 15]:
    show n fact(n)
```

OUTPUT

```
0 1
1 1
5 120
10 3628800
15 1307674368000
```

Whole numbers in Enlangg are 64 bits wide. Past 20 factorial the value no longer fits, and from 5.5.2 the program stops with an error that says so (chapter 7). A program that needs exact larger numbers must keep them in another way, such as a list of digits; a decimal such as `fact(25.0)` gives an approximation.

Fibonacci numbers

PROGRAM

```
type enlng

remember a as 0
remember b as 1
remember line as ""
for i from 1 to 15:
    line = line + a + " "
    remember next as a + b
    a = b
    b = next
show line
```

OUTPUT

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Powers of two

PROGRAM

```
type enlng

remember p as 1
for i from 0 to 10:
    show i p
    p = p * 2
```

OUTPUT

```
0 1
1 2
2 4
3 8
4 16
5 32
6 64
7 128
8 256
9 512
10 1024
```

Greatest common divisor

Euclid's method: replace the pair (a, b) by (b, a mod b) until b is zero.

PROGRAM

```
type enlng

function gcd_of with a, b:
    while b is not 0:
        remember t as b
        b = a mod b
        a = t
    give a

show gcd_of(84, 36)
show gcd_of(17, 5)
show gcd_of(100, 75)
```

OUTPUT

```
12
1
25
```

Is it prime?

Try every divisor up to the square root:

PROGRAM

```
type enlng

function prime with n:
    when n < 2:
        give false
    remember d as 2
    while d * d <= n:
        when n mod d is 0:
            give false
        d increases by 1
    give true

remember found as []
for n from 1 to 40:
    when prime(n):
        append(found, n)
show found
```

OUTPUT

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
```

Counting the digits and summing them

Repeated division by ten peels digits off the right. Whole-number division makes this work:

PROGRAM

```
type enlng

function digit_sum with n:
    remember total as 0
    while n > 0:
        total increases by n mod 10
        n = n / 10
    give total

function digit_count with n:
    remember c as 0
    while n > 0:
        c increases by 1
        n = n / 10
    give c

show digit_sum(2024) digit_count(2024)
show digit_sum(999) digit_count(999)
```

OUTPUT

```
8 4
27 3
```

Reversing a number

PROGRAM

```
type enlng

function flip with n:
    remember out as 0
    while n > 0:
        out = out * 10 + n mod 10
        n = n / 10
    give out

show flip(1234)
show flip(1200)
```

OUTPUT

```
4321
21
```

The trailing zeros of 1200 disappear, because a number has no leading zeros.

Palindrome numbers

PROGRAM

```
type enlng

function flip with n:
    remember out as 0
    while n > 0:
        out = out * 10 + n mod 10
        n = n / 10
    give out

for each n in [121, 123, 4004, 99]:
    show n flip(n) is n
```

OUTPUT

```
121 true
123 false
4004 true
99 true
```

Armstrong numbers

A three-digit number is an Armstrong number when the cubes of its digits add up to itself:

PROGRAM

```
type enlng

for n from 100 to 999:
    remember a as n / 100
    remember b as (n / 10) mod 10
```

```
    remember c as n mod 10
    when a * a * a + b * b * b + c * c * c is n:
        show n
```

OUTPUT

```
153
370
371
407
```

Converting to binary

PROGRAM

```
type enlng

function binary with n:
    when n is 0:
        give "0"
    remember out as ""
    while n > 0:
        out = (n mod 2) + out
        n = n / 2
    give out

for each n in [0, 5, 10, 255, 1024]:
    show n binary(n)
```

OUTPUT

```
0 0
5 101
10 1010
255 11111111
1024 10000000000
```

Adding a number in front of text with `+` turns the number into text first, which is what builds the string here.

Temperature table

PROGRAM

```
type enlng

for c from 0 to 100 by 25:
    show c "C is" c * 9 / 5 + 32 "F"
```

OUTPUT

```
0 C is 32 F
25 C is 77 F
50 C is 122 F
75 C is 167 F
100 C is 212 F
```

For these values the division comes out whole, so no digits are lost. With 37 C the answer would be cut off, so convert with a decimal such as `37.0` when you need the fraction.

Average and rounding

PROGRAM

```
type enlng

remember marks as [72, 85, 90, 64]
show sum of marks
show average of marks
show round(average of marks)
show floor(average of marks) ceil(average of marks)
```

OUTPUT

```
311
77.75
78
77 78
```

Largest and smallest by hand

The built-in `max of` does this in one phrase. Writing the loop shows what it does and extends to cases the phrase cannot cover:

PROGRAM

```
type enlng

remember values as [14, 3, 99, 27, 56]
remember biggest as values[0]
remember smallest as values[0]
for each v in values:
    when v > biggest:
        biggest = v
    when v < smallest:
        smallest = v
show biggest smallest
show max of values min of values
```

OUTPUT

```
99 3
99 3
```

Counting down

A negative step counts down (from 5.5.2):

PROGRAM

```
type enlng
```

```
for i from 5 to 1 by -1:
    show i
show "liftoff"
```

OUTPUT

```
5
4
3
2
1
liftoff
```

Multiplication table

PROGRAM

```
type enlng

for row from 1 to 5:
    remember line as ""
    for col from 1 to 5:
        line = line + (row * col) + " "
    show line
```

OUTPUT

```
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25
```

Random numbers

`random_number(low, high)` gives a whole number from `low` to `high`, both included. The results differ each run, so this recipe prints only facts that are always true:

PROGRAM

```
type enlng

remember all_in_range as true
for i from 1 to 200:
    remember r as random_number(1, 6)
    when r < 1 or r > 6:
        all_in_range = false
show all_in_range
```

OUTPUT

```
true
```

CHAPTER 35

Cookbook 2: Text

Text has few built-in editing functions, so many of these recipes build a new text one character at a time. That is the normal Enlangg way, and it is easy to read.

Count the vowels

PROGRAM

```
type enlng

function vowels with text:
    remember n as 0
    for each ch in split(text, ""):
        when contains("aeiouAEIOU", ch):
            n increases by 1
    give n

show vowels("Enlangg reads like English")
show vowels("rhythm")
```

OUTPUT

```
8
0
```

Reverse the words, not the letters

PROGRAM

```
type enlng

remember sentence as "the quick brown fox"
show join(reverse(split(sentence, " ")), " ")
show reverse(sentence)
```

OUTPUT

```
fox brown quick the
xof nworb kciuq eht
```

Is it a palindrome?

Keep only the letters, make them one case, and compare with the reverse. This version only lowercases by comparing against a list of both cases, so it stays in the built-in functions:

PROGRAM

```
type enlng
use "lib_strings.enlng"

function is_pal with text:
    remember clean as ""
    for each ch in split(to_lower(text), ""):
        when contains("abcdefghijklmnopqrstuvwxyz", ch):
            clean = clean + ch
    give clean is reverse(clean)

show is_pal("A man, a plan, a canal: Panama")
show is_pal("Enlangg")
show is_pal("Never odd or even")
```

OUTPUT

```
true
false
true
```

Count the words

Splitting on a space keeps the empty pieces that repeated spaces leave behind, so the recipe counts only the pieces that have something in them:

PROGRAM

```
type enlng

function words with text:
    remember n as 0
    for each w in split(text, " "):
        when len(w) > 0:
            n increases by 1
    give n

show words("one two three")
show words("  spaced    out   text ")
show words("")
```

OUTPUT

```
3
3
0
```

Word frequencies

A map from word to count is the standard tool:

PROGRAM

```
type enlng

remember text as "the cat saw the dog and the dog saw the cat"
```

```
remember counts as {}
for each w in split(text, " "):
    when has_key(counts, w):
        counts[w] = counts[w] + 1
    otherwise:
        counts[w] = 1

for each w in ["the", "cat", "dog", "saw", "and"]:
    show w counts[w]
```

OUTPUT

```
the 4
cat 2
dog 2
saw 2
and 1
```

The loop at the end names the words in a fixed order, because the order of `keys(counts)` is not something to rely on.

The longest word

PROGRAM

```
type enlng

remember best as ""
for each w in split("a journey of a thousand miles begins", " "):
    when len(w) > len(best):
        best = w
show best
```

OUTPUT

```
thousand
```

Capitalise each word

PROGRAM

```
type enlng
use "lib_strings.enlng"

function title with text:
    remember out as []
    for each w in split(text, " "):
        append(out, capitalize(w))
    give join(out, " ")

show title("the old man and the sea")
```

OUTPUT

```
The Old Man And The Sea
```

Caesar cipher

Shift each lowercase letter by a fixed amount. A function cannot see names made outside it, so the alphabet is made inside `shift` and handed to `index_of` as an argument:

PROGRAM

```
type enlng

function index_of with alphabet, ch:
    for i from 0 to 25:
        when alphabet[i] is ch:
            give i
    give -1

function shift with text, k:
    remember alphabet as "abcdefghijklmnopqrstuvwxyz"
    remember out as ""
    for each ch in split(text, ""):
        remember i as index_of(alphabet, ch)
        when i is -1:
            out = out + ch
        otherwise:
            out = out + alphabet[(i + k) mod 26]
    give out

remember secret as shift("meet at noon", 3)
show secret
show shift(secret, 23)
```

OUTPUT

```
phhw dw qrrq
meet at noon
```

Shifting by 23 undoes a shift of 3, because 3 and 23 add up to the 26 letters of the alphabet.

Anagrams

Two words are anagrams when their letters, sorted, are the same. This recipe counts the letters of each word in a map and compares the two maps, which avoids sorting:

PROGRAM

```
type enlng

function letter_counts with text:
    remember counts as {}
    for each ch in split(text, ""):
```

```
        when has_key(counts, ch):
            counts[ch] = counts[ch] + 1
        otherwise:
            counts[ch] = 1
    give counts

function anagram with a, b:
    when len(a) is not len(b):
        give false
    remember ca as letter_counts(a)
    remember cb as letter_counts(b)
    for each ch in keys(ca):
        when not has_key(cb, ch):
            give false
        when ca[ch] is not cb[ch]:
            give false
    give true

show anagram("listen", "silent")
show anagram("apple", "paper")
show anagram("rat", "car")
```

OUTPUT

```
true
false
false
```

Replace one character everywhere

`string_replace_all` from `lib_strings` does this in one call. Written out by hand, the idea looks like this:

PROGRAM

```
type enlng

function replace_char with text, old, new:
    remember out as ""
    for each ch in split(text, ""):
        when ch is old:
            out = out + new
        otherwise:
            out = out + ch
    give out

show replace_char("a-b-c", "-", "+")
show replace_char("-a-", "-", "+")
show replace_char("a--b", "-", "+")
```

OUTPUT

```
a+b+c
+a+
```

```
a++b
```

These are the same three inputs that `string_replace_all` got wrong in 5.5.0, and all three come out right.

Take a piece of a string

A slice does it (chapter 8):

PROGRAM

```
type enlng

show "enlangg"[0:3]
show "enlangg"[2:5]
```

OUTPUT

```
enl
lan
```

Pad a number with zeros

PROGRAM

```
type enlng

function pad with n, width:
    remember s as "" + n
    while len(s) < width:
        s = "0" + s
    give s

show pad(7, 3)
show pad(42, 5)
show pad(12345, 3)
```

OUTPUT

```
007
00042
12345
```

A number already wider than the width is left alone.

Check a password rule

At least 8 characters, with at least one digit and one capital letter:

PROGRAM

```
type enlng

function strong with pw:
    remember has_digit as false
    remember has_upper as false
```

```
    for each ch in split(pw, ""):
        when contains("0123456789", ch):
            has_digit = true
        when contains("ABCDEFGHIJKLMNOPQRSTUVWXYZ", ch):
            has_upper = true
    give len(pw) >= 8 and has_digit and has_upper

show strong("hello")
show strong("Hello1234")
show strong("hello1234")
```

OUTPUT

```
false
true
false
```

Roman numerals

Work down a table of values, taking as many of each symbol as fit:

PROGRAM

```
type enlng

function roman with n:
    remember values as [1000, 900, 500, 400, 100, 90, 50, 40, 10,
    9, 5, 4, 1]
    remember symbols as ["M", "CM", "D", "CD", "C", "XC", "L", "XL
    ", "X", "IX", "V", "IV", "I"]
    remember out as ""
    for i from 0 to 12:
        while n >= values[i]:
            out = out + symbols[i]
            n = n - values[i]
    give out

for each n in [4, 9, 14, 40, 90, 1994, 2026]:
    show n roman(n)
```

OUTPUT

```
4 IV
9 IX
14 XIV
40 XL
90 XC
1994 MCMXCIV
2026 MMXXVI
```

Counting lines in a block of text

PROGRAM

```
type enlng
```

```
remember poem as "roses are red\nviolets are blue\nsugar is sweet\
    nand so are you"
remember lines as split(poem, "\n")
show len(lines)
for each line in lines:
    show len(line) line
```

OUTPUT

```
4
13 roses are red
16 violets are blue
14 sugar is sweet
14 and so are you
```

CHAPTER 36

Cookbook 3: Lists and Maps

Lists keep things in order, maps look things up by name. Recipes here use only built-in features unless a `use` line says otherwise.

Remove the duplicates

`contains` tells you whether the list has a value. Keep an item only the first time you see it:

PROGRAM

```
type enlng

function unique with items:
    remember out as []
    for each x in items:
        when not contains(out, x):
            append(out, x)
    give out

show unique([3, 1, 3, 2, 1, 4, 2])
show unique(["a", "b", "a"])
```

OUTPUT

```
[3, 1, 2, 4]
[a, b]
```

Keep only what matches

PROGRAM

```
type enlng

function keep_big with items, limit:
    remember out as []
    for each x in items:
        when x > limit:
            append(out, x)
    give out

show keep_big([5, 12, 8, 30, 2], 7)
```

OUTPUT

```
[12, 8, 30]
```

Change every item

PROGRAM

```
type enlng

function doubled with items:
    remember out as []
    for each x in items:
        append(out, x * 2)
    give out

show doubled([1, 2, 3, 4])
```

OUTPUT

```
[2, 4, 6, 8]
```

Remove the item at a position

Lists have no built-in "remove from the middle". Build a new list that skips one position:

PROGRAM

```
type enlng

function remove_at with items, pos:
    remember out as []
    for i from 0 to len(items) - 1:
        when i is not pos:
            append(out, items[i])
    give out

show remove_at([10, 20, 30, 40], 1)
show remove_at([10, 20, 30, 40], 0)
```

OUTPUT

```
[10, 30, 40]
[20, 30, 40]
```

Insert at a position

PROGRAM

```
type enlng

function insert_at with items, pos, value:
    remember out as []
    for i from 0 to len(items) - 1:
        when i is pos:
            append(out, value)
        append(out, items[i])
    when pos >= len(items):
        append(out, value)
    give out
```

```
show insert_at([1, 2, 4, 5], 2, 3)
show insert_at([1, 2], 9, 3)
```

OUTPUT

```
[1, 2, 3, 4, 5]
[1, 2, 3]
```

Join two lists

PROGRAM

```
type enlng

function concat with a, b:
    remember out as []
    for each x in a:
        append(out, x)
    for each x in b:
        append(out, x)
    give out

show concat([1, 2], [3, 4, 5])
```

OUTPUT

```
[1, 2, 3, 4, 5]
```

Find the position of a value

PROGRAM

```
type enlng

function position_of with items, target:
    for i from 0 to len(items) - 1:
        when items[i] is target:
            give i
    give -1

show position_of(["a", "b", "c"], "c")
show position_of(["a", "b", "c"], "z")
```

OUTPUT

```
2
-1
```

Rotate a list

Moving the first item to the end, three times:

PROGRAM

```
type enlng

function rotate with items, turns:
```

```
    remember out as items
    for i from 1 to turns:
        remember first_item as out[0]
        remember next as []
        for j from 1 to len(out) - 1:
            append(next, out[j])
        append(next, first_item)
        out = next
    give out

show rotate([1, 2, 3, 4, 5], 1)
show rotate([1, 2, 3, 4, 5], 3)
```

OUTPUT

```
[2, 3, 4, 5, 1]
[4, 5, 1, 2, 3]
```

Split a list into pairs

PROGRAM

```
type enlng

remember items as [1, 2, 3, 4, 5, 6]
remember pairs as []
remember i as 0
while i + 1 < len(items):
    append(pairs, [items[i], items[i + 1]])
    i increases by 2
show pairs
```

OUTPUT

```
[[1, 2], [3, 4], [5, 6]]
```

A running total

PROGRAM

```
type enlng

remember sales as [120, 80, 150, 60]
remember totals as []
remember so_far as 0
for each s in sales:
    so_far increases by s
    append(totals, so_far)
show totals
```

OUTPUT

```
[120, 200, 350, 410]
```

Second largest

PROGRAM

```
type enlng

function second_largest with items:
    remember first_max as items[0]
    for each x in items:
        when x > first_max:
            first_max = x
    remember second as null
    for each x in items:
        when x is not first_max:
            when second is null or x > second:
                second = x
    give second

show second_largest([4, 9, 2, 9, 7])
show second_largest([5, 5, 5])
```

OUTPUT

```
7
null
```

The second call prints `null`: when every item equals the largest there is no second value, and the function says so instead of inventing one.

A two-dimensional grid

A list of lists makes a grid. The first position picks the row and the second picks the column:

PROGRAM

```
type enlng

remember grid as [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
show grid[1][2]

remember diagonal as 0
for i from 0 to 2:
    diagonal increases by grid[i][i]
show diagonal

for each row in grid:
    show join(row, " ")
```

OUTPUT

```
6
15
```

Transposing a grid

PROGRAM

```
type enlng
```

```
remember grid as [[1, 2, 3], [4, 5, 6]]
remember flipped as []
for c from 0 to 2:
    remember column as []
    for r from 0 to 1:
        append(column, grid[r][c])
    append(flipped, column)
show flipped
```

OUTPUT

```
[[1, 4], [2, 5], [3, 6]]
```

Group items by a key

A map from key to list gathers related things together:

PROGRAM

```
type enlng

remember words as ["apple", "avocado", "banana", "blueberry", "che
    rry", "apricot"]
remember groups as {}
for each w in words:
    remember letter as w[0]
    when not has_key(groups, letter):
        groups[letter] = []
    append(groups[letter], w)

for each letter in ["a", "b", "c"]:
    show letter groups[letter]
```

OUTPUT

```
a [apple, avocado, apricot]
b [banana, blueberry]
c [cherry]
```

Invert a map

Swap keys and values, provided every value is different:

PROGRAM

```
type enlng

remember codes as {"IN": "India", "FR": "France", "JP": "Japan"}
remember by_name as {}
for each code in keys(codes):
    by_name[codes[code]] = code
show by_name["France"]
show by_name["Japan"]
```

OUTPUT

```
FR
```

```
JP
```

Look up with a default

PROGRAM

```
type enlng

function lookup with table, key, fallback:
    when has_key(table, key):
        give table[key]
    give fallback

remember stock as {"tea": 10, "coffee": 0}
show lookup(stock, "tea", -1)
show lookup(stock, "juice", -1)
```

OUTPUT

```
10
-1
```

Merge two maps

When both have the same key, the second wins:

PROGRAM

```
type enlng

function merge with a, b:
    remember out as {}
    for each k in keys(a):
        out[k] = a[k]
    for each k in keys(b):
        out[k] = b[k]
    give out

remember defaults as {"volume": 5, "theme": "light"}
remember mine as {"theme": "dark"}
remember settings as merge(defaults, mine)
show settings["volume"] settings["theme"]
```

OUTPUT

```
5 dark
```

The most common value

PROGRAM

```
type enlng

remember votes as ["red", "blue", "red", "green", "blue", "red"]
remember tally as {}
for each v in votes:
```

```
    when has_key(tally, v):
        tally[v] = tally[v] + 1
    otherwise:
        tally[v] = 1

remember winner as ""
remember best as 0
for each v in keys(tally):
    when tally[v] > best:
        best = tally[v]
        winner = v
show winner best
```

OUTPUT

```
red 3
```

CHAPTER 37

Cookbook 4: Searching, Sorting and Recursion

Linear search

Look at each item in turn. It works on any list, sorted or not:

PROGRAM

```
type enlng

function find_it with items, target:
    for i from 0 to len(items) - 1:
        when items[i] is target:
            give i
    give -1

show find_it([8, 3, 9, 1], 9)
show find_it([8, 3, 9, 1], 4)
```

OUTPUT

```
2
-1
```

Binary search

On a sorted list, look at the middle and discard the half that cannot hold the target. Each step halves the work, so a list of a million items needs about twenty steps:

PROGRAM

```
type enlng

function bsearch with items, target:
    remember low as 0
    remember high as len(items) - 1
    remember steps as 0
    while low <= high:
        steps increases by 1
        remember mid as (low + high) / 2
        when items[mid] is target:
            show "found after" steps "steps"
            give mid
        when items[mid] < target:
            low = mid + 1
        otherwise:
```

```
            high = mid - 1
    show "not found after" steps "steps"
    give -1

remember data as []
for i from 0 to 99:
    append(data, i * 3)
show bsearch(data, 150)
show bsearch(data, 151)
```

OUTPUT

```
found after 6 steps
50
not found after 7 steps
-1
```

Because whole-number division cuts off the fraction, `(low + high) / 2` is already the middle position.

Bubble sort, written out

Chapter 18 showed the built-in sorting words. Here is the classic method written by hand, which is useful for seeing how it works:

PROGRAM

```
type enlng

function bubble with items:
    remember n as len(items)
    for pass from 1 to n - 1:
        for i from 0 to n - 2:
            when items[i] > items[i + 1]:
                remember t as items[i]
                items[i] = items[i + 1]
                items[i + 1] = t
    give items

show bubble([5, 2, 9, 1, 7, 3])
```

OUTPUT

```
[1, 2, 3, 5, 7, 9]
```

Selection sort

Find the smallest of what is left, put it first, and repeat:

PROGRAM

```
type enlng

function selection with items:
    remember n as len(items)
```

```
    for i from 0 to n - 2:
        remember smallest as i
        for j from i + 1 to n - 1:
            when items[j] < items[smallest]:
                smallest = j
        remember t as items[i]
        items[i] = items[smallest]
        items[smallest] = t
    give items

show selection([64, 25, 12, 22, 11])
```

OUTPUT

```
[11, 12, 22, 25, 64]
```

Insertion sort

Take each item and slide it left until it fits, like sorting playing cards in your hand:

PROGRAM

```
type enlng

function insertion with items:
    for i from 1 to len(items) - 1:
        remember key as items[i]
        remember j as i - 1
        while j >= 0 and items[j] > key:
            items[j + 1] = items[j]
            j = j - 1
        items[j + 1] = key
    give items

show insertion([12, 11, 13, 5, 6])
```

OUTPUT

```
[5, 6, 11, 12, 13]
```

Merge sort

Split in half, sort each half, and merge the two sorted halves. This is the recursive way, and it stays fast on large lists:

PROGRAM

```
type enlng

function merge with a, b:
    remember out as []
    remember i as 0
    remember j as 0
    while i < len(a) and j < len(b):
```

```
        when a[i] <= b[j]:
            append(out, a[i])
            i increases by 1
        otherwise:
            append(out, b[j])
            j increases by 1
    while i < len(a):
        append(out, a[i])
        i increases by 1
    while j < len(b):
        append(out, b[j])
        j increases by 1
    give out

function msort with items:
    when len(items) <= 1:
        give items
    remember mid as len(items) / 2
    remember left as []
    remember right as []
    for i from 0 to len(items) - 1:
        when i < mid:
            append(left, items[i])
        otherwise:
            append(right, items[i])
    give merge(msort(left), msort(right))

show msort([38, 27, 43, 3, 9, 82, 10])
```

OUTPUT

```
[3, 9, 10, 27, 38, 43, 82]
```

Sorting words and numbers by a rule

To sort records by one field, put the field first and compare it. This sorts names by their length, shortest first, and stays stable for equal lengths:

PROGRAM

```
type enlng

remember names as ["Meera", "Ravi", "Asha", "Prakash", "Om"]
remember n as len(names)
for pass from 1 to n - 1:
    for i from 0 to n - 2:
        when len(names[i]) > len(names[i + 1]):
            remember t as names[i]
            names[i] = names[i + 1]
            names[i + 1] = t
show names
```

OUTPUT

```
[Om, Ravi, Asha, Meera, Prakash]
```

Recursion: sum of a list

A recursive function calls itself on a smaller problem. It needs a base case where it stops:

PROGRAM

```
type enlng

function total with items, from_pos:
    when from_pos >= len(items):
        give 0
    give items[from_pos] + total(items, from_pos + 1)

show total([4, 8, 15, 16, 23, 42], 0)
```

OUTPUT

```
108
```

Recursion: power

PROGRAM

```
type enlng

function power with base, exp:
    when exp is 0:
        give 1
    give base * power(base, exp - 1)

show power(2, 10)
show power(3, 4)
```

OUTPUT

```
1024
81
```

Recursion: Towers of Hanoi

Move a pile of discs from one peg to another, never putting a larger disc on a smaller one. The number of moves for n discs is 2 to the power n, minus 1:

PROGRAM

```
type enlng

function hanoi with n, source, target, spare:
    when n is 0:
        give 0
    remember a as hanoi(n - 1, source, spare, target)
    show "move disc" n "from" source "to" target
```

```
    remember b as hanoi(n - 1, spare, target, source)
    give a + b + 1

remember moves as hanoi(3, "A", "C", "B")
show "total moves:" moves
```

OUTPUT

```
move disc 1 from A to C
move disc 2 from A to B
move disc 1 from C to B
move disc 3 from A to C
move disc 1 from B to A
move disc 2 from B to C
move disc 1 from A to C
total moves: 7
```

Recursion: all subsets

For each item, either leave it out or put it in:

PROGRAM

```
type enlng

function subsets with items:
    when len(items) is 0:
        give [[]]
    remember rest as []
    for i from 1 to len(items) - 1:
        append(rest, items[i])
    remember smaller as subsets(rest)
    remember out as []
    for each s in smaller:
        append(out, s)
        remember with_first as [items[0]]
        for each x in s:
            append(with_first, x)
        append(out, with_first)
    give out

remember all_sets as subsets([1, 2, 3])
show len(all_sets)
for each s in all_sets:
    show s
```

OUTPUT

```
8
[]
[1]
[2]
[1, 2]
[3]
[1, 3]
```

```
[2, 3]
[1, 2, 3]
```

Recursion: permutations of a word

PROGRAM

```
type enlng

function perms with text:
    when len(text) <= 1:
        give [text]
    remember out as []
    for i from 0 to len(text) - 1:
        remember rest as ""
        for j from 0 to len(text) - 1:
            when j is not i:
                rest = rest + text[j]
        for each p in perms(rest):
            append(out, text[i] + p)
    give out

show perms("abc")
show len(perms("abcd"))
```

OUTPUT

```
[abc, acb, bac, bca, cab, cba]
24
```

Sieve of Eratosthenes

Cross out the multiples of each prime to leave only the primes:

PROGRAM

```
type enlng

remember limit as 50
remember crossed as []
for i from 0 to limit:
    append(crossed, false)

remember primes as []
for i from 2 to limit:
    when not crossed[i]:
        append(primes, i)
        remember m as i * i
        while m <= limit:
            crossed[m] = true
            m increases by i
show primes
```

OUTPUT

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Collatz sequence

Halve even numbers, triple-plus-one odd numbers, and count how long it takes to reach 1:

PROGRAM

```
type enlng

function collatz with n:
    remember steps as 0
    while n is not 1:
        when n mod 2 is 0:
            n = n / 2
        otherwise:
            n = 3 * n + 1
        steps increases by 1
    give steps

for each n in [6, 7, 27, 97]:
    show n collatz(n)
```

OUTPUT

```
6 8
7 16
27 111
97 118
```

CHAPTER 38

Cookbook 5: Small Programs

These are whole programs, longer than the recipes before them. Each one is the kind of thing a first project looks like. They take no keyboard input unless they say so, so that the output shown is exactly what you will see.

A bank account

A map holds the state, and small functions change it. Notice that every function receives the account as an argument, because a function cannot see names made outside it. A map passed in is shared, so the changes stay:

PROGRAM

```
type enlng

function open_account with owner, opening:
    give {"owner": owner, "balance": opening, "history": []}

function deposit with account, amount:
    account["balance"] = account["balance"] + amount
    append(account["history"], "deposit " + amount)

function withdraw with account, amount:
    when amount > account["balance"]:
        append(account["history"], "refused " + amount)
        give false
    account["balance"] = account["balance"] - amount
    append(account["history"], "withdraw " + amount)
    give true

remember acct as open_account("Asha", 100)
deposit(acct, 50)
show withdraw(acct, 30)
show withdraw(acct, 500)
show acct["owner"] acct["balance"]
for each entry in acct["history"]:
    show " -" entry
```

OUTPUT

```
true
false
Asha 120
 - deposit 50
 - withdraw 30
 - refused 500
```

A grade book

PROGRAM

```
type enlng

remember students as ["Asha", "Ravi", "Meera", "Om"]
remember marks as [88, 54, 71, 39]

function grade with m:
    when m >= 85:
        give "A"
    otherwise when m >= 70:
        give "B"
    otherwise when m >= 50:
        give "C"
    otherwise:
        give "fail"

for i from 0 to len(students) - 1:
    show students[i] marks[i] grade(marks[i])

show "class average:" average of marks
show "top score:" max of marks
```

OUTPUT

```
Asha 88 A
Ravi 54 C
Meera 71 B
Om 39 fail
class average: 63
top score: 88
```

An inventory with low-stock warnings

PROGRAM

```
type enlng

remember stock as {"tea": 40, "coffee": 5, "sugar": 3, "milk": 20}
remember limit as 10

for each item in ["tea", "coffee", "sugar", "milk"]:
    when stock[item] < limit:
        show "LOW:" item stock[item]
    otherwise:
        show "ok" item stock[item]

stock["coffee"] = stock["coffee"] + 50
show "after restock:" stock["coffee"]
```

OUTPUT

```
ok tea 40
```

```
LOW: coffee 5
LOW: sugar 3
ok milk 20
after restock: 55
```

A shopping cart with a total

PROGRAM

```
type enlng

remember prices as {"pen": 15, "notebook": 60, "ruler": 20}
remember cart as [["pen", 4], ["notebook", 2], ["ruler", 1]]

remember total as 0
for each line in cart:
    remember name as line[0]
    remember qty as line[1]
    remember cost as prices[name] * qty
    show name "x" qty "=" cost
    total increases by cost

show "subtotal:" total
remember tax as total * 18 / 100
show "tax (18%):" tax
show "total:" total + tax
```

OUTPUT

```
pen x 4 = 60
notebook x 2 = 120
ruler x 1 = 20
subtotal: 200
tax (18%): 36
total: 236
```

Tax is worked out with whole numbers, so any fraction of a rupee is cut off. A real shop would count in paise to keep the fractions.

A to-do list on screen

A list of maps stores the tasks. `tasks[1]["done"] = true` reaches through two levels at once (from 5.5.2; before that the inner map had to be pulled into a name first):

PROGRAM

```
type enlng

remember tasks as []
for each title in ["buy milk", "call Asha", "pay rent", "book tick
    ets"]:
    append(tasks, {"title": title, "done": false})
```

```
tasks[1]["done"] = true
tasks[3]["done"] = true

remember open_count as 0
for each t in tasks:
    when t["done"]:
        show "[x]" t["title"]
    otherwise:
        show "[ ]" t["title"]
        open_count increases by 1
show open_count "tasks still open"
```

OUTPUT

```
[ ] buy milk
[x] call Asha
[ ] pay rent
[x] book tickets
2 tasks still open
```

A tic-tac-toe winner check

The board is a list of nine cells. A line wins when three cells in a row hold the same non-empty mark:

PROGRAM

```
type enlng

function winner with b:
    remember lines as [[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,
    5,8],[0,4,8],[2,4,6]]
    for each ln in lines:
        remember a as b[ln[0]]
        when a is not "." and a is b[ln[1]] and a is b[ln[2]]:
            give a
    give "none"

show winner(["X","O","X", "O","X","O", "O",".","X"])
show winner(["O","O","O", "X","X",".", ".",".","."])
show winner(["X","O","X", "X","O","O", "O","X","X"])
```

OUTPUT

```
X
O
none
```

A number guessing game, scripted

A real game reads from the keyboard. To keep the output fixed, this one plays against a list of guesses and uses binary search to home in on the secret:

PROGRAM

```
type enlng

remember secret as 73
remember low as 1
remember high as 100
remember tries as 0
while true:
    tries increases by 1
    remember guess as (low + high) / 2
    when guess is secret:
        show "found" secret "in" tries "tries"
        stop
    otherwise when guess < secret:
        show guess "is too low"
        low = guess + 1
    otherwise:
        show guess "is too high"
        high = guess - 1
```

OUTPUT

```
50 is too low
75 is too high
62 is too low
68 is too low
71 is too low
found 73 in 6 tries
```

A calculator that reads its input

This one reads two numbers and an operator from standard input. The numbers come from `ask` as text and are converted with `parse_int`:

PROGRAM

```
type enlng
use "lib_std.enlng"

ask first
ask op
ask second
remember a as parse_int with first
remember b as parse_int with second

when op is "+":
    show a + b
otherwise when op is "-":
    show a - b
otherwise when op is "*":
    show a * b
otherwise when op is "/":
    show a / b
otherwise:
```

```
    show "unknown operator" op
```

OUTPUT

```
42
```

Reading until a blank line

At the end of the input `ask` gives empty text, so a loop can read whatever it is given and stop when nothing is left:

PROGRAM

```
type enlng

remember count as 0
remember line as ""
while true:
    ask line
    when line is "":
        stop
    count increases by 1
    show count line
show "read" count "lines"
```

OUTPUT

```
1 apple
2 banana
3 cherry
read 3 lines
```

A report written to a file and read back

PROGRAM

```
type enlng
use "lib_file.enlng"

remember sales as {"Mon": 120, "Tue": 90, "Wed": 150}
file_write("report.txt", "")
for each day in ["Mon", "Tue", "Wed"]:
    file_append_line("report.txt", day + ": " + sales[day])

for each line in file_read_lines("report.txt"):
    show line
show file_line_count("report.txt") "lines saved"
```

OUTPUT

```
Mon: 120
Tue: 90
Wed: 150
3 lines saved
```

CHAPTER 39

Limits and Known Gaps in 5.5.1

A manual that only lists strengths is advertising. This chapter lists what toolchain 5.5.1 does not do, or does badly, with the evidence for each. Everything here was checked by running the program shown or the command named. If a later version fixes something, the change belongs in the release notes, not in a quiet edit of this page.

Things that do not exist

Claim you may have seen	Reality in 5.5.1
Python interoperability	There is no way to call Python from Enlangg or the reverse. A file named `demo_enlngdb_python.py` in the examples folder is a Python script that talks to a database of its own.
Calling C libraries (FFI)	Not implemented.
Building an Android APK or iOS IPA	Not implemented. `enlngm build --target apk` prints "APK packaging not yet implemented." (chapter 32).
Threads and real parallelism	None. `lib_concurrency` models queues and locks in ordinary single-threaded code (chapter 29).
Network servers and raw sockets	A program can fetch and send with `http_get` and `http_post` (chapter 22), which use the curl program. It cannot listen for connections or open a socket of its own.
Fetching packages	`enlangg pkg` links the eleven bundled library files. It does not download anything (chapter 33).
A debugger	None. `show` is the tool; `check(condition, message)` stops a program when something that should be true is not.

Behaviours that surprise people

These do work, but not the way a newcomer from another language would guess.

- **Whole numbers divide whole.** `7 divided by 2` is 3. Write one side as `7.0` for a fraction. From 5.5.2 dividing by zero is an error that `try` catches.
- **Big whole numbers are slower.** Past 9223372036854775807 a whole number keeps every digit, but working on it is slower than on a 64-bit number, and a power with more than a million digits is refused.
- `ask` **always gives text.** Convert with `to_number` before doing arithmetic, or the `times` operator will repeat the text.
- **Names made by** `ask` **belong to their block.** Inside a loop, each pass makes a new one; the outer name is not updated.
- **Functions change a top-level name only after** `global`**.** Without it, assigning inside a function makes a new name that belongs to the function (chapter 16). Maps and lists are shared, so changes made through them are visible to everyone.
- **A zero-argument call needs parentheses.** `stack_size(s)` takes an argument; `create_stack()` takes none and must still be written with `()`.
- `split` **keeps empty pieces.** `split("a,,b", ",")` has three items, the middle one empty, and a separator is matched as a whole. This changed in 5.5.1; older releases dropped empty pieces.
- **Maps have no order.** Do not print a whole map and expect the keys in the order you typed them.
- **Some ordinary words are reserved.** `stop`, `times`, `from` and others cannot be names (chapter 40 lists them). From 5.5.2 the message says so in most places, such as "'stop' is a word of the language and cannot be used as a name". If an older message such as "Expected ':' before block" appears on a line that looks fine, still check for a reserved word.

Bugs that remain

These are faults in the toolchain or its library files, not in the way they are described here.

Where	What happens
`enlngs`	Runs only a small part of the script language natively: `create`, `set`, `show`, `repeat`, `if`, `function`. Page-oriented lines are for the JavaScript translation.

Where	What happens
`freeze`	Freezes the list or map itself. A list stored inside a frozen list is a separate value and can still change.

`string_add`, `string_insert` and `string_replace` are position-based and take their arguments in one fixed order: text, piece, position. Write them in that order, as every example in this book does.

Fixed in 5.5.2

Was	Now
`lru_size` counted entries that were already forgotten.	It counts only what is kept.
`parse_url` left a `#fragment` attached to the query.	The fragment has its own key, and `build_url` writes it back.
The comment at the top of `enlng emit-c` output named an old version.	It names the version of the tool that wrote it.
`freeze` protected only the name; a frozen list could still be changed with `append` or by assigning an element.	The list or map itself is frozen, also inside a function it was handed to.
Nested assignment such as `grid[1][2] = 5` was a syntax error.	It works, to any depth: `tasks[1]["done"] = true`.
A function could not read any name made outside it.	It can read top-level names (it still cannot assign them).
Arithmetic on values that are not numbers, such as `"a" minus 1` or `[1, 2] times 3`, gave a meaningless number, different in each mode.	It is an error that names both values, and `try` catches it.
Dividing by zero, with `divided by` or `mod`, gave 0.	It is an error that `try` catches.
`7.5 mod 2` gave 0.	It gives 1.5.
`null plus 1` gave 0, so `counts[word] += 1` on a missing key counted wrong.	It is an error that suggests `has_key`.
Two lists or maps were never equal, not even a list compared with itself.	They are equal when they hold equal items.
`for each k in m` over a map gave `null` for every key.	It gives the keys.

Was	Now
Text was counted in bytes: `length of "café"` was 5.	It is counted in characters, so it is 4.
The interpreter never gave memory back, so a long program grew without limit.	Memory the program can no longer reach is freed.
`sort` took seconds on a few thousand items, and big maps got slower as they grew.	Both stay fast on tens of thousands of items.
`for i from 3 to 1 by -1` did not run.	A negative step counts down; a step of 0 is an error.
Whole numbers past 64 bits wrapped around silently: `9223372036854775807 plus 1` was a large negative number.	They keep every digit: `9223372036854775807 plus 1` is 9223372036854775808.
A function could not be written inside another function, and there were no functions without a name.	Inner functions and `function with x: ...` keep the values of the outer names they use (chapter 15).
Built-in functions such as `sqrt` could not be stored or passed.	They can.
Two files with a function of the same name: the first one silently won.	The program refuses to start and names the clash. `use "file" as name` keeps a file's names apart (chapter 20).
There was no way to read the words typed after the file name, the environment, JSON, patterns, dates, folders or the web.	`arguments()`, `env`, `to_json`/`from_json`, the `regex_` functions, `format_time` and `date_parts`, the folder functions, `run_command` and `http_get`/`http_post` (chapter 22).
With `--native`, a name that is also a word of C, such as `int`, `double` or `argc`, stopped the program from compiling.	Every name is given a prefix in the C code, so any name works.

New in 5.5.2: `try:` with `on error as name:` to catch an error, and `fail "message"` to raise one (chapter 21). Classes with fields, methods, inheritance from one or several parents, `new`, `call`/`tell`/`ask` and `is a` (chapter 43). Slices such as `text[0:3]` and `items[1:]` for text and lists, also written `slc(text, 0, 3)` (chapters 8 and 12). Using a word of the language as a name now says so, for example "'stop' is a word of the language and cannot be used as a name", and `@stop` makes it a name anyway (chapter 40). Functions are values that can be stored and passed to other functions (chapter 15). `global` lets a function change

a top-level name (chapter 16). `to_number` and `to_text` convert between text and numbers, and text is counted in characters rather than bytes (chapter 8). The built-in interpreter frees memory the program no longer uses.

Classes have these limits: there are no private fields and no shared (static) fields; fields are reached with `of self` inside a method, never by their bare name; parents must be defined before the class that inherits from them; and a class cannot be created inside a function or a block.

Fixed in 5.5.1

The first edition of this book listed these as faults of 5.5.0. They are fixed from 5.5.1, and the chapters mention each one where it applies.

Was	Now
`freeze` did not stop a later assignment.	Assigning to a frozen name is an error when the file is read.
`split` dropped empty pieces and treated each character of the separator as a separator.	It keeps empty pieces and matches the separator as a whole, which also repairs `string_replace_all` and `string_format_1/2/3`.
`arrange` and `arrange_list` in `lib_std` could not be called.	They are named `arrange_text` and `arrange_items`.
`set_remove` left a `null` in the set.	It removes the item.
`file_word_count` counted only space-separated chunks.	A line break also ends a word.
`enlngs` did not calculate `total * 2` and wrote invalid JavaScript for `show "a" b`.	It has a real expression parser, and several values in one `show` work in both modes.

Platform limits

- The preview windows of `enlngf` and `enlngm` use Windows features. On Linux and macOS, `enlngf build`, `enlngd compile` and `enlngs compile` still work because they only write files.
- The installers are not code-signed, so Windows SmartScreen or macOS Gatekeeper may warn you the first time you run them (chapter 2).

- Programs run through the built-in interpreter by default. For compute-heavy programs, `enlng run --native` or `enlng build` compile through a C compiler if one is installed. If none is, the tool says so and `enlng run` still works.

Speed

The numbers on the Enlangg website come from one Windows machine and one small program: starting and printing "hello" took about 12 milliseconds, against about 73 for a Python script doing the same. That says something about start-up time and nothing about speed on large problems.

For 5.5.2 a few larger programs were timed on one Windows machine, with the interpreter built as a 64-bit program, the way the release builds it, and compared with Python 3.13 on the same machine:

Program	Enlangg 5.5.1	Enlangg 5.5.2	Python
fib(30), recursive	0.24 s	0.20 s	0.08 s
2 million additions in a loop	0.06 s	0.07 s	0.04 s
A map with 50,000 keys	0.95 s	0.02 s	0.01 s
Sorting 50,000 numbers	2.42 s	0.02 s	0.005 s
3 million short texts made and dropped	0.55 s, 174 MB	0.84 s, 8 MB	0.23 s

The built-in interpreter is an ordinary interpreter: on most programs it is within a few times of Python, and slower than compiled C. Each figure is the best of three runs and includes starting the program. Measure your own program before you decide. A 32-bit build made with an old compiler (GCC 6.3) ran the same programs up to thirty times slower, so if your copy is much slower than these numbers, check how it was built.

How to report a problem

If a program in this book behaves differently on your machine, check `enlng --version` first. The outputs here were produced with 5.5.1. If the version matches and the output differs, that is worth reporting at the project's GitHub page, with the program and its output.

CHAPTER 40

Reference: Words of the Language

These are the words the reader of an Enlangg file treats as part of the language. They are reserved in the sense that matters: a few of them cannot be used as the name of a value or a parameter, and using one gives a syntax error that does not always say why. The list below was taken from the toolchain's own source.

```
action and arrange as ask at break by change continue count
decreases define discard display divided each else enlng equal every
false for forget freeze from function give greater if import in
increases input is it keep least len length less minus mod modulo most
multiplied needs none not nothing null of on or otherwise pair plus
print remember repeat return reverse show skip slc sort step stop swap
than that the times to true type until use when where while with
```

Not every word on that list refuses to be a name. A short experiment shows the difference: `count`, `first`, `last`, `item`, `list`, `text`, `name`, `value`, `result`, `key`, `start`, `end` and `next` all worked as names. `stop`, `times` and `from` did not:

PROGRAM

```
type enlng

remember count as 3
remember first as "a"
show count first
```

OUTPUT

```
3 a
```

PROGRAM WITH A MISTAKE

```
type enlng

remember times as 3
show times
```

WHAT enlng PRINTS

```
-- Enlang SyntaxError ---------------------------------------------
   -------
Location: program.enlng:3
```

```
    2 |
    3 | remember times as 3
      | ^
    4 | show times

What: Words such as plus, minus, times, divided, mod, and, or and
    not are words of the language and cannot be used as a name. Pi
    ck another name
Why:  The compiler encountered an unexpected token during syntax e
    valuation.

Suggestions:
  * Check statement syntax against Enlang language conventions.
----------------------------------------------------------------
    ------
```

From 5.5.2 the message says what went wrong, for a value, a function, a parameter or a loop name: "'stop' is a word of the language and cannot be used as a name". For the arithmetic and logic words, as above, it lists them: "Words such as plus, minus, times, ...". The safe habit is still to avoid every word in the list as a name (or to write it with `@`, described next), and to pick names that describe the data, such as `total`, `items`, `price`, `index`.

Using a word of the language as a name

From 5.5.2, writing `@` directly before a word makes it an ordinary name, even when it is a word of the language. `@times` is a name; `times` without `@` is still the operator, so both can appear on one line:

PROGRAM

```
type enlng

remember @times as 3
show 2 times @times

function @show with @stop:
    give @stop plus 1
show @show(4)
```

OUTPUT

```
6
5
```

Three rules:

- Write the `@` every time the name is used, not only where it is made. Without it the word keeps its normal meaning.

- `@` does not change what the word itself does. Defining `@show` makes a new function; `show` still prints.
- For a word that is already allowed as a name, the `@` makes no difference: `@total` and `total` are the same name.

`@` works for values, functions, parameters, loop names, fields and methods. A `@` that is not followed directly by a name, as in `@ 5`, is an error when the file is read.

What the words do

Group	Words	Used for
Making names	`remember`, `define`, `keep`, `freeze`, `forget`, `discard`, `as`	Create a value, mark it constant, or let it go.
Showing and asking	`show`, `print`, `display`, `ask`, `input`	Print values; read a line of text.
Deciding	`when`, `if`, `otherwise`, `else`, `is`, `not`, `and`, `or`	Choose between blocks.
Comparing	`equal`, `less`, `greater`, `least`, `most`, `than`, `at`	Words for `==`, `<`, `>`, `<=`, `>=`, written after `is`, as in `is at least`.
Arithmetic	`plus`, `minus`, `times`, `multiplied`, `divided`, `by`, `mod`, `modulo`	The same as `+ - * / %`.
Changing numbers	`increases`, `decreases`	`x increases by 1`, `x decreases by 2`.
Looping	`for`, `each`, `in`, `from`, `to`, `step`, `while`, `until`, `repeat`, `every`, `break`, `continue`, `stop`, `skip`	Repeat a block; leave or skip an iteration.
Functions	`function`, `action`, `with`, `give`, `return`, `needs`	Define and use named blocks of code.
Lists and text	`len`, `length`, `count`, `of`, `reverse`, `slc`, `sort`, `swap`, `pair`, `arrange`, `change`, `that`, `the`, `it`, `on`, `where`	Work with collections and their parts; several also connect phrases.
Files and modules	`use`, `import`, `type`	Bring in another file; declare a file's kind on its first line.
Plain values	`true`, `false`, `null`, `none`, `nothing`	The values that mean yes, no and absent.

From 5.5.2 a few more words take part in classes (chapter 43): `class`, `blueprint`, `model`, `inherits`, `extends`, `method`, `new`, `instantiate`, `call`, `tell`, `self`. They are not reserved. They are read as class words only in the exact shapes that chapter shows (for example `class Dog:` or `tell rex to speak`), so a program that uses `new` or `call` as an ordinary name keeps working. `try`, `error` and `fail` (chapter 21) work the same way.

Two of these deserve a note.

`stop` **and** `break` both leave the current loop. The cookbook uses `stop`.

`action` is another word for `function`. Library files use `action` and the rest of this book uses `function`; they behave the same way.

The type line

The first line of an Enlangg program is `type enlng`. The other languages of the toolchain start theirs with their own name: `type enlngdb` for a database script. The line tells the tool what it is reading. Chapter 17 explains what happens when it is missing.

Operators as symbols

Words	Symbol
`plus`	`+`
`minus`	`-`
`times`, `multiplied by`	`*`
`divided by`	`/`
`mod`, `modulo`	`%`
`is`, `is equal to`	`==`
`is not`	`!=`
`is less than`, `is greater than`	`<`, `>`
`is at most`, `is at least`	`<=`, `>=`

The words and the symbols can be mixed freely in one program, though one style per file reads best.

CHAPTER 41

Reference: Every Command

This chapter collects the command lines of all seven tools in one place. The help text below is the real output of each tool's own `--help` in toolchain 5.5.1, so it matches what you will see. After each one is a plain note on what the book has and has not confirmed.

enlng

TERMINAL

```
$ enlng --help
Enlang Sovereign Programming Language Compiler v5.6.0
Bare-Metal Native C99 Execution Engine (Zero Python, Zero Node.js)

Usage:
  enlng run <file.enlng>              Run immediately (built-in in
    terpreter, no C compiler needed)
  enlng run --native <file.enlng>     Compile with gcc, then run (
    faster for heavy programs)
  enlng build <file.enlng> -o <out>   Compile to standalone native
     executable (.exe)
  enlng add <folder|file|git url> [name]   Install a package into
    enlng_packages (then: use "name")
  enlng install                       Install every package listed
     in enlng_packages.txt
  enlng remove <name>                 Remove a package
  enlng packages                      List the installed packages
  enlng lsp                           Editor help over LSP: errors
    , hover, completion, go to definition
  enlng debug <file.enlng>            Step through a program, with
     breakpoints and values
  enlng emit-c <file.enlng>           Emit generated ANSI C99 sour
    ce code to stdout
  enlng <file.enlng>                  Direct run
  enlng                               Interactive mode: write line
    s one by one
  enlng test [file or folder]         Run every file ending in _te
    st.enlng
  enlng --version                     Display version information
```

Confirmed in this book: `run`, `build`, `emit-c`, `--version`, and `run --native`. Chapter 5 covers them.

enlngdb

TERMINAL

```
$ enlngdb --help
EnlngDB Pure C Native Database Engine v2.0.0-pure-c-native
Usage:
  enlngdb <script.enlngdb>       Execute natural English database
    script
  enlngdb -e "<query>"           Execute inline statement
  enlngdb --version              Display version information
```

Confirmed: running a script file, `-e`, and the statements listed in chapter 30.

enlngf

TERMINAL

```
$ enlngf --help
enlngf - Enlangg Sovereign Frontend & Web Studio Engine v5.3.0 (Pu
    re C Zero-Python)

Usage:
  enlngf <file.enlngf>                          Launch native Win32
     Desktop GUI Window
  enlngf run <file.enlngf>                      Launch native Win32
     Desktop GUI Window
  enlngf gui <file.enlngf>                      Launch native Win32
     Desktop GUI Window
  enlngf serve <file.enlngf> [--port <port>]   Serve live over emb
    edded WinSock2 HTTP server
  enlngf build <file.enlngf> [-o <out.html>]   Compile frontend ma
    rkup to standalone HTML5
  enlngf --version                              Display version inf
    ormation
```

Confirmed: `build`. Not shown, because they need a window or a browser: the bare form, `run`, `gui`, and `serve`.

enlngd

TERMINAL

```
$ enlngd --help
enlngd - Enlangg Sovereign Design Tokens & Style Engine v5.3.0 (Pu
    re C Zero-Python)

Usage:
  enlngd <file.enlngd>                          Inspect and resolve
     design tokens & styles
  enlngd inspect <file.enlngd>                  Display detailed de
    sign token table
  enlngd compile <file.enlngd> [-o <out.css>]  Export design token
    s to CSS3 (Web Target)
  enlngd --version                              Display version inf
```

```
    ormation
```

Confirmed: `inspect` and `compile`.

enlngs

TERMINAL

```
$ enlngs --help
enlngs - Enlangg Sovereign Reactive Script Engine v5.3.0 (Pure C Z
    ero-Python)

Usage:
  enlngs <file.enlngs>                          Execute natively on
     Sovereign Script Engine
  enlngs run <file.enlngs>                      Execute natively on
     Sovereign Script Engine
  enlngs compile <file.enlngs> [-o <out.js>]   Export reactive scr
    ipt to JavaScript (Web Target)
  enlngs -e "<script>"                          Execute inline scri
    pt statement
  enlngs --version                              Display version inf
    ormation
```

Confirmed: `-e`, running a file, and `compile`. See the cautions in chapter 32.

enlngm

TERMINAL

```
$ enlngm --help
enlngm - Enlangg Sovereign Mobile Native & HAL Engine v5.3.0 (Pure
     C Zero-Python)

Usage:
  enlngm <file.enlngm>                          Launch Native Smart
    phone Simulator Window
  enlngm run <file.enlngm>                      Launch Native Smart
    phone Simulator Window
  enlngm build <file.enlngm> [--target <apk|ipa>] [-o <out>] Build
     production package
  enlngm export <file.enlngm> [-o <out.dart>]  Export to Flutter/D
    art source code
  enlngm --version                              Display version inf
    ormation
```

Confirmed: `export`, and that `build --target apk` is not implemented. The window forms need a screen and were not run.

enlangg

TERMINAL

```
$ enlangg --help
=================================================================
    ===
  Enlangg Sovereign Toolchain CLI v5.0.0-sovereign-universal
  Pure C Zero-Python 7-in-1 Sovereign Ecosystem
=================================================================
    ===

Unified CLI Commands:
  enlangg run <file.ext>                      Run any file (.enlng
    , .enlngdb, .enlngs, .enlngf, .enlngd, .enlngm)
  enlangg run --aot <file.enlng>              Compile to native ma
    chine code & execute instantly
  enlangg compile <file.enlng> [-o <out.exe>] AOT compile natural
    English to native C machine code (.exe)
  enlangg emit-c <file.enlng> [-o <out.c>]    Emit clean ISO C99 s
    ource with Scoped Arena Memory

Domain Engines (Native Pure C):
  enlangg db <script.enlngdb>                 Execute conversation
    al microsecond database script
  enlangg db -e "<query>"                     Execute inline datab
    ase statement in C (<0.05ms)
  enlangg s <logic.enlngs>                    Execute script nativ
    ely on Pure C In-Memory VM
  enlangg s -e "<script>"                     Execute inline scrip
    t statement in C VM
  enlangg f <app.enlngf>                      Launch Pure C Native
     Win32 Desktop Window GUI
  enlangg f serve <app.enlngf> [--port 3000]  Serve live over Pure
     C WinSock2 HTTP Server
  enlangg f build <app.enlngf> [-o <out.html>] Compile frontend ma
    rkup to standalone HTML5
  enlangg d <theme.enlngd>                    Inspect and resolve
    design tokens & styles
  enlangg d compile <theme.enlngd> [-o <.css>] Export design token
    s to CSS3 stylesheet
  enlangg m <app.enlngm>                      Launch Pure C Native
     Smartphone Simulator (390x844)
  enlangg m build <app.enlngm> --target apk   Build mobile product
    ion package (HAL ARM64)
  enlangg build <app.enlngm> --target apk     Direct shortcut to m
    obile production compiler

Dedicated Standalone Executables in PATH:
  enlangg   Universal master toolchain dispatcher

  enlng     Core general-purpose computing with arena memory (.exe
    )
  enlngdb   Pure C conversational zero-SQL database engine (.edb f
    ormat)
  enlngs    Pure C sovereign in-memory script VM & interpreter
```

```
  enlngf    Pure C sovereign desktop window GUI & WinSock2 web stu
    dio
  enlngd    Pure C sovereign design tokens & style resolver
  enlngm    Pure C sovereign smartphone simulator & HAL packager

Documentation & Live Playground: https://enlangg.vercel.app
```

Confirmed: `run`, `fmt` with `--check` and `--write`, `pkg init`, and `pkg add`. The help text promises more than that, including `run --aot`, `compile`, and mobile builds. Read it as a list of command names, and use the earlier chapters for what each one really does.

Exit codes

A command that succeeds ends with code 0. One that finds an error ends with a code that is not 0. That is what lets a shell script stop on failure:

TERMINAL

```
$ enlng run good.enlng
fine
$ echo exit code %errorlevel%
exit code 0
```

On Linux or macOS the last line would be `echo "exit code $?"`.

Environment

The installers add the toolchain folder to your `PATH` so that the commands are found from any folder. Chapter 2 describes where the folder is and how to run the installer without changing the path. The standard library is read from the `lib` folder beside the programs, which is why `use "lib_math.enlng"` works from any directory.

CHAPTER 42

Glossary

Action — another word for a function in Enlangg. The library files use `action`; this book uses `function`.

Argument — a value you hand to a function when you call it. The names that receive the arguments are called parameters.

ASCII — the numbering of the basic English letters, digits and punctuation used by computers. The reason every capital letter sorts before every small letter.

Backend — the part of a compiler that does the final work. Enlangg has two: the built-in interpreter, which runs a program straight away, and the C backend, which writes C code for a C compiler to build into a native program.

Base case — in a recursive function, the case that is answered directly without calling itself. Without one, the function never stops.

Binary search — finding a value in a sorted list by repeatedly checking the middle and discarding the half that cannot hold it.

Block — the group of indented lines that belongs to a line ending in a colon, such as the body of a loop or a function.

Boolean — a value that is `true` or `false`.

Call — to run a function by writing its name and its arguments in parentheses.

Comment — a line that starts with `#`. The tool ignores it; it is for the human reader.

Compile — to translate a program into another form. `enlng build` compiles to a native program; `enlngd compile` turns a theme into CSS; `enlngs compile` turns a script into JavaScript.

Constant — a value that is not meant to change. `freeze` declares one, and from 5.5.1 the tool refuses to run a program that assigns to it again.

Deque — a list you can add to and take from at both ends. Short for "double-ended queue".

EnlngDB — the English-language database tool, and the name of its script language.

Floating point — a number with a fractional part, such as 2.5. Contrast with a whole number.

Function — a named block of code that can be called with arguments and can give a value back.

Graph — a set of points with connections between some pairs.

Interpreter — a program that reads another program and carries out its instructions directly. `enlng run` uses one.

Iteration — one pass through the body of a loop.

Key — the name used to look up a value in a map.

List — an ordered collection of values written in square brackets. Positions count from zero.

Map — a collection of key and value pairs written in braces. It has no order.

Module — a file of Enlangg code that other files can bring in with `use`.

Native — compiled to the machine's own instructions, as opposed to interpreted.

Null — the value that means "nothing here". Many library functions give it when asked for something that is not there.

Overflow — what happens when a number is too big for the space reserved for it. In Enlangg a whole number result that does not fit in 64 bits becomes a big whole number with every digit kept, so it never overflows.

Parameter — the name inside a function that receives an argument.

Priority queue — a collection that always gives you the smallest value next.

Queue — first in, first out.

Recursion — a function calling itself on a smaller version of the problem.

Scope — where a name can be seen. A name made inside a function or a block belongs to it.

Stack — last in, first out.

Standard library — the eleven Enlangg files in the `lib` folder, imported with `use`.

String — another word for text.

Token — the smallest piece of a program the tool recognises: a word, a number, a symbol. In `enlngd`, also a named value such as a colour.

Toolchain — the set of seven programs, `enlng`, `enlngdb`, `enlngf`, `enlngd`, `enlngs`, `enlngm` and `enlangg`, that ship together.

Trie — a tree of letters used to store words so that prefixes can be found quickly.

Truncate — to cut off the fractional part. Whole-number division truncates.

Variable — a name that holds a value which can change.

CHAPTER 43

Classes and Objects

From 5.5.2 Enlangg has classes. A class describes a kind of thing: which values it holds (its **fields**) and what it can do (its **methods**). An **object** is one thing of that kind. The words are the ones used everywhere else in the language: `remember` for a value, `of` to reach into an object, and `call`, `tell` and `ask` to make an object do something. There is no dot.

A first class

PROGRAM

```
type enlng

class Dog:
    remember name as "unknown"
    remember tricks as []
    method speak:
        show name of self "barks"
    method learn with trick:
        append(tricks of self, trick)
    method count_tricks:
        give count of tricks of self

remember rex as new Dog with name as "Rex"
tell rex to speak
tell rex to learn with "sit"
tell rex to learn with "roll"
show ask rex to count_tricks
```

OUTPUT

```
Rex barks
2
```

- `remember name as "unknown"` inside a class gives a field and its starting value. Every new object gets its own copy, so two dogs never share one list of tricks.
- `method speak:` defines something the object can do. Inside a method the word `self` is the object it was called on. Fields are reached with `of self`, never by their bare name.
- `new Dog` makes an object. `with name as "Rex"` sets fields by name; any field left out keeps its starting value.

Three ways to call a method

You write	Use it when
`tell rex to speak`	You only want the effect
`ask rex to count_tricks`	You want the answer back, as in `show ask rex to count_tricks`
`call count_tricks on rex`	The same as `ask`; it reads well as a statement or a value

Inputs come after `with`: `tell rex to learn with "sit"`, or several separated by commas. Both modes give the same answer, so pick the one that reads best.

Reading and changing a field

Use the same `of` forms as for any map:

PROGRAM

```
type enlng

class Dog:
    remember name as "unknown"
    remember age as 1

remember rex as new Dog with name as "Rex"
show name of rex
change name of rex to "Max"
age of rex increases by 1
show name of rex age of rex
```

OUTPUT

```
Rex
Max 2
```

Inheritance

A class can build on another with `inherits from` (or `extends`). It gets every field and method of the parent, and may replace a method by writing one with the same name.

PROGRAM

```
type enlng

class Animal:
    remember name as "unknown"
    method speak:
        show name of self "makes a sound"
```

```
class Dog inherits from Animal:
    method speak:
        show name of self "barks"

class Puppy inherits from Dog:
    remember age as 0

remember pets as [new Animal, new Dog with name as "Rex", new Pupp
    y with name as "Bit"]
for each pet in pets:
    tell pet to speak
```

OUTPUT

```
unknown makes a sound
Rex barks
Bit barks
```

The loop shows why this is useful: each object answers `speak` in its own way, whatever its kind. A parent's method can still be reached with `from`:

PROGRAM

```
type enlng

class Animal:
    method describe:
        give "an animal"

class Dog inherits from Animal:
    method describe:
        give "a dog, which is " + call describe from Animal on sel
    f

remember d as new Dog
show ask d to describe
```

OUTPUT

```
a dog, which is an animal
```

More than one parent

List the parents with commas. When two parents have a method of the same name, the one written first wins. If a parent is shared by two others, its methods still run only once.

PROGRAM

```
type enlng

class Machine:
    method describe:
        give "machine"

class Walker:
```

```
    method describe:
        give "walker"

class Robot inherits from Machine, Walker:
    remember name as "R2"

remember r as new Robot
show ask r to describe
```

OUTPUT

```
machine
```

Class names must be defined before a class that inherits from them. Parents whose order cannot be reconciled are refused with a message when the file is read.

Is it a Dog?

`is a` asks whether an object is of a class, or of a class that inherits from it:

PROGRAM

```
type enlng

class Animal:
    remember legs as 4
class Dog inherits from Animal:
    remember name as "x"

remember rex as new Dog
show rex is a Dog
show rex is a Animal
show 42 is a Dog
```

OUTPUT

```
true
true
false
```

When something goes wrong

Asking for a method that does not exist, or for the wrong number of inputs, is an error you can catch with `try` (chapter 21):

PROGRAM

```
type enlng

class Dog:
    method speak:
        show "woof"

remember rex as new Dog
```

```
try:
    tell rex to fly
on error as problem:
    show "caught:" problem
try:
    tell 42 to speak
on error as problem:
    show "caught:" problem
```

OUTPUT

```
caught: Class Dog has no method 'fly'
caught: This value is not an object, so it has no method 'speak'
```

Mistakes in the class itself, such as a parent that is not defined or a field name that does not exist in `with`, are found when the file is read.

Good to know

- Method names follow the rules for function names. Words the language already uses (`show`, `print`, `give`, `sort`, `reverse`) cannot be method names.
- A name such as `new`, `call`, `tell` or `class` still works as an ordinary value name in programs that do not use classes in that spot.
- `freeze` works on objects as it does on maps: a frozen object refuses every change, also inside methods.
- There are no private fields and no shared (static) fields. Every field can be read and changed with `of`.
- A class and a `try` work in the interpreter and with `--native`; the output is the same.

CHAPTER 44

Everyday Tools: Defaults, Text Templates and List Helpers

This chapter collects the features that make everyday programs shorter. Every example is run by the real tool when the book is built, so the output you see is what you get.

Default values for inputs

An input can have a default, written with `as` (or `=` in the bracket form). The default is used when the call leaves that input out. Inputs with a default must come last.

PROGRAM

```
type enlng

function greet with name, greeting as "Hello":
    give greeting + ", " + name

show greet("Asha")
show greet("Rao", "Hi")
```

OUTPUT

```
Hello, Asha
Hi, Rao
```

Giving `null` on purpose also picks the default. A call with too few or too many inputs stops with a clear message that names the range:

PROGRAM

```
type enlng

function add with x, y as 5:
    give x plus y

show add(1), add(1, 2)
try:
    show add()
on error as e:
    show e
```

OUTPUT

```
6 3
'add' needs 1 to 2 inputs but was given 0.
```

Anonymous functions can have defaults too: `function with a, b as 10: a times b`.

Text templates with f"..."

Put `f` in front of a string and write any expression inside curly braces. Write `{{` and `}}` for a literal brace.

PROGRAM

```
type enlng

remember name as "Asha"
remember age as 30
show f"Hi {name}, next year you are {age plus 1}!"
show f"{{braces}} and {[1, 2]}"
```

OUTPUT

```
Hi Asha, next year you are 31!
{braces} and [1, 2]
```

Number and text literals

Numbers can be written in hexadecimal (`0xFF`), binary (`0b1010`), with an exponent (`1e3`) and with underscores for reading (`1_000_000`). Text in triple quotes may span lines and contain double quotes.

PROGRAM

```
type enlng

show 0xFF, 0b1010, 1e3, 1_000_000
remember poem as """first line
second "quoted" line
"""
show poem
```

OUTPUT

```
255 10 1000 1000000
first line
second "quoted" line
```

A stray symbol such as `$`, `&` or `?` is now an error with a hint. It is no longer skipped silently.

List helpers

These functions are written in Enlng itself and are added to a program only when it uses them. A function you define with the same name always wins.

Function	What it does
`map(list, f)`	a new list with `f` applied to each item
`filter(list, f)`	the items for which `f` is true
`reduce(list, f, start)`	folds the list into one value
`any(list, f)`, `all(list, f)`	true if some or every item passes (`f` is optional)
`count_if(list, f)`, `find_if(list, f, missing)`	count or find by a test
`sum(list)`, `product(list)`	total and product
`range(n)`, `range(a, b)`, `range(a, b, step)`	a list of numbers, `b` not included
`zip(a, b)`, `enumerate(list, first)`	pairs of items, or of index and item
`first`, `last`, `take`, `drop`, `chunk`, `flatten`	pieces of lists
`sort_by(list, f)`, `min_by`, `max_by`	order or pick by a key
`shuffle(list)`, `random_choice(list)`	random order, random item

PROGRAM

```
type enlng

remember nums as [5, 3, 8, 1, 9, 2]
show map(nums, function with x: x times 2)
show filter(nums, function with x: x > 4)
show reduce(nums, function with a, b: a plus b, 0)
show range(10, 0, -3)
show zip([1, 2, 3], ["a", "b"])
show sort_by(["ccc", "a", "bb"], function with s: len(s))
show chunk([1, 2, 3, 4, 5], 2)
```

OUTPUT

```
[10, 6, 16, 2, 18, 4]
[5, 8, 9]
28
[10, 7, 4, 1]
[[1, a], [2, b]]
[a, bb, ccc]
[[1, 2], [3, 4], [5]]
```

More built-in functions

Besides the helpers above, the language now has built-ins for text (`upper`, `lower`, `trim`, `replace`, `starts_with`, `ends_with`, `pad_left`), numbers (`sin`, `cos`,

`sqrt`, `gcd`, `clamp`, `round_to`, the `bit_` family), data (`to_json`, `from_json`, `parse_csv`, `regex_find`, `regex_replace`), files and folders (`path_join`, `list_folder`, `make_folder`), time (`time_now`, `format_time`), hashing (`sha256`, `base64_encode`) and the web (`http_get`, `http_post`). Chapter 22 lists them all.

PROGRAM

```
type enlng

show upper("abc"), replace("a-b-c", "-", "+"), pad_left("5", 3, "0
    ")
show to_json({"n": 3, "t": ["a", "b"]})
show regex_find_all("\\d+", "order 66 and 7")
show sha256("abc")
```

OUTPUT

```
ABC a+b+c 005
{"n":3,"t":["a","b"]}
[66, 7]
ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

CHAPTER 45

Plain Sentences: Choosing, Taking Apart, Tests, Doing Things Together and the Network

Every sentence in this chapter has the usual Enlng shape. An action word comes first (`remember`, `show`, `when`, `check`, `send`). Connector words join the parts (`as`, `from`, `in`, `to`, `on`, `at`, `where`). Silent words (`the`, `that`, `it`) may be added and are skipped. Every example was run when the book was built.

Choosing by a value

Write `when ... is:` and then one value per line. Several values can share a line with `or`, and `otherwise` takes everything else:

PROGRAM

```
type enlng

for each color in ["red", "blue", "pink"]:
    when the color is:
        "red": show "stop"
        "green" or "blue": show "go"
        otherwise: show "wait"
```

OUTPUT

```
stop
go
wait
```

Picking one of two values

PROGRAM

```
type enlng

remember age as 20
remember the label as "adult" if the age is at least 18 otherwise
    "child"
show label
```

OUTPUT

```
adult
```

Is it in the list

`is in` is the other way round from `contains`. It works for lists, text and the keys of a map:

PROGRAM

```
type enlng

remember numbers as [1, 2, 3]
when 3 is in the numbers:
    show "3 is there"
when 9 is not in the numbers:
    show "9 is not there"
show "ell" is in "hello"
```

OUTPUT

```
3 is there
9 is not there
true
```

Power

A whole number to a whole power gives a whole number:

PROGRAM

```
type enlng

show 2 to the power of 10
show 5 squared
```

OUTPUT

```
1024
25
```

Taking a list or map apart

From a list, the names take the items in order. From a map, each name takes the value of the key with the same name:

PROGRAM

```
type enlng

remember x and y from [3, 4]
show x y
remember person as {"name": "Asha", "age": 30}
remember the name and the age from the person
show name age
for each who and score in [["Asha", 90], ["Rao", 75]]:
    show who score
for each key and value in {"tea": 5}:
    show key value
```

OUTPUT

```
3 4
Asha 30
Asha 90
Rao 75
tea 5
```

Making a list from a list

PROGRAM

```
type enlng

remember nums as [5, 12, 8, 20]
show every n in the nums where n is greater than 10
show n times 2 for every n in nums
show n plus 1 for every n in nums where n < 10
```

OUTPUT

```
[12, 20]
[10, 24, 16, 40]
[6, 9]
```

Giving inputs by name

Inputs given by name can come in any order, after the ones given in order. An input with a default can be left out:

PROGRAM

```
type enlng

function area with width, height:
    give width times height
show area with width as 3 and height as 4
show area(height as 2, width as 5)

function greet with name, greeting as "Hello":
    give greeting + ", " + name
show greet("Asha", greeting as "Hi")
```

OUTPUT

```
12
10
Hi, Asha
```

Checking the kind of a value

`is a number`, `is a whole number`, `is a decimal`, `is a text`, `is a list`, `is a map` and `is a function` give true or false. `check that ...` stops with a message when something does not hold:

PROGRAM

```
type enlng

function half with value:
    check that value is a number
    give value divided by 2
show half(8)
remember answers as ["eight"]
try:
    show half(answers[0])
on error as problem:
    show problem
```

OUTPUT

```
4
This check did not hold.
```

A check at the start of a function also works before the program runs. When a call gives the function a value whose kind is clear from the file alone, such as `half("eight")`, or a name remembered once as `"eight"`, nothing runs and the mistake is reported at once:

TERMINAL

```
-- Enlang KindError ----------------------------
Location: kinds.enlng:6

What: 'half' checks that its input 'value' is a number,
      but this gives it text.
```

Values that come from a list, a file, the user or a calculation are only known while the program runs, so the check catches them then, as above.

Tests

A test is a named group of checks. When a check compares with `is`, a failure says what was expected and what came out:

PROGRAM

```
type enlng

function add with a, b:
    give a plus b

test "adding works":
    check that add(1, 2) is 3
    check that add(-1, 1) is 0
```

OUTPUT

```
passed: adding works

1 passed, 0 failed
```

When a check fails, the test says so, and the program ends with an error code. With `check that add(2, 2) is 5`, the output is:

TERMINAL

```
FAILED: wrong on purpose
        Expected 5 but got 4

0 passed, 1 failed
```

`enlng test` runs every file ending in `_test.enlng` in the folder and the folders inside it. `enlng test file.enlng` runs one file. A file with a failed test ends with an error code, so other tools can see it.

Ending the program

PROGRAM

```
type enlng

for i from 1 to 5:
    when i is 3:
        stop the program
    show i
```

OUTPUT

```
1
2
```

Doing things at the same time

The lines inside `run together:` run at the same time, and the program goes on when all of them are done. The names they remember are there after the block. This saves time when the lines wait for something: a web page, the network, a command or the clock. Here three waits of one second take about one second in all:

PROGRAM

```
type enlng

function slow_square with n:
    sleep(1)
    give n times n

remember start as time_now()
run together:
    remember a as slow_square(3)
    remember b as slow_square(4)
    remember c as slow_square(5)
show a, b, c
```

```
show time_now() minus start is less than 3
```

OUTPUT

```
9 16 25
true
```

Each line must stand on its own: a line cannot use a name another line of the same block remembers. When a line stops with an error, the whole block gives that error, so `try` can catch it. Lines that only work out numbers or text do not get faster, because only one line runs Enlng at a time; the others go on while it waits. Lines that show something may show it in any order.

Files that are not text

`read bytes from` gives the bytes of any file as a list of numbers from 0 to 255. `write ... to` writes a list of numbers as bytes, and anything else as text:

PROGRAM

```
type enlng

write [72, 105] to "two_bytes.tmp"
show read bytes from "two_bytes.tmp"
delete_file("two_bytes.tmp")
```

OUTPUT

```
[72, 105]
```

A small web server

`serve` calls your function with each request: a map with `method`, `path` and `body`. Give back text, a map or list (sent as JSON), or a map with `status` and `body`:

```
type enlng

function answer with request:
    when the request["path"] is:
        "/": give "Hello from Enlng"
        "/json": give {"ok": true}
        otherwise: give {"status": 404, "body": "nothing here"}

serve answer on port 8080
```

Network connections

```
type enlng

remember the server as listen on port 8080
remember the client as accept from the server
remember the line as receive from the client
```

```
send "hello" to the client
close the client

remember the link as connect to "example.com" at port 80
```

`receive from` gives the text that has arrived so far, or empty text when the other side has closed the connection.

Interactive mode

Type `enlng` with no file to write lines one at a time. A line that is only a value shows the value. A line ending in `:` waits for the indented lines that follow it, until an empty line. `exit` leaves.

TERMINAL

```
enlng> remember x as 5
enlng> x times 2
10
enlng> exit
```

CHAPTER 46

Packages, the Debugger and Editor Help

Three tools go with the language. Packages let you share code between programs. The debugger runs a program one statement at a time. Editor help shows mistakes while you type. None of them add anything to the language itself, except that `use` now also accepts the name of a package.

Packages

A package is a folder with a file called `main.enlng` in it, or a single `.enlng` file. `enlng add` copies it into a folder called `enlng_packages` next to your program:

TERMINAL

```
$ enlng add ../shared/geometry
Installed 'geometry'. Use it with:  use "geometry"
$ enlng add ../shared/greeter.enlng
Installed 'greeter'. Use it with:  use "greeter"
```

The source can be a folder, a file, or a git address (`https://...`, `git@...` or `file://...`). A second word gives the package another name: `enlng add ../shared/geometry shapes`. Git addresses are fetched with the `git` program, which must be installed.

Then `use` takes the name, with no `.enlng` and no folder:

```
type enlng

use "geometry"
use "greeter"

show circle_area(2)
show hello("Asha")
```

Enlng looks for a file with that name first. When there is none, it looks for `enlng_packages/name.enlng`, then `enlng_packages/name/main.enlng`, in the folder of the program and in each folder above it. The folder named by the `ENLNG_PACKAGES` environment variable is looked at last. A package can use its own files with `use "helpers.enlng"`, and those are found next to its `main.enlng`.

Every package you add is written to `enlng_packages.txt`, one per line with the place it came from. Keep that file with your program. On another computer, or after deleting the folder, one command brings everything back:

TERMINAL

```
$ enlng install
Installed 'geometry'. Use it with:  use "geometry"
Installed 'greeter'. Use it with:  use "greeter"
2 packages installed, 0 failed.
```

`enlng packages` lists what is installed, and `enlng remove greeter` takes one away. A name may only have letters, digits, `_`, `-` and `.`. A folder without `main.enlng` is refused, so a package that cannot be used is never left behind.

A package is the code of someone else, and `use` runs it with the same power as your own code. Only add packages you trust.

The debugger

`enlng debug file.enlng` stops before the first statement and waits for a command:

TERMINAL

```
$ enlng debug total.enlng
Debugging total.enlng. Type help to see the commands.

->   7 | remember total as 0
(debug) next

->   8 | for each size in [2, 3]:
(debug) next

->   9 |     total = total plus area(size, 4)
(debug) step

->   4 |     remember result as width times height
(debug) print width plus 1
width plus 1 = 3  (a number)
(debug) where
  area (called from line 9)
  the top of the program
(debug) continue
20
```

Command	What it does
`step`, `s`	Run the next statement, going into the functions it calls
`next`, `n`	Run the next statement, going over the functions it calls

Command	What it does
`out`, `o`	Run until the current function gives back its value
`continue`, `c`	Run until a breakpoint, or the end
`break 12`, `break area`	Stop at line 12, or at the start of the function `area`
`breaks`, `delete 12`	List the breakpoints, or remove the one at line 12
`print total`, `p`	Show the value of a name or of any expression, with its kind
`vars`	Show every name and value here, and the ones at the top of the program
`where`, `w`	Show the calls that led here
`list`, `l`	Show the lines around this one
`quit`, `q`	End the program

`enlng debug file.enlng --break 12` starts with a breakpoint already set. A mistake in `print` is only reported; the debugger goes on. When there are no more commands to read (the input ended), the program simply runs to the end.

Functions from other files, such as a `use`d file or a package, are run through without stopping, because their lines are not lines of the file on the screen. The debugger reads its commands from the same keyboard as `ask`, so a program that asks for input shares it. It works with the built-in interpreter only, not with `--native`. Only the main flow of the program is followed: the lines of a `run together:` block run without stopping.

Editor help

`enlng lsp` is a small server for the Language Server Protocol. An editor starts it and talks to it; you do not run it yourself. It gives:

- **Mistakes while you type.** They are the same ones `enlng run` reports, including a call that gives a function the wrong kind of value. The file is read again when you press Enter or save, not on every key.
- **Hover.** On a word of the language it explains it. On a function or class of your file it shows the line that makes it.
- **Finishing words.** Names from your file, built-in functions and the words of the language.

- **Go to definition** for functions, classes and names made at the top of the file, and **the outline** of the file.

To use it in an editor that supports LSP, set the server command to `enlng lsp` for files ending in `.enlng`. In Neovim, for example:

TERMINAL

```
vim.api.nvim_create_autocmd("FileType", {
  pattern = "enlng",
  callback = function()
    vim.lsp.start({ name = "enlng", cmd = { "enlng", "lsp" } })
  end,
})
```

The server reads a file's own lines to find names, so it knows the functions and names written in that file, not those of a `use`d file or package. The VS Code extension in `vscode-enlang` does not use this server yet.